

УНИВЕРЗИТЕТ У БЕОГРАДУ
ФАКУЛТЕТ ОРГАНИЗАЦИОНИХ НАУКА

ЗАВРШНИ РАД

**Софтверски систем за изнајмљивање
аутомобила у Jakarta EE окружењу**

Ментор

**Др Милош Милић,
Ванредни професор**

Студент

Димитрије Митић 2020/0492

Београд, 2025. године

САДРЖАЈ

1. Увод	1
2. Преглед коришћених технологија	3
2.1. Java	3
2.2. JАva Spring boot.....	5
2.3. MySQL.....	8
2.4. TypeScript.....	8
2.5. Angular.....	9
2.6. REST	10
3. Студијски пример.....	12
3.1. Прикупљање корисничких захтева	14
3.1.1. Вербални опис	14
3.1.2. Случајеви коришћења.....	15
3.2. Анализа.....	28
3.2.1. Понашање софтверског система - Системски дијаграми секвенци.....	28
3.2.2. Понашање софтверског система - уговори.....	56
3.2.3. Структура софтверског система.....	59

3.3.	Пројектовање	66
3.3.1.	Пројектовање корисничког интерфејса	67
3.3.2.	Пројектовање апликационе логике	100
3.3.4.	Пројектовање брокера базе података	121
3.3.5.	Пројектовање складишта података	123
3.3.7.	Коначна архитектура софтверског система	126
3.4.	Имплементација	127
4.	Тестирање.....	141
5.	Закључак.....	143
6.	Литература.....	145

1. УВОД

Потреба за превозом стара је готово колико и човечанство. Од најранијих дана, људи су тражили начине да лакше пређу велике удаљености и да превезу терет. Прво средство за превоз биле су животиње, најчешће коњи, затим су настала запрежна кола која су дуг период била основно средство транспорта[1]. Са наглим растом индустрије и технолошким напретком у XIX веку, јавила се идеја да се замени животињска снага механичком. Први технички успешан аутомобил настао је крајем XIX века, док је први успешан масовно произведен аутомобил *Ford Model T*. [2] Од првог појављивања на тржишту, аутомобили су постали симбол престижа и статуса и поседовање аутомобила је представљало луксуз. Аутомобилска индустрија је непрестано расла, аутомобили су се усавршавали, постајали су бржи, поузданији и постали су приступачнији човечанству. Уследило је прилагођавање саобраћајне инфраструктуре аутомобилима, и временом су постали неопходни за свакодневни живот. [3]

Паралелно са развојем аутомобила, развијала се и идеја о њиховом изнајмљивању. Још у античком периоду постојала је пракса изнајмљивања коња и кочија, у старом Риму, кочије су се могле изнајмити за превоз људи и робе[4]. Тај концепт пословања временом је прерастао у услугу изнајмљивања возила. Прва компанија за изнајмљивање моторних возила настала је почетком XX века у Омахи, држава Небраска, у Сједињеним Америчким Државама, а убрзо након тога су настале и друге велике фирме.[5]

У модерном добу, са развојем туризма, све динамичнијим начином живота и повећаном потребом за мобилношћу, изнајмљивање возила је постало саставни део транспортне и туристичке инфраструктуре. Међутим, традиционални процес изнајмљивања возила често подразумева обимну папирологију, ручно потписивање уговора, вишеструке телефонске позиве и физички одлазак у пословницу. То не само да одузима време, већ повећава и ризик од грешака и неусклађености у комуникацији између клијената и агенција.

Због тога се јавља потреба за дигиталним решењем које ће омогућити једноставније, брже и ефикасније управљање процесом резервације возила. Веб апликација за изнајмљивање возила је осмишљена тако да модернизује и унапреди начин на који

корисници приступају услузи изнајмљивања возила. Систем омогућава унос датума преузимања и враћања возила, избор локације, преглед доступних возила са свим релевантним подацима као што су цена, тип возила и стање, као и једноставну резервацију.

За развој ове веб апликације примењена је упрошћена Ларманова метода, која комбинује итеративни развој и архитектуру засновану на нивоима, омогућавајући флексибилан и робустан развојни процес[6]. Животни циклус система, према овој методи, обухвата пет кључних фаза[6]:

1. Прикупљање корисничких захтева
2. Анализа
3. Пројектовање
4. Имплементација
5. Тестирање

Друго поглавље овог рада даје преглед технологија коришћених у имплементацији. Детаљно је обрађена серверска страна апликације, развијена у програмском језику *Java* уз помоћ *Spring Boot* развојног оквира. Такође, описан је *SQL*, језик за рад са подацима, као и *MySQL*, систем за управљање релационим базама података. Клијентска страна апликације реализована је коришћењем *TypeScript* програмског језика и *Angular* развојног оквира.

Треће поглавље приказује практичну примену поменутих пет фаза упрошћене Ларманове методе на конкретном примеру система за изнајмљивање возила.

Четврто поглавље доноси закључна разматрања и предлаже потенцијалне правце за даље унапређење и проширење функционалности система. Пето поглавље садржи списак литературе и извора коришћених приликом израде овог рада.

2. ПРЕГЛЕД КОРИШЋЕНИХ ТЕХНОЛОГИЈА

У овом сегменту рада представљене су технологије које су коришћене за израду софтверског система за изнајмљивање возила. Серверска логика и комуникација са базом података имплементирани су у програмском језику *Java*, користећи *Spring Boot* развојни оквир. За развој клијентског дела апликације, изабран је *TypeScript* у комбинацији са *Angular* развојним оквиром.

Апликација је пројектована као SPA (енгл. *Single Page Application*). SPA је тип апликације која учитава само једну страницу и динамички мења њен садржај без поновног освежавања целе странице[7]. Овај приступ омогућава брже и глаткије корисничко искуство, јер се комуникација са сервером одвија асинхроно путем API (енгл. *Application Programming Interface*) позива који доприноси бољем корисничком искуству кроз бржу навигацију, смањено оптерећење сервера и ефикасније коришћење мрежних ресурса[8].

2.1.JAVA

Java је објектно-оријентисан програмски језик вишег нивоа, оригинално развијен од стране компаније „*Sun Microsystems*“ која је данас у власништву компаније „*Oracle*“ [9]. Java је међу најпопуларнијим језицима на свету, намењен изради различитих врста апликација, од мобилних апликација преко стандардних десктоп апликација (енгл. *Desktop application*) и система до корпоративних решења [10]. Јава представља снажан и општенаменски програмски језик који је познат по својој стабилности, безбедности и преносивости.

Основу Јава окружења чине три кључне целине[9]:

1. Језик Јава омогућава програмерима да пишу изворни код у облику класа и објеката. То је објектно-оријентисани језик који је релативно лак за учење и читање, а истовремено подржава концепте који су потребни за израду сложених пословних апликација.

2. Јава виртуелна машина *JVM* (енгл. *Java Virtual Machine – JVM*) представља окружење за извршавање Јава програма. *JVM* делује као апстрактни слој између програмског кода и оперативног система, омогућава да се Јава програми извршавају на различитим оперативним системима без промене кода.
3. Јава екосистем обухвата широк скуп библиотека, алата и оквира који омогућавају бржи развој и интеграцију апликација са различитим технологијама. Ово укључује библиотеке за рад са базама података, веб серверима, графичким интерфејсом и безбедносним протоколима.

Јава је позната по једноставности, преносивости, сигурности платформе, а прати и принцип „Напиши једном, покрећи свуда“ (енгл. *Write Once, Run Anywhere*). Неке од особина Јаве су [10]:

1. Објектно-оријентисан приступ - Јава користи концепт класа и објеката, омогућавајући програмерима да моделују реалне ентитете и њихове међусобне односе. Подршка за наслеђивање, полиморфизам и учурење (енгл. *Encapsulation*) олакшава организовање и одржавање сложених апликација [11].
2. Преносивост (енгл. *portability*) - Јава апликације се компајлирају у бајткод (енгл. *bytecode*), који се извршава на Јава виртуелној машини – *JVM*. Ово омогућава да исти програм ради на различитим оперативним системима без измена у коду, што је чини веома погодном за развој мултиплатформских решења [12].
3. Сигурност - Јава нуди механизме за заштиту од неовлашћеног приступа меморији и штетног кода. Контроле приступа, аутоматско управљање меморијом и проверена извршна средина доприносе високим безбедносним стандардама [12].
4. Аутоматско управљање меморијом (енгл. *garbage collector*) - Јава користи аутоматски систем који ослобађа непотребну меморију, чиме програмер није принуђен да ручно управља меморијом и смањује могућност грешака као што је неослобађање меморије (енгл. *Memory Leak*) [12].

5. Велика стандардна библиотека (енгл. *standard library*) - Јава нуди обимну библиотеку програмских класа и интерфејса која покрива различите функционалности. Унутар ове библиотеке налазе се пакети за рад са датотекама, мрежну комуникацију, базе података, графички интерфејс, криптографију и друге. Користећи ову стандардну библиотеку, програмери могу брже и ефикасније да развијају апликације, јер већина потребних функционалности није потребно имплементирати од нуле [9].
6. Вишенитност и конкурентност (енгл. *multithreading and concurrency*) - Јава подржава извршавање више нити (енгл. *threads*) конкурентно унутар апликације. Ово омогућава ефикасно коришћење процесорских ресурса и повећава продуктивност апликација које захтевају конкурентно обављање више задатака. Вишенитност је нарочито важан у развоју апликација које захтевају брзу реакцију на корисничке захтеве и обраду великог обима података [12].
7. Интерпретација и динамичка везаност - Јава дозвољава динамичко повезивање кода током извршавања, што олакшава модуларни развој и коришћење спољних библиотека без потребе за поновном компилацијом целе апликације [10].

2.2.JAVA SPRING BOOT

Spring Boot развојни оквир је један од најпознатијих и најчешће коришћених Јава оквира (енгл. *framework*) за развој веб и пословних апликација. Он је модеран развојни оквир отвореног кода (енгл. *Open source*) који одржава компанија „Pivotal“, заснован је на програмском језику Java, који омогућава брз и ефикасан развој веб апликација, пословних система и микросервиса. *Spring Boot* је заснован на *Spring* оквиру, али има знатно једноставнију употребу и конфигурацију [13].

Spring Boot уводи механизам аутоматске конфигурације (енгл. *Auto-Configuration*), уграђени веб сервер и приступ заснован на унапред дефинисаним претпоставкама (енгл. *opinionated approach*). Овај приступ подразумева да развојни оквир, на основу присутних зависности у пројекту, самостално доноси одговарајуће одлуке о подразумеваним подешавањима. На пример, ако се у пројекат укључи зависност за рад са *MySQL* базом

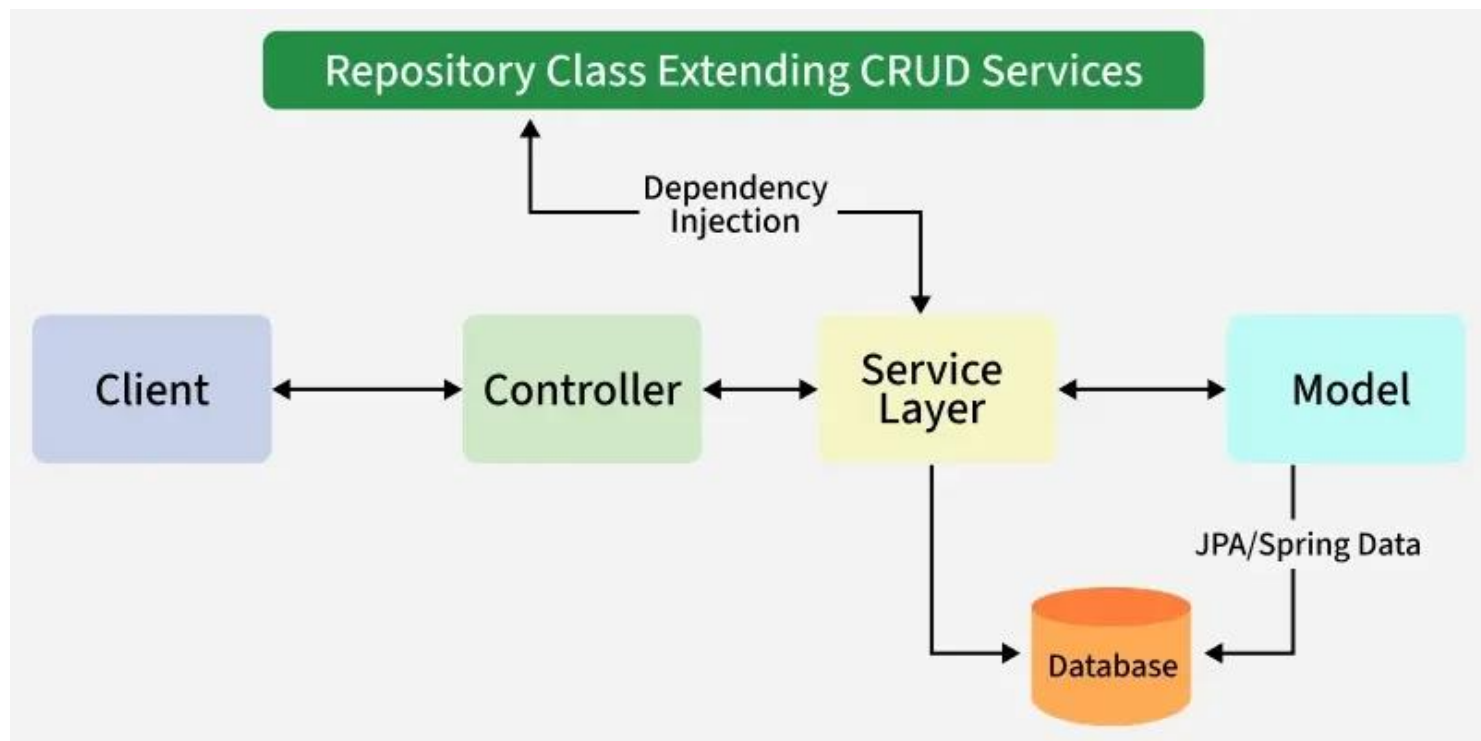
података, *Spring Boot* ће аутоматски конфигурисати све што је потребно за успостављање конекције и приступ подацима. На тај начин, програмер је ослобођен сложених поставки и може да се у потпуности посвети имплементацији пословне логике [14].

Једна од кључних предности овог оквира јесте могућност креирања самосталних (енгл. *standalone*) апликација које се покрећу без потребе за екстерним апликационим сервером [15]. Поред тога, његова модерна архитектура омогућава развој апликација које се извршавају у *cloud* окружењу, лако се интегришу са платформама као што су *Docker*, *Kubernetes*, *AWS* и друге [16].

Spring Boot апликација се састоји из више логичких слојева, који заједно чине целокупну архитектуру система[13]:

1. Презентациони слој (енгл. *Client layer*) представља спољни систем или корисника који шаље захтеве апликацији. Управља захтевима, аутентификацијом и конверзијом *JSON* података у објекте. У овом слоју се налази *frontend* логика апликације.
2. Контролер слој (енгл. *Controller layer*) прима захтеве од клијента, обрађује их и шаље одговор. Прослеђује обраду пословној логици сервисног слоја.
3. Пословни слој (енгл. *Service layer*) садржи сву пословну логику и сервисне класе. Комуницира са репозиторијумом ради добијања или ажурирања података. Користи механизам убацивања зависности (енгл. *Dependency Injection*) за приступ потребним сервисима.
4. Перзистентни слој (енгл. *Repository layer*) обавља *CRUD* (енгл. *create, read, update, delete* - креирање, читање, ажурирање, брисање) операције над базом података. Проширење је *Spring Data JPA* или других механизма за перзистенцију.
5. Модел слој (енгл. *Model Layer*) представља ентитете базе података и доменске моделе. Мапира се на табеле у бази података помоћу *JPA/Spring Data*.
6. Слој базе података (енгл. *Database layer*) је физичка база података у којој се чувају сви подаци. *Spring Boot* комуницира са њом кроз *JPA/Spring Data* компоненте.

Архитектура апликације и односи између наведених слојева приказани су на следећој слици:



Слика 1 Архитектура Java Boot Spring пројекта [13]

Свеукупни ток захтева у *Spring Boot* апликацији изгледа овако:

Корисник шаље *HTTP* захтев, који се у контролеру обрађује и прослеђује пословном слоју. Сервисна логика затим користи репозиторијум за приступ бази података, а резултат се враћа назад ка клијенту у облику одговора [13].

2.3.MYSQL

SQL је структурирани упитни језик (енгл. *Structured Query Language*) који се користи за рад са базама података. Његова основна улога је управљање подацима - омогућава да се подаци уносе, мењају, бришу и приказују, као и дефинисање структуре самих база. *SQL* се користи у готово свим савременим системима за управљање базама података, укључујући и *MySQL*. Уз помоћ *SQL* упита могу се правити једноставни или сложени упити, повезивати табеле и обезбедити тачност и поузданост података [17].

MySQL је један од најпопуларнијих система за управљање релационим базама података (енгл. *Relational Database Management System - RDBMS*), заснован на *SQL* језику. Развијен је од стране компаније „*MySQL AB*“, а данас је у власништву корпорације „*Oracle*“. *MySQL* је познат по својој поузданости, брзини, једноставности употребе и отвореном коду, због чега се широко користи у веб апликацијама и серверским решењима [18].

Основна сврха *MySQL* система је управљање подацима у табелама које су међусобно повезане преко кључева. Свака табела представља скуп података о одређеним ентитетима, а односи између табела омогућавају ефикасну организацију и повезивање тих података. Употребом *SQL* упита, омогућено је извршавање сложених анализа, лак приступ подацима или ажурирање података у бази [18].

2.4.TYPESCRIPT

TypeScript је програмски језик отвореног кода развијен од стране „*Microsoft*“ компаније, који представља строги надскуп *JavaScript* језика. Његова основна сврха је увођење опционог система статичке типизације, што омогућава развојним тимовима да пишу чистији и робуснији код. Поред статичке типизације, *TypeScript* уводи и додатне функционалности као што су интерфејси (енгл. *interfaces*), енумерације, генерички типови и подршка за модуларну архитектуру. Ове особине доприносе бољој организацији кода, лакшем одржавању и већој скалабилности великих апликација. Провера типова током компајлирања помаже у раном откривању грешака

које би се у стандардном *JavaScript* језику појавиле тек током извршавања. Због ових карактеристика, *TypeScript* је постао индустријски стандард за развој комплексних веб апликација [19].

2.5.ANGULAR

Angular је платформа отвореног кода (енгл. *Open-source*) за развој апликација, коју одржава тим из компаније *Google* и заједница програмера. Заснован на *TypeScript* програмском језику, овај развојни оквир намењен је креирању динамичних и интерактивних једностраничних апликација (енгл. *Single Page Applications - SPA*) кроз структуриран, компонентни приступ. *TypeScript* програмски језик омогућава коришћење статичке типизације, класа и интерфејса (енгл. *interfaces*), као и модуларног приступа у развоју апликација. *Angular* није само библиотека, већ комплетан екосистем који пружа решења за све аспекте развоја, укључујући рутирање, управљање формама, клијент-сервер комуникацију и тестирање [20].

Основни концепти *Angular* развојног оквира су следећи [20]:

1. Компоненте (енгл. *Components*) представљају градивне блокове апликације. Свака компонента обухвата *TypeScript* класу која дефинише логику, *HTML template* који дефинише приказ и *CSS* за стилизовање. Компоненте промовишу поновну употребу кода и омогућавају креирање сложених корисничких интерфејса кроз састављање мањих, самосталних јединица.
2. Модули (енгл. *Modules*) групишу компоненте, директиве, сервисе и друге делове апликације. Они омогућавају бољу организацију кода, подршку за одложено учитавање (енгл. *lazy loading*) и развој заснован на функцијама.
3. Шаблони (енгл. *Templates*) су *HTML* структуре које дефинишу изглед компоненти. Они користе *Angular* синтаксу, као што су интерполација и директиве, како би повезали податке из *TypeScript* класе са елементима у моделу докумената (енгл. *Document Object Model - DOM*), омогућавајући динамичке и интерактивне приказе.

4. Директиве (енгл. *Directives*) проширују функционалност *HTML* елемената. Постоје структурне директиве, као што су *ngIf* и *ngFor*, које мењају *DOM* структуру додавањем или уклањањем елемената, и атрибутне директиве, као што су *ngModel* и *ngClass*, које мењају понашање или стил елемената.
5. Сервиси (енгл. *Services*) омогућавају коришћење истих података и логике у више компоненти. Користе се за слање захтева ка серверу, дељење података између компоненти и избегавање понављања кода (енгл. *Don't Repeat Yourself - DRY*).
6. Убацивање зависности (енгл. *Dependency Injection - DI*) је принцип пројектовања софтвера који омогућава да компоненте и сервиси добију потребне зависности, попут других сервиса или података, без да их сами стварају. Ово олакшава повезивање различитих делова апликације, чини код модуларнијим, лакшим за тестирање и одржавање.
7. Усмеравање (енгл. *Routing*) - омогућава навигацију између различитих приказа апликације без учитавања целе странице. Путање се дефинишу као *URL* адресе који се повезују са специфичним компонентама. *Angular* рутер подржава одложено учитавање модула (енгл. *Lazy loading*), заштиту руте (енгл. *Route guards*) и угнежђене руте.
8. Форме (енгл. *Forms*) - нуде алате за креирање и валидацију корисничких уноса. *Angular* подржава форме управљане кроз шаблон (енгл. *template-driven forms*) и реактивне форме (енгл. *Reactive forms*), омогућавајући динамичко управљање пољима, проверу валидности и слање података ка серверу.

2.6.REST

Representational State Transfer (REST) није технологија или језик, већ представља архитектонски стил, односно скуп правила и принципа који дефинишу како клијент и сервер треба да комуницирају путем *HTTP* протокола. Уместо комплексних протокола, *REST* користи снагу самог *HTTP* протокола. Комуникација је заснована на идеји ресурса. Сваки део податка у систему представља ресурс којем се приступа путем јединствене адресе (*URL* адресе), као што је */api/cars/5* [21].

За манипулацију тим ресурсима користе се стандардне *HTTP* методе [21]:

- *GET* за добављање података.
- *POST* за креирање новог ресурса.
- *PUT* или *PATCH* за ажурирање постојећег ресурса.
- *DELETE* за брисање ресурса.

Подаци се најчешће размењују у *JSON* формату, који је лако читљив и једноставан за обраду на различитим платформама, што омогућава да серверски програм израђен коришћењем *Java* технологија и клијентски програм израђен у *Angular* развојном оквиру размењују податке. Један од најважнијих принципа је и непостојање стања (енгл. *statelessness*), где сваки захтев од клијента мора садржати све податке потребне за обраду, што чини систем скалабилнијим и поузданијим [21].

3. СТУДИЈСКИ ПРИМЕР

У овом поглављу се налази детаљно објашњење упрошћене Ларманове методе коришћене за израду овог софтверског система. Упрошћена Ларманова метода се базира на комбинацији стратегије управљања према случајевима коришћења и објектно-оријентисане методе пројектовања софтвера [6].

Целокупан животни циклус развоја софтвера, према овој методи, структуриран је кроз пет узастопних, логички повезаних фаза [6]:

1. Прикупљање захтева од корисника
2. Анализа
3. Пројектовање
4. Имплементација
5. Тестирање

Прва фаза развоја је прикупљање захтева, где се идентификују и документују сва својства и услови које систем мора да задовољи. Ови захтеви се деле у две основне групе. Функционални захтеви специфицирају конкретне акције и функције које систем мора да обавља, а описују се кроз вербални модел и модел случајева коришћења. Са друге стране, нефункционални захтеви дефинишу атрибуте квалитета, као што су перформансе, поузданост, сигурност и употребљивост [6].

Након дефинисања захтева, у фази анализе се креира апстрактни модел који детаљно описује пословну логику система, без улажења у технолошке детаље имплементације. Структура система се у овој фази формализује кроз израду концептуалног (доменског) и релационог модела, док се динамичко понашање система и интеракције између његових делова моделују помоћу системских секвенчних дијаграма и системских операција [6].

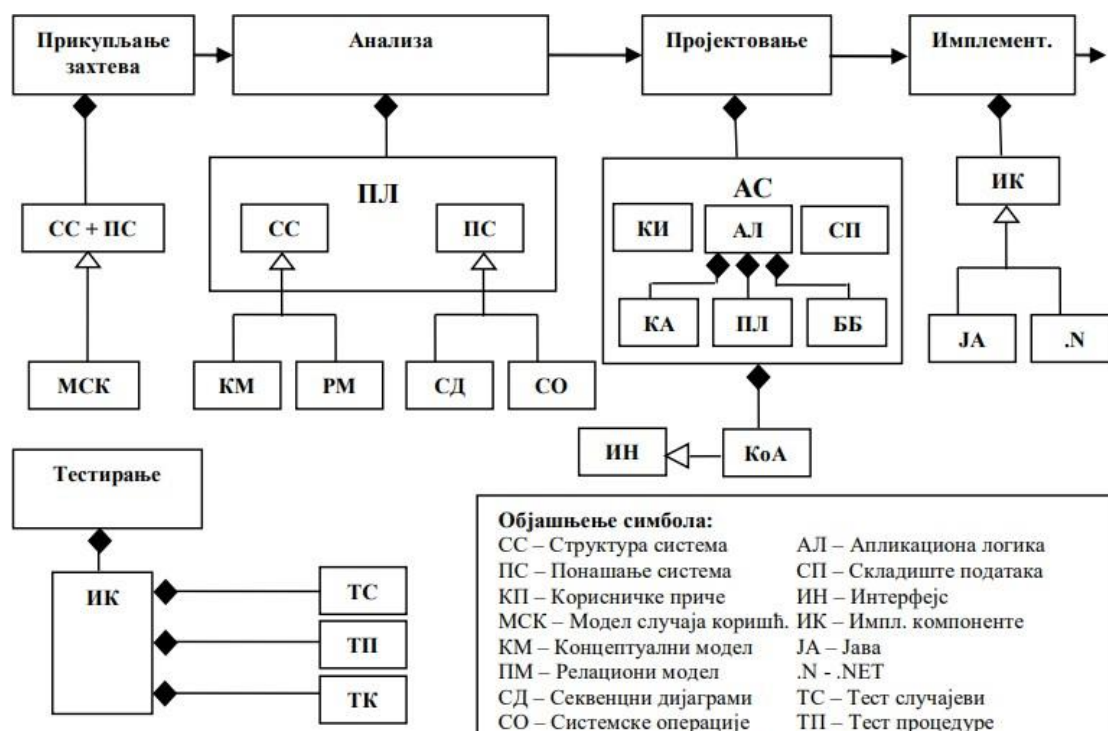
У фази пројектовања се дефинише тронивојска софтверска архитектура, која се састоји од презентационог нивоа (корисничког интерфејса), нивоа апликативне логике и нивоа за складиштење података. Апликациона логика је додатно подељена на компоненте: контролер, који управља током извршења захтева, пословну логику, која је преузета из фазе анализе и брокер базе података, који служи као посредник ка складишту. Сваки

ниво се реализује кроз скуп јасно дефинисаних софтверских компоненти [6].

У фази имплементације развијају се све компоненте система користећи одабране технологије, у овом случају *Javu* и *Spring Boot*. Ова фаза обухвата писање функционалног кода за сваки модул, као и повезивање различитих компоненти у целину која функционише у складу са пројектованом архитектуром. Поред тога, ова фаза укључује и проверу међусобне комуникације модула, управљање зависностима и осигуравање да сви делови система правилно раде заједно, чиме се гарантује стабилност и поузданост апликације.

Финална фаза животног циклуса је тестирање, чији је циљ верификација и валидација софтвера, односно провера да ли систем ради у складу са дефинисаним захтевима. Тестирање се спроводи на више нивоа: прво се врши јединично тестирање компоненти (енгл. *unit testing*) кроз израду тест случајева и процедура, а затим и интеграционо тестирање, којим се проверава да ли све компоненте исправно функционишу заједно као јединствен систем [6].

Целокупан процес развоја, са међусобним везама између фаза и артефаката који у њима настају, визуелно је представљен на следећој слици [6].



Слика 2 Процес развоја софтверског система [6]

3.1.ПРИКУПЉАЊЕ КОРИСНИЧКИХ ЗАХТЕВА

Прикупљање корисничких захтева је прва фаза развоја софтвера. У њој је потребно детаљно утврдити на који начин систем на крају израде треба да функционише. Од великог је значаја јасно и недвосмислено дефинисати захтеве да би се у даљем процесу израде софтвера заобишли неспоразуми и самим тим смањио трошак и време израде. Фаза прикупљања захтева се документује кроз моделе случајева коришћења (СК), који се заснивају на спецификацијама кључних процеса. Ови модели детаљно описују интеракције корисника са системом, пружајући јасну слику о томе како ће се софтвер користити у стварним условима [6].

3.1.1. Вербални опис

Основни задатак је развој софтверског система за дигитализацију пословања *Rent-a-car* агенције. Систем мора да подржи две кључне улоге са јасно раздвојеним функционалностима: клијента и администратора.

За клијента, апликација треба да пружи модеран и интуитиван кориснички интерфејс који омогућава једноставан процес изнајмљивања возила. Примарна функционалност је преглед целокупне флоте доступних возила, уз могућност динамичког филтрирања по категоријама. Корисник мора бити у могућности да изабере жељени период преузимања и враћања возила, након чега је систем дужан да транспарентно израчуна и прикаже укупну цену за свако возило у понуди. Након регистрације и пријаве на систем, корисник може да пошаље формални захтев за резервацију за одабрано возило и период. Поред тога, систем мора да обезбеди и секцију где корисник може да прати статус својих послатих захтева.

За администратора, систем треба да представља свеобухватну административну алатку за управљање свим аспектима пословања. То укључује модул за комплетно управљање флотом, са могућношћу додавања потпуно нових модела возила као и додавање, ажурирање и брисање конкретних возила унутар тих модела. Администратор такође мора имати увид у систем свих регистрованих клијената. Централна функционалност административног дела је систем за обраду захтева, где администратор може да прегледа

и управља свим пристиглим захтевима за изнајмљивање. Администратор може да одобри захтев, чиме се захтев претвара у званичан уговор, а статус возила мења у изнајмљено или одбије што поништава захтев и статус возила остаје исти, односно доступан.

3.1.2. Случајеви коришћења

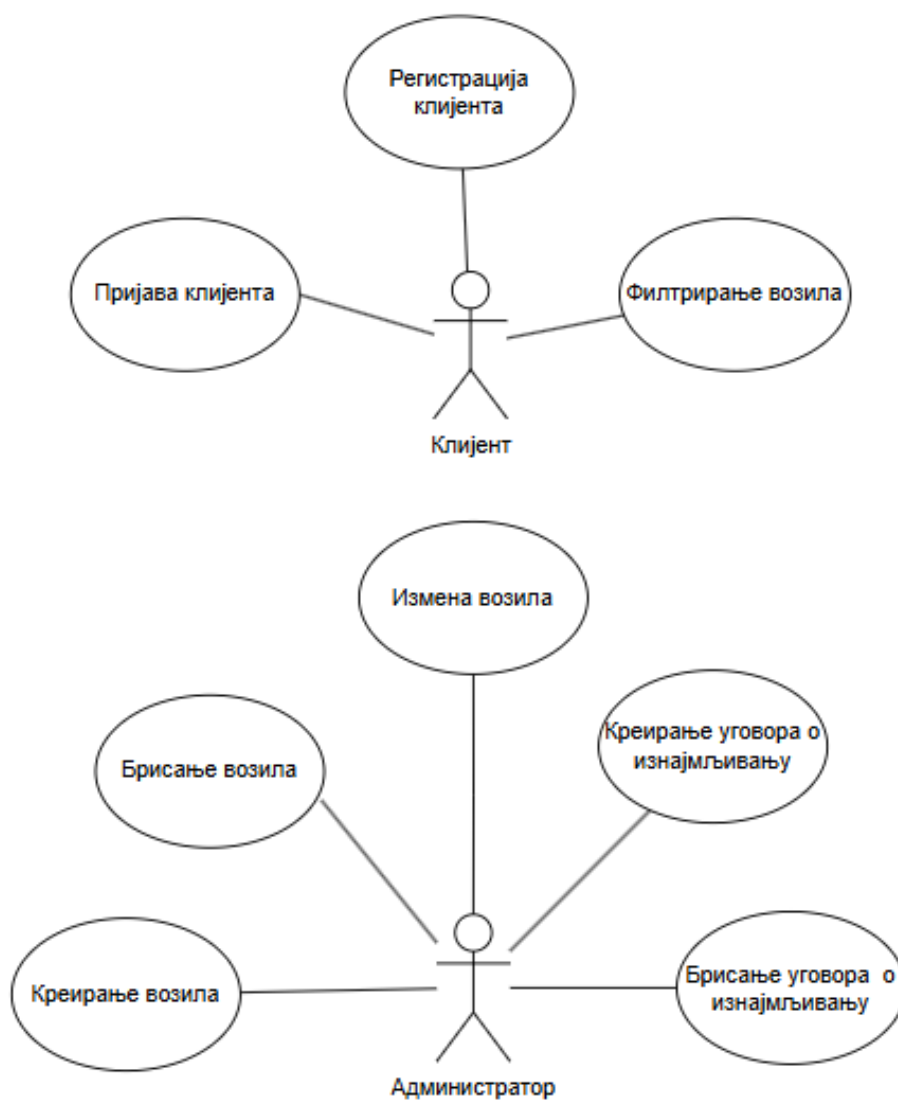
На основу горе наведеног вербалног описа препознајемо случајеве коришћења софтверског система. То су:

1. Пријава клијента
2. Регистрација клијента
3. Филтрирање возила
4. Креирање захтева за изнајмљивање
5. Пријава администратора
6. Креирање возила
7. Брисање возила
8. Измена возила
9. Претрага клијента
10. Креирање клијента
11. Брисање клијента
12. Измена клијента
13. Креирање уговора о изнајмљивању
14. Брисање уговора о изнајмљивању

Од наведених случајева коришћења, даћемо детаљан опис за следеће:

1. Пријава клијента
2. Регистрација клијента
3. Филтрирање возила
4. Креирање возила
5. Брисање возила
6. Измена возила
7. Креирање уговора о изнајмљивању
8. Брисање уговора о изнајмљивању

Модел случајева коришћења визуелно је приказан на следећој слици.



Слика 3 Модел случајева коришћења

СК1: Случај коришћења - Пријава клијента

Назив СК

Пријава клијента

Актори СК

Клијент

Учесници СК

Клијент и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и приказује форму за пријаву клијента.

Основни сценарио СК

1. Клијент уноси податке за пријаву клијента. (АПУСО)
2. Клијент контролише да ли је коректно унео податке за пријаву клијента. (АНСО)
3. Клијент позива систем да пријави клијента са задатим вредностима. (АПСО)
4. Систем пријављује клијента. (СО)
5. Систем враћа пријављеног клијента, приказује клијенту поруку: „Klijent je uspešno prijavljen!”. (ИА)

Алтернативна сценарија

- 5.1. Уколико систем не може да пријави клијента, приказује клијенту поруку: „Pogrešno korisničko ime ili lozinka!”. (ИА)

СК2: Случај коришћења - Регистрација клијента

Назив СК

Регистрација клијента

Актори СК

Клијент

Учесници СК

Клијент и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и приказује форму за регистрацију клијента.

Основни сценарио СК

1. Клијент уноси податке за регистрацију клијента. (АПУСО)
2. Клијент контролише да ли је коректно унео податке за регистрацију клијента. (АНСО)
3. Клијент позива систем да региструје клијента са задатим вредностима. (АПСО)
4. Систем региструје клијента. (СО)
5. Систем враћа регистрованог клијента, приказује клијенту поруку: „Klijent je uspešno registrovan!”. (ИА)

Алтернативна сценарија

- 5.1. Уколико систем не може да региструје клијента, приказује клијенту поруку: „Ovaj klijent već ima nalog!”. (ИА)

СКЗ: Случај коришћења – Филтрирање возила

Назив СК

Филтрирање возила

Актори СК

Клијент

Учесници СК

Клијент и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и приказује почетну страницу. Учитана је листа возила.

Основни сценарио СК

1. Клијент бира критеријуме на основу којих жели да претражује возила (АПУСО)
2. Клијент позива систем да прикаже доступна возила по задатим критеријумима. (АПСО)
3. Систем тражи доступна возила по задатим критеријумима. (СО)
4. Систем приказује кориснику доступна возила. (ИА)

Алтернативна сценарија

- 4.1. Уколико систем не може да нађе возила, приказује клијенту поруку: „Nažalost nema dostupnih vozila." (ИА)

СК4: Случај коришћења – Креирање возила

Назив СК

Креирање возила

Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и администратор је пријављен под својом шифром. Форма за креирање возила је приказана. Учитана је листа модела возила.

Основни сценарио СК

1. Администратор уноси податке за креирање возила. (АПУСО)
2. Администратор контролише да ли је коректно унео податке за креирање возила. (АНСО)
3. Администратор позива систем да креира возило са задатим вредностима. (АПСО)
4. Систем креира возило. (СО)
5. Систем приказује администратору креирано возило и поруку: „Vozilo je uspešno kreirano.” (ИА)

Алтернативна сценарија

- 5.1. Уколико систем не може да креира возило, приказује администратору поруку: „Sistem ne može da kreira vozilo.” (ИА)

СК5: Случај коришћења – Брисање возила

Назив СК

Брисање возила

Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и администратор је пријављен под својом шифром. Учитана је листа возила.

Основни сценарио СК

1. Администратор бира критеријуме на основу којих жели да претражује возила (АПУСО)
2. Администратор позива систем да прикаже доступна возила по задатим критеријумима. (АПСО)
3. Систем тражи доступна возила по задатим критеријумима. (СО)
4. Систем приказује администратору доступна возила. (ИА)
5. Администратор бира возило која жели да избрише. (АПУСО)
6. Администратор позива систем да избрише возило. (АПСО)
7. Систем брише возило. (СО)
8. Систем приказује администратору поруку: „Vozilo je uspešno obrisano.” (ИА)

Алтернативна сценарија

4.1. Уколико **систем** не може да пронађе доступна возила, приказује поруку о грешци: „Nažalost nema dostupnih vozila.“. Прекида се извршење сценарија. (ИА)

8.1. Уколико **систем** не може да избрише **возило**, приказује **администратору** поруку: „Vozilo ne može da se obriše.” (ИА)

СК6: Случај коришћења – Измена возила

Назив СК

Измена возила

Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и администратор је пријављен под својом шифром. Приказана је форма за приказ возила која позива систем да учита листу возила. Учитана је листа возила и модела возила.

Основни сценарио СК

1. **Администратор бира** критеријуме на основу којих жели да претражује возила (АПУСО)
2. **Администратор позива** систем да прикаже доступна **возила** по задатим критеријумима. (АПСО)
3. **Систем тражи** доступна **возила** по задатим критеријумима. (СО)
4. **Систем приказује** кориснику доступна **возила**. (ИА)
5. **Администратор бира возило** које жели да измени. (АПУСО)
6. **Администратор позива** систем да учита изабрано **возило**. (АПСО)
7. **Систем учитава возило** за измену. (СО)
8. **Систем приказује администратору возило** за измену и поруку „Vozilo је uspešno izabrano.” (ИА)
9. **Администратор мења** податке о **возилу**. (АПУСО)
10. **Администратор позива систем** да сачува измењене податке о **возилу**. (АПСО)
11. **Систем чува** измењене податке о **возилу**. (СО)
12. **Систем приказује администратору измењено возило** и поруку: „Vozilo је uspešno izmenjeno.” (ИА)

Алтернативна сценарија

- 4.1. Уколико **систем** не може да пронађе доступна возила, приказује поруку о грешци: „Nažalost nema dostupnih vozila.“. Прекида се извршење сценарија. (ИА)
- 8.1. Уколико **систем** не може да прикаже возило за измену, приказује поруку о грешци: „Sistem ne može da prikaže vozilo.“. Прекида се извршење сценарија. (ИА)
- 12.1. Уколико **систем** не може да измени **возило**, приказује **администратору** поруку: „Sistem ne može da izmeni vozilo po zadatim vrednostima.” (ИА)

СК7: Случај коришћења – Креирање уговора о изнајмљивању

Назив СК

Креирање уговора о изнајмљивању

Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов

Систем је укључен и администратор је пријављен под својом шифром. Форма за креирање уговора о изнајмљивању је приказана. Учитана је листа клијента и листа возила.

Основни сценарио СК

1. Администратор уноси податке за креирање уговора о изнајмљивању. (АПУСО)
2. Администратор контролише да ли је коректно унео податке за креирање уговора о изнајмљивању. (АНСО)
3. Администратор позива систем да креира уговор о изнајмљивању са задатим вредностима. (АПСО)
4. Систем креира уговор о изнајмљивању. (СО)
5. Систем приказује администратору креиран уговор о изнајмљивању и поруку: „Ugovor o iznajmljivanju je uspešno kreiran.” (ИА)

Алтернативна сценарија

- 5.1. Уколико систем не може да креира уговор о изнајмљивању, приказује администратору поруку: „Sistem ne može da kreira ugovor o iznajmljivanju.”. (ИА)

СК8: Случај коришћења – Брисање уговора о изнајмљивању

Назив СК

Брисање уговора о изнајмљивању

Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов

Систем је укључен и администратор је пријављен под својом шифром. Приказана је форма за приказ уговора о изнајмљивању. Учитана је листа уговора о изнајмљивању.

Основни сценарио СК

1. Администратор бира уговор о изнајмљивању који жели да избрише. (АПУСО)
2. Администратор позива систем да избрише уговор о изнајмљивању са задатим вредностима. (АПСО)
3. Систем брише уговор о изнајмљивању. (СО)
4. Систем приказује администратору поруку: „Ugovor o iznajmljivanju je uspešno izbrisan!”. (ИА)

Алтернативна сценарија

- 4.1. Уколико систем не може да избрише уговор о изнајмљивању, приказује администратору поруку: „Sistem ne može da izbriše ugovor o iznajmljivanju.”. (ИА)

3.2. АНАЛИЗА

На основу фазе прикупљања корисничких захтева спроводи се фаза анализе. Фаза анализе служи да опише понашање и структуру софтверског система [6]. Понашање се описује помоћу системских дијаграма секвенци, док се структура описује помоћу концептуалног и релационог модела [6].

3.2.1. Понашање софтверског система - Системски дијаграми секвенци

Понашање система је описано преко системских дијаграма секвенци за сваки случај коришћења [6]. Дијаграми секвенци представљају интеракцију између актора и система у одређеном редоследу. Актори позивају догађаје (присак дугмета), док систем преко посредника прима и позива одговарајуће системске операције [6].

ДС1: Дијаграм секвенце случаја коришћења - Пријава клијента

Назив СК

Пријава клијента

Актори СК

Клијент

Учесници СК

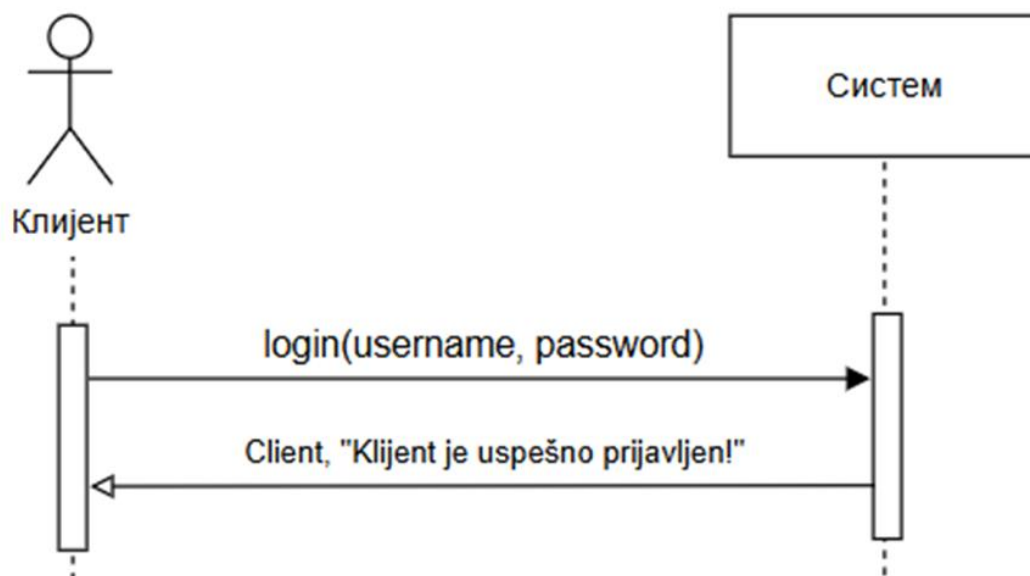
Клијент и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и приказује форму за пријаву клијента.

Основни сценарио СК

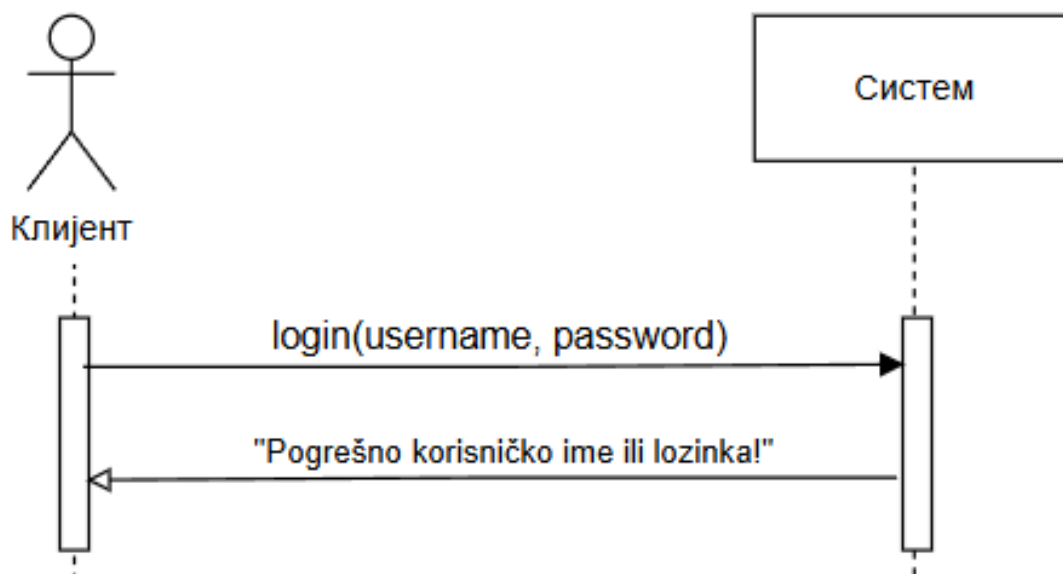
1. **Клијент** позива **систем** да пријави **клијента** са задатим вредностима. (АПСО)
2. **Систем** **враћа** пријављеног **клијента**, приказује **клијенту** поруку: „**Klijent** је uspešno prijavljen!”. (ИА)



Дијаграм 1 Пријава клијента - основни сценарио

Алтернативна сценарија

- 2.1. Уколико **систем** не може да пријави **клијента**, приказује **клијенту** поруку: „Pogrešno korisničko ime ili lozinka!“. (ИА)



Дијаграм 2 Пријава клијента - алтернативни сценарио

На основу системских дијаграма секвенци уочавају се следеће системске операције:

1. signal login(username, password)

ДС2: Дијаграм секвенце случаја коришћења - Регистрација клијента

Назив СК

Регистрација клијента

Актори СК

Клијент

Учесници СК

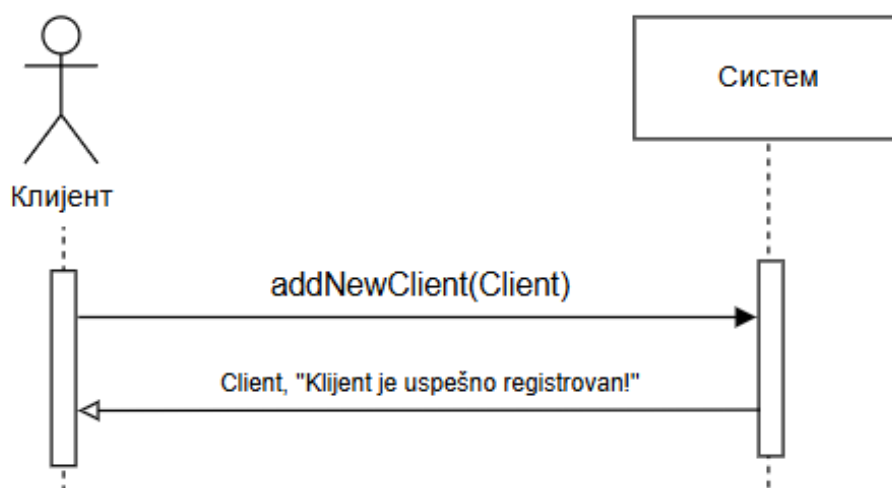
Клијент и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и приказује форму за регистрацију клијента.

Основни сценарио СК

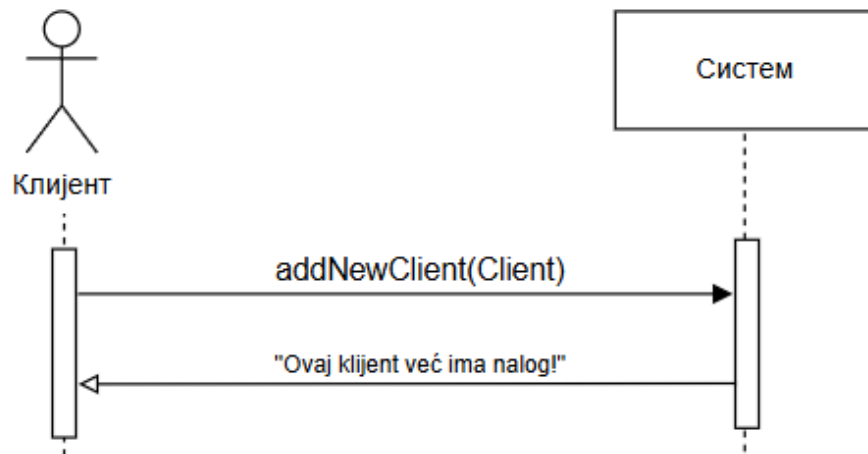
1. Клијент позива систем да региструје клијента са задатим вредностима. (АПСО)
2. Систем враћа регистрованог клијента, приказује клијенту поруку: „Klijent je uspešno registrovan!” (ИА)



Дијаграм 3 Регистрација клијента – основни сценарио

Алтернативна сценарија

- 2.1. Уколико **систем** не може да региструје **клијента**, приказује **клијенту** поруку:
„Ovaj **kljijent** već ima nalog!” (ИА)



Дијаграм 4 Регистрација клијента – алтернативни сценарио

На основу системских дијаграма секвенци уочавају се следеће системске операције:

1. signal addNewClient(Client)

ДСЗ: Дијаграм секвенце случаја коришћења – Филтрирање возила

Назив СК

Филтрирање возила

Актори СК

Клијент

Учесници СК

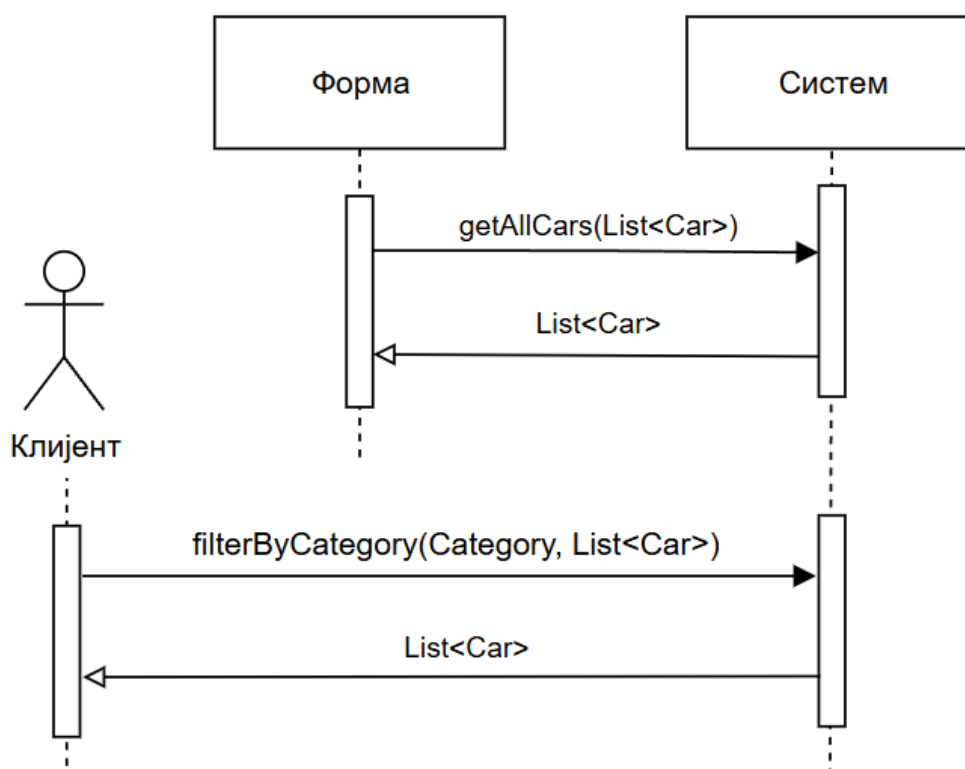
Клијент и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и приказује почетну страницу. Учитана је листа возила.

Основни сценарио СК

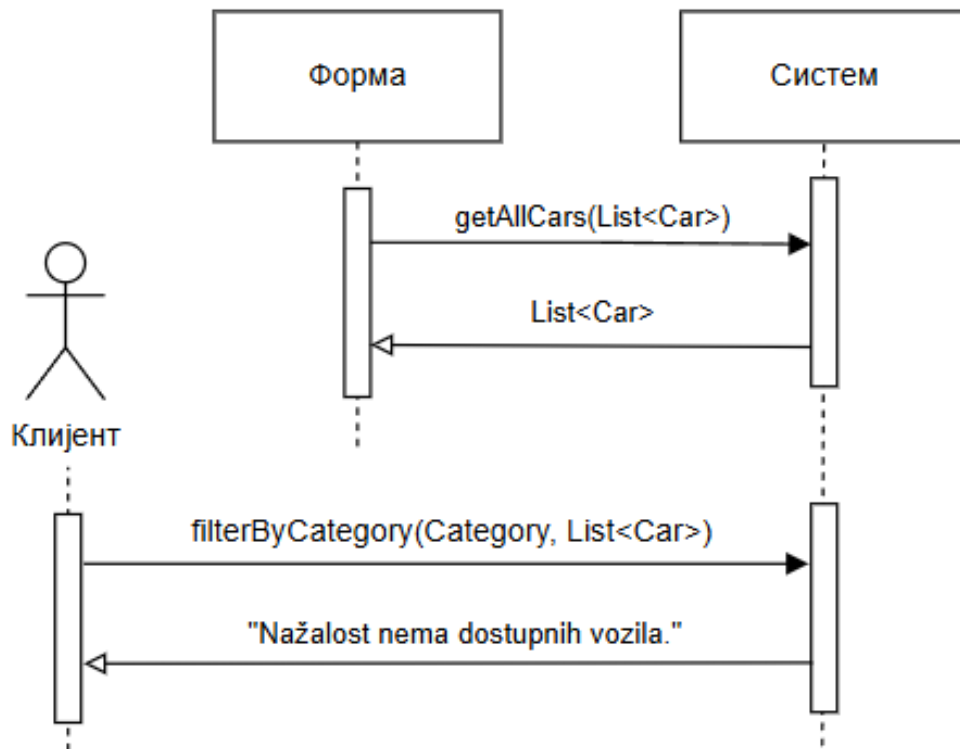
1. Форма **позива** систем да учита листу возила. (АПСО)
2. Систем **приказује** форми листу возила. (ИА)
3. **Клијент** **позива** систем да прикаже доступна **возила** по задатим критеријумима. (АПСО)
4. **Систем** **приказује** кориснику доступна возила. (ИА)



Дијаграм 5 Филтрирање возила – основни сценарио

Алтернативна сценарија

- 4.1. Уколико **систем** не може да нађе **возила**, приказује **клијенту** поруку: „ Nažalost nema dostupnih vozila ”. (ИА)



Дијаграм 6 Филтрирање возила - алтернативна сценарија

На основу системских дијаграма секвенци уочавају се следеће системске операције:

1. signal `getAllCars(List<Car>)`
2. signal `filterByCategory(Category, List<Car>)`

ДС4: Дијаграм секвенце случаја коришћења – Креирање возила

Назив СК

Креирање возила

Актори СК

Администратор

Учесници СК

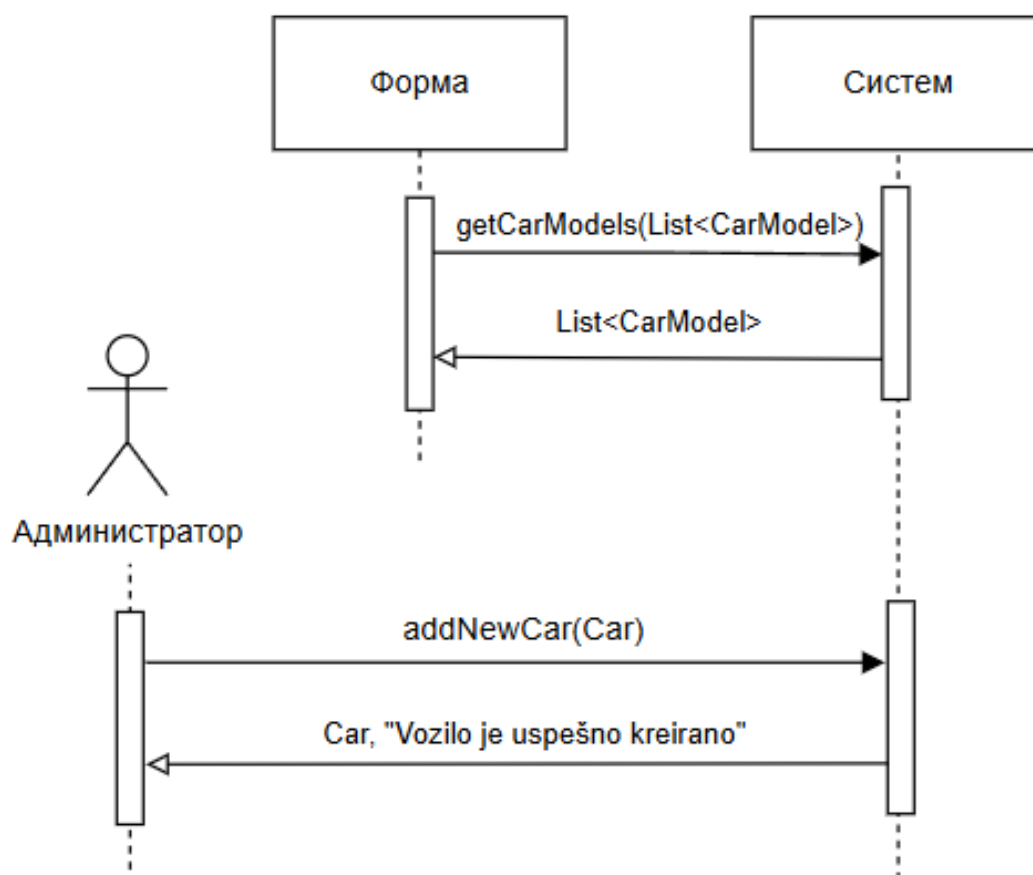
Администратор и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и администратор је пријављен под својом шифром. Форма за креирање возила је приказана. Учитана је листа модела возила.

Основни сценарио СК

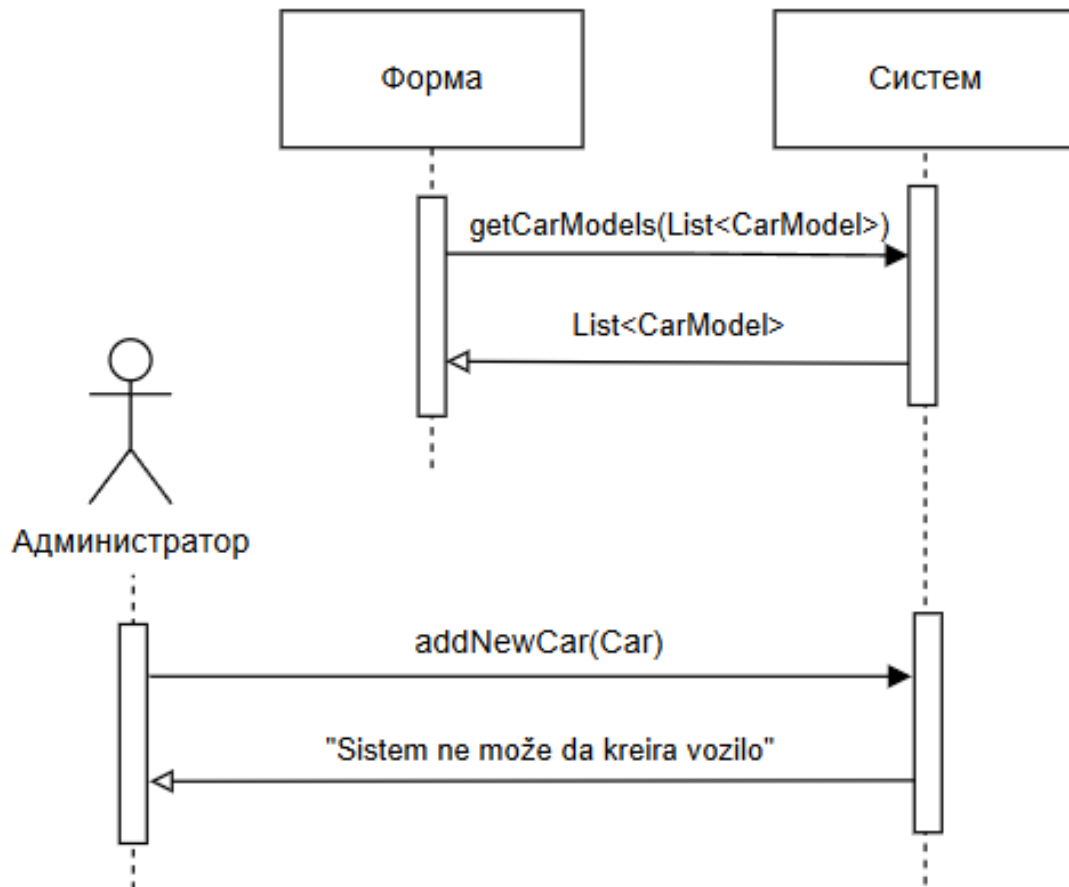
1. Форма **позива** систем да учита листу модела возила. (АПСО)
2. Систем **приказује** форми листу модела возила. (ИА)
3. **Администратор** **позива** **систем** да креира **возило** са задатим вредностима. (АПСО)
4. **Систем** **приказује** **администратору** креирано **возило** и поруку: „Vozilo je uspešno kreirano.”. (ИА)



Дијаграм 7 Креирај возило – основни сценарио

Алтернативна сценарија

- 4.1. Уколико **систем** не може да креира **возило**, приказује **администратору** поруку:
„**Sistem** ne može da kreira **vozilo**.” (ИА)



Дијаграм 8 Креирај возило – алтернативна сценарија

На основу системских дијаграма секвенци уочавају се следеће системске операције:

1. signal `getCarModels(List<CarModel>)`
2. signal `addNewCar(Car)`

ДС5: Дијаграм секвенце случаја коришћења – Брисање возила

Назив СК

Брисање возила

Актори СК

Администратор

Учесници СК

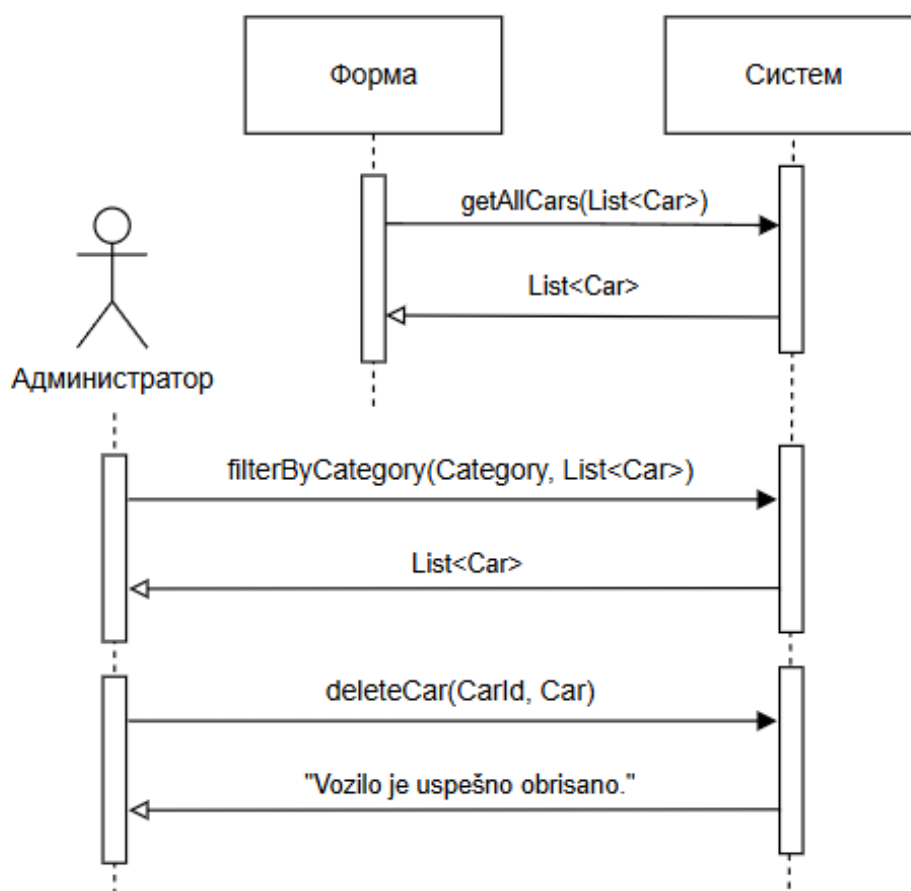
Администратор и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и администратор је пријављен под својом шифром. Учитана је листа возила.

Основни сценарио СК

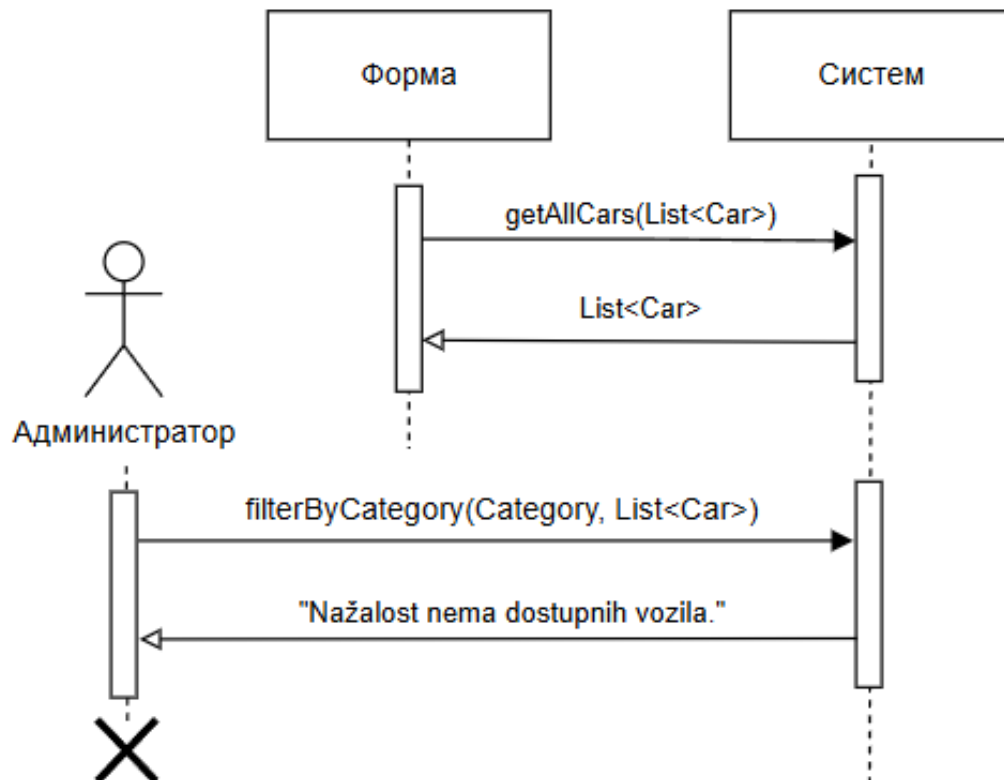
1. Форма **позива** систем да учита листу возила. (АПСО)
2. Систем **приказује** форми листу возила. (ИА)
3. **Администратор** **позива** систем да прикаже доступна **возила** по задатим критеријумима. (АПСО)
4. **Систем** **приказује** кориснику доступна **возила**. (ИА)
5. **Администратор** **позива** **систем** да избрише **возило**. (АПСО)
6. **Систем** **приказује** **администратору** поруку: „Vozilo je uspešno obrisano.” (ИА)



Дијаграм 9 Брисање возила - основни сценарио

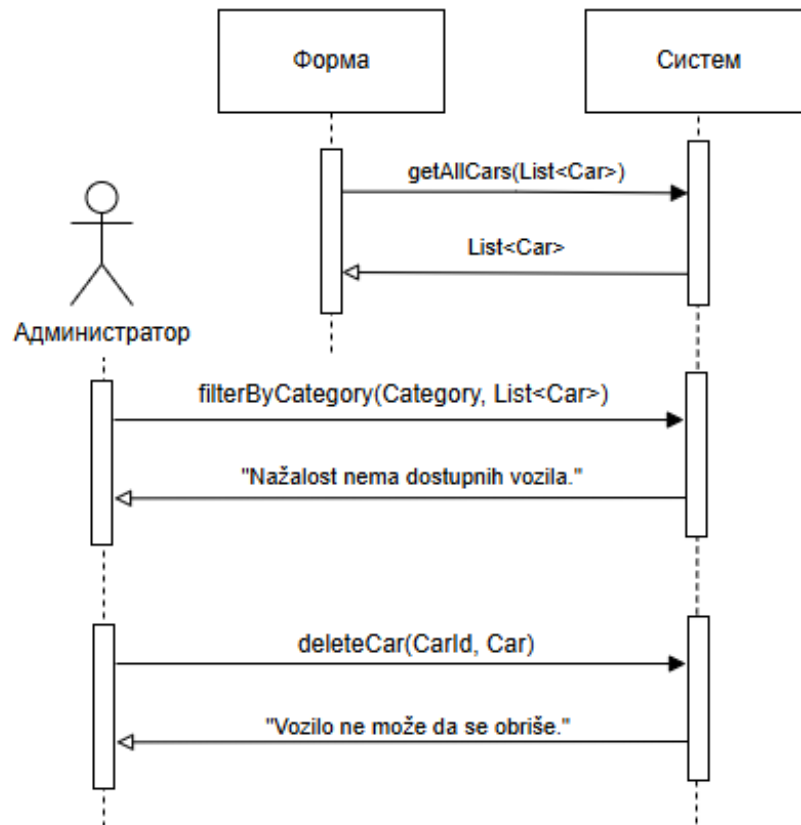
Алтернативна сценарија

- 4.1. Уколико **систем** не може да пронађе доступна возила, приказује поруку о грешци: „Nažalost nema dostupnih vozila.“. Прекида се извршење сценарија. (ИА)



Дијаграм 10 Брисање возила - алтернативна сценарија 4.1.

- 6.1. Уколико **систем** не може да избрише **возило**, приказује **администратору** поруку: „Vozilo ne može da se obriše.” (ИА)



Дијаграм 11 Брисање возила - алтернативна сценарија 6.1

На основу системских дијаграма секвенци уочавају се следеће системске операције:

1. signal getAllCars(List<Car>)
2. signal filterByCategory(Category, List<Car>)
3. signal deleteCar(CarId, Car)

ДС6: Дијаграм секвенце случаја коришћења – Измена возила

Назив СК

Измена возила

Актори СК

Администратор

Учесници СК

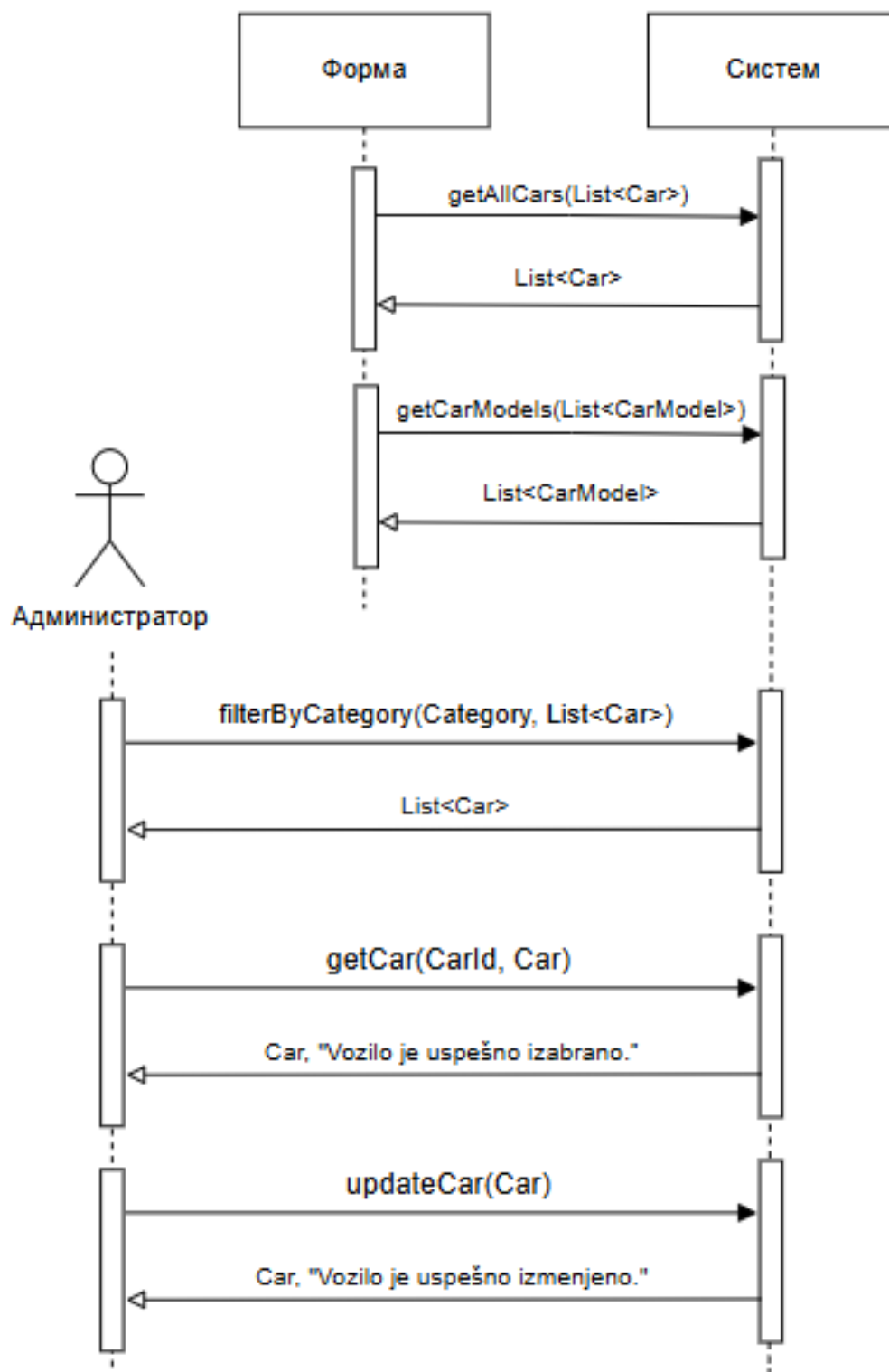
Администратор и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и администратор је пријављен под својом шифром. Приказана је форма за приказ возила која позива систем да учита листу возила. Учитана је листа возила и модела возила.

Основни сценарио СК

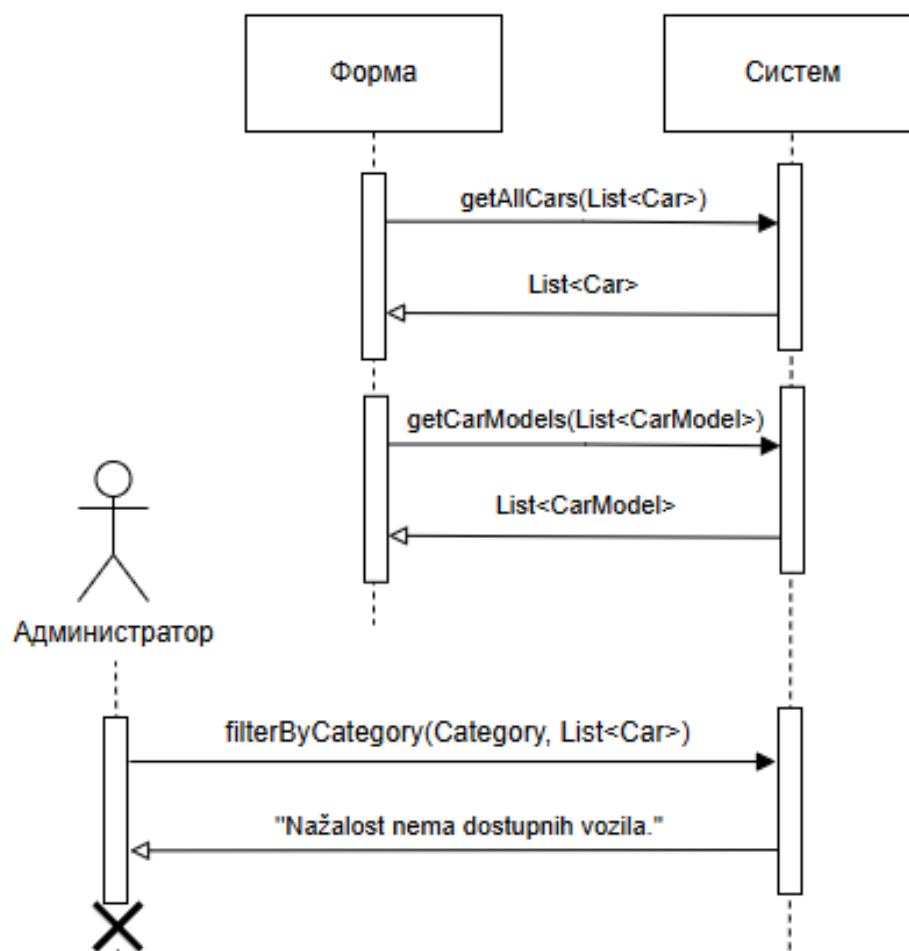
1. Форма **позива** систем да учита листу возила. (АПСО)
2. Систем **приказује** форми листу возила. (ИА)
3. Форма **позива** систем да учита листу модела возила. (АПСО)
4. Систем **приказује** форми листу модела возила. (ИА)
5. **Администратор позива** систем да прикаже доступна возила по задатим критеријумима. (АПСО)
6. **Систем приказује** кориснику доступна возила. (ИА)
7. **Администратор позива** систем да учита изабрано возило. (АПСО)
8. **Систем приказује администратору возило** за измену и поруку „Vozilo je uspešno izabrano.” (ИА)
9. **Администратор позива систем** да сачува измењене податке о возилу. (АПСО)
10. **Систем приказује администратору измењено возило** и поруку: „Vozilo je uspešno izmenjeno.” (ИА)



Дијаграм 12 Измена возила – основни сценарио

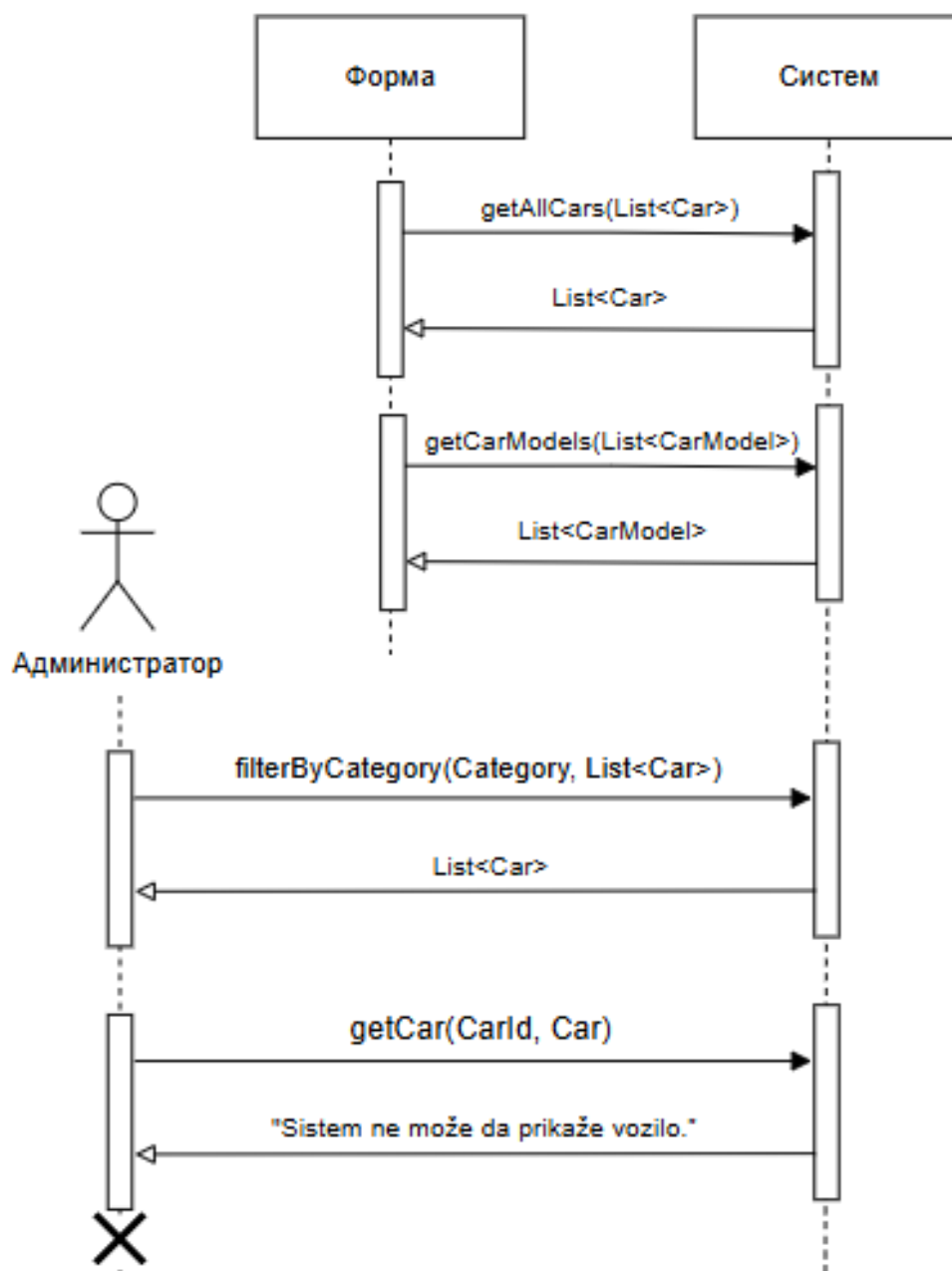
Алтернативна сценарија

- 6.1. Уколико **систем** не може да пронађе доступна возила, приказује поруку о грешци: „Nažalost nema dostupnih vozila.“. Прекида се извршење сценарија. (ИА)



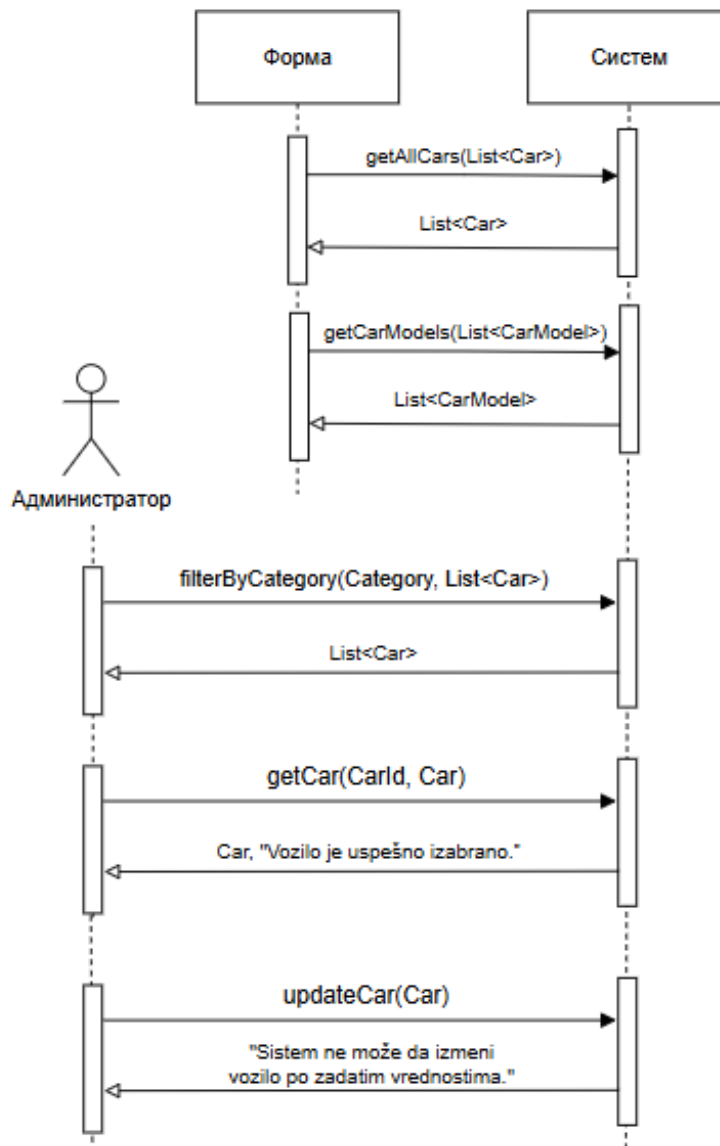
Дијаграм 13 Измена возила – алтернативна сценарија 6.1.

- 8.1. Уколико **систем** не може да прикаже возило за измену, приказује поруку о грешци: „Sistem ne može da prikaže vozilo.“. Прекида се извршење сценарија. (ИА)



Дијаграм 14 Измена возила – алтернативна сценарија 8.1.

10.1. Уколико **систем** не може да измени **возило**, приказује **администратору** поруку: „**Sistem** ne može da izmeni **vozilo** po zadatim vrednostima.” (ИА)



Дијаграм 15 Измена возила – алтернативна сценарија 10.1.

На основу системских дијаграма секвенци уочавају се следеће системске операције:

1. signal `getAllCars(List<Car>)`
2. signal `getCarModels(List<CarModel>)`
3. signal `filterByCategory(Category, List<Car>)`
4. signal `getCar(CarId, Car)`
5. signal `updateCar(Car)`

ДС7: Дијаграм секвенце случаја коришћења – Креирање уговора о изнајмљивању

Назив СК

Креирање уговора о изнајмљивању

Актори СК

Администратор

Учесници СК

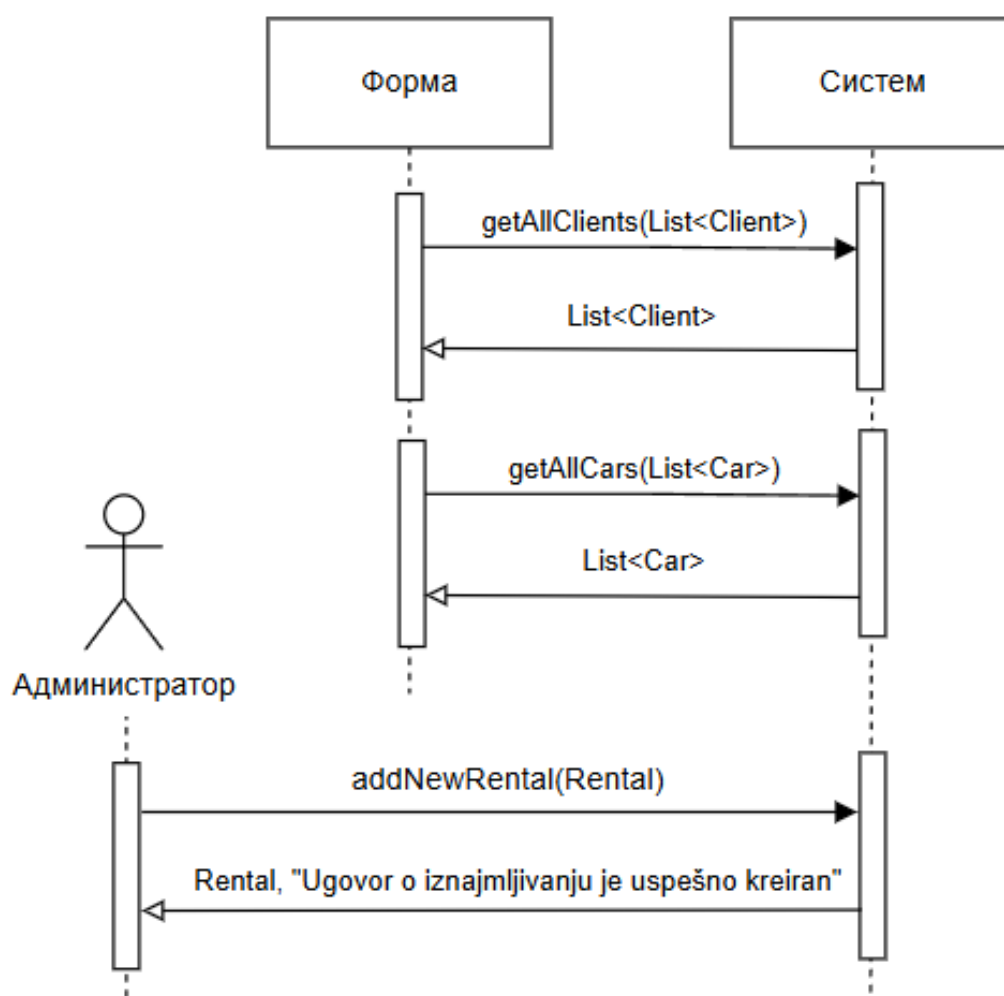
Администратор и систем (програм)

Предуслов

Систем је укључен и администратор је пријављен под својом шифром. Форма за креирање уговора о изнајмљивању је приказана. Учитана је листа клијента и листа возила.

Основни сценарио СК

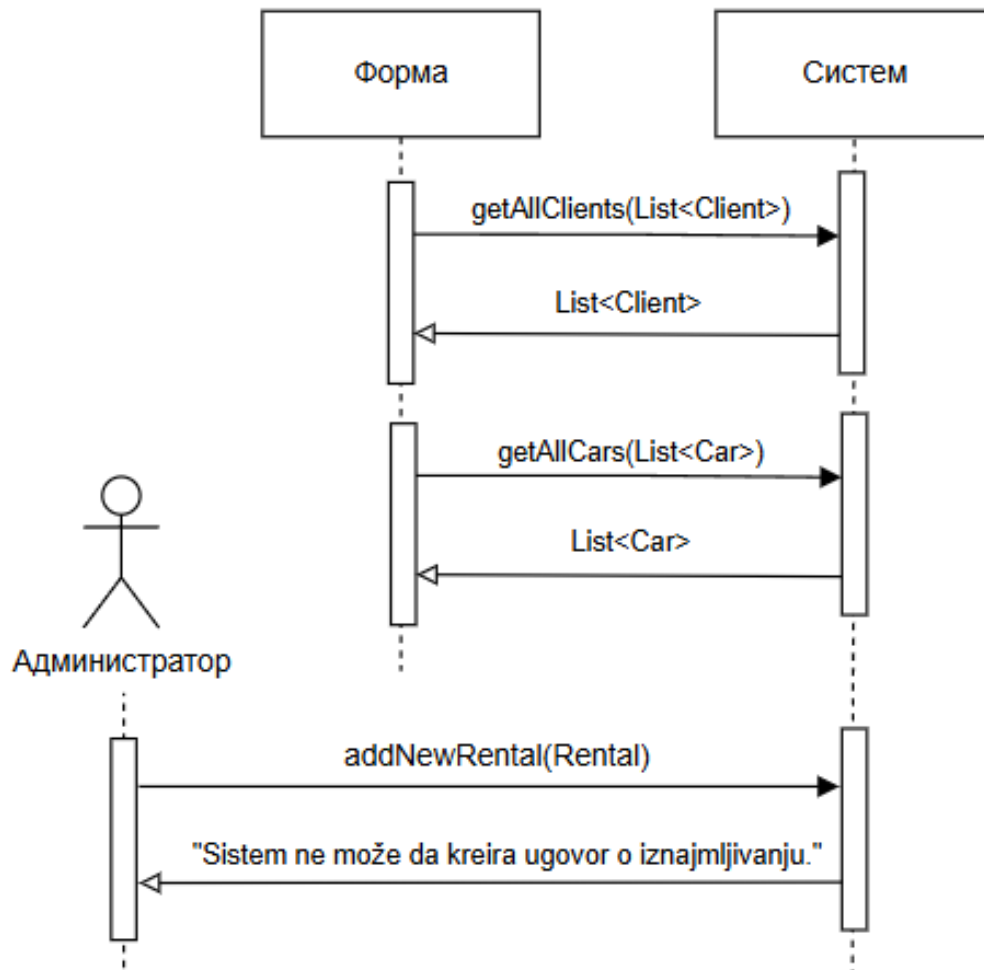
1. Форма **позива** систем да учита листу клијената. (АПСО)
2. Систем **приказује** форми листу клијената. (ИА)
3. Форма **позива** систем да учита листу возила. (АПСО)
4. Систем **приказује** форми листу возила. (ИА)
5. **Администратор** **позива** **систем** да креира уговор о изнајмљивању са задатим вредностима. (АПСО)
6. **Систем** **приказује** **администратору** креиран уговор о изнајмљивању и поруку: „Ugovor o iznajmljivanju je uspešno kreiran.” (ИА)



Дијаграм 16 Креирање уговора о изнајмљивању - основни сценарио

Алтернативна сценарија

- 6.1. Уколико **систем** не може да креира **уговор о изнајмљивању**, приказује **администратору** поруку: „**Sistem** ne može da kreira ugovor o iznajmljivanju.”.
(ИА)



Дијаграм 17 Креирање уговора о изнајмљивању - алтернативна сценарија

На основу системских дијаграма секвенци уочавају се следеће системске операције:

1. signal getAllClients(List<Client>)
2. signal getAllCars(List<Car>)
3. signal addNewRental(Rental)

ДС8: Дијаграм секвенце случаја коришћења – Брисање уговора о изнајмљивању

Назив СК

Брисање уговора о изнајмљивању

Актори СК

Администратор

Учесници СК

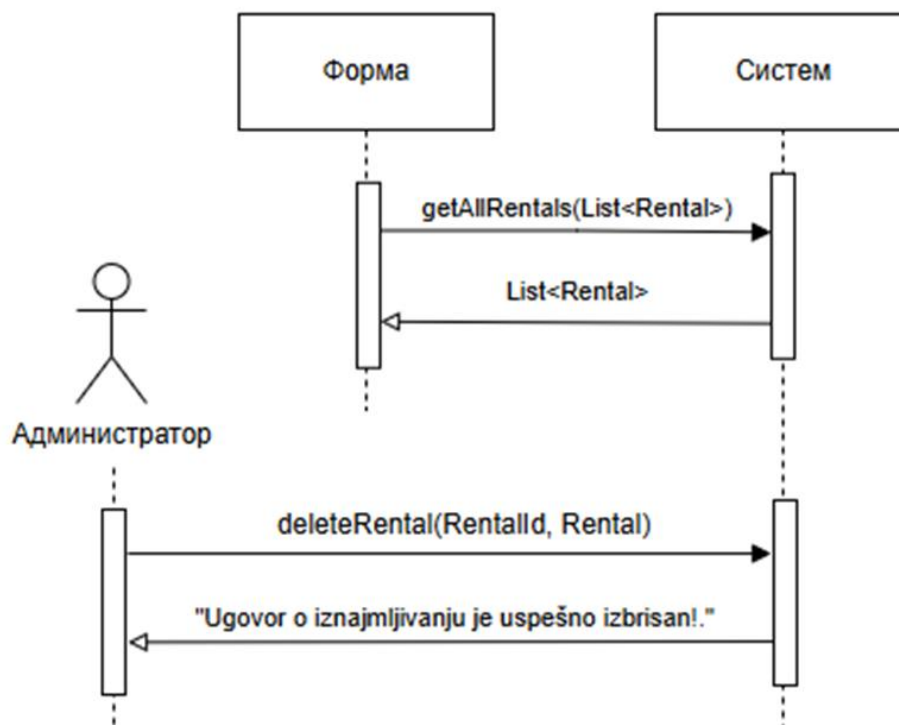
Администратор и систем (програм)

Предуслов

Систем је укључен и администратор је пријављен под својом шифром. Приказана је форма за приказ уговора о изнајмљивању. Учитана је листа уговора о изнајмљивању.

Основни сценарио СК

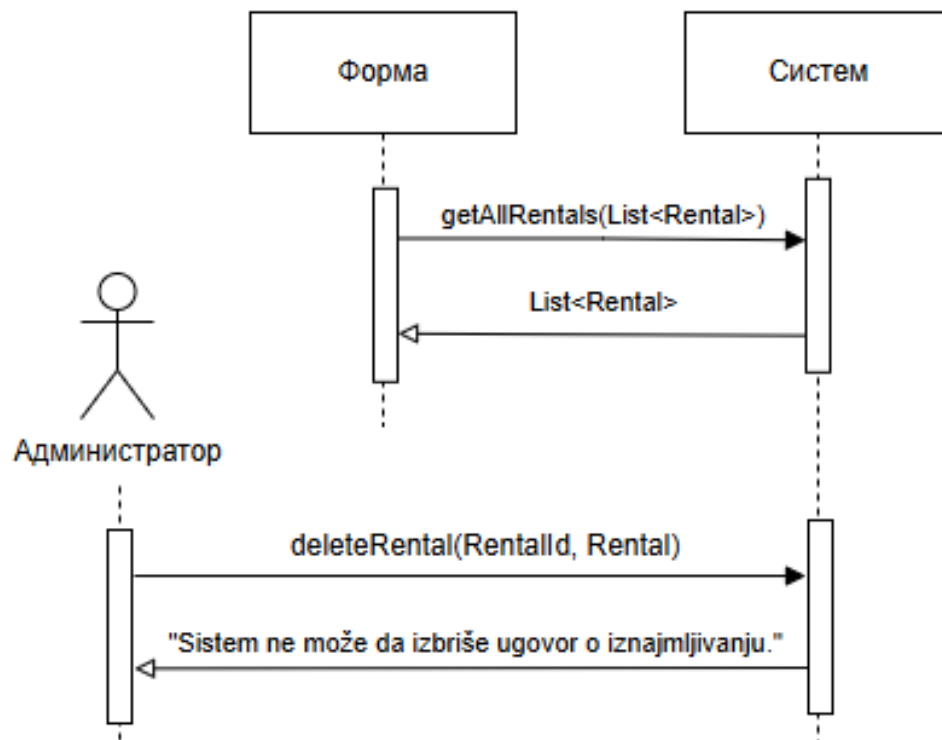
1. Форма **позива** систем да учита листу **уговора о изнајмљивању**. (АПСО)
2. Систем **приказује** форми листу **уговора о изнајмљивању**. (ИА)
3. **Администратор** **позива** **систем** да избрише **уговор о изнајмљивању** са задатим вредностима. (АПСО)
4. **Систем** **приказује** **администратору** поруку: „Ugovor o iznajmljivanju je uspešno izbrisan!”. (ИА)



Дијаграм 18 Брисање уговора о изнајмљивању – основни сценарио

Алтернативна сценарија

- 4.1. Уколико **систем** не може да избрише **уговор о изнајмљивању**, приказује **администратору** поруку: „**Sistem ne može da izbriše ugovor o iznajmljivanju.**”.
(ИА)



Дијаграм 19 Брисање уговора о изнајмљивању - алтернативна сценарија

На основу системских дијаграма секвенци уочавају се следеће системске операције:

1. signal `getAllRentals(List<Rental>)`
2. signal `deleteRental(RentalId, Rental)`

На основу свих наведених системских дијаграма секвенци може се уочити следећих 13 системских операција:

1. signal login(username, password)
2. signal addNewClient(Client)
3. signal getAllCars(List<Car>)
4. signal filterByCategory(Category, List<Car>)
5. signal getCarModels(List<CarModel>)
6. signal addNewCar(Car)
7. signal deleteCar(CarId, Car)
8. signal getCar(CarId, Car)
9. signal updateCar(Car)
10. signal getAllClients(List<Client>)
11. signal addNewRental(Rental)
12. signal getAllRentals(List<Rental>)
13. signal deleteRental(RentalId, Rental)

3.2.2. Понашање софтверског система – уговори

На основу системских операција уочених у фази анализе формирају се уговори. Свака системска операција описује понашање софтверског система. Системска операција мора да има свој потпис, име методе и опционо улазне и/или излазне аргументе. Уговори описују понашање системских операција, шта операција треба да ради, али не зашто то ради [6].

Једна системска операција је везана за један уговор. Сваки уговор се састоји из следећих секција [6]:

- Операција - обухвата назив системске операције и њене улазне и излазне аргументе.
- Веза са СК -наводи случај коришћења или више њих у којима се дата системска операција позива.
- Предуслови - описују стање у коме систем мора бити да би се операција извршила.
- Постуслови - дефинишу стање система након извршења операције; уколико услови нису испуњени, резултати операције се поништавају.

Уговор УГ1: login(username, password): signal

Веза са СК: СК1

Предуслови: /

Постуслови: Клијент је успешно пријављен.

Уговор УГ2: addNewClient(Client): signal

Веза са СК: СК2

Предуслови: Структурна и вредносна ограничења над објектом типа *Client* морају бити задовољена.

Постуслови: Креиран је нови објекат типа *Client*.

Уговор УГ3: getAllCars(List<Car>): signal

Веза са СК: СК3, СК5, СК6, СК7

Предуслови: /

Постуслови: /

Уговор УГ4: filterByCategory(Category, List<Car>): signal

Веза са СК: СК3, СК5, СК6

Предуслови: /

Постуслови: Систем приказује возила која припадају задатој категорији.

Уговор УГ5: getCarModels(List<CarModel>): signal

Веза са СК: СК4, СК6

Предуслови: /

Постуслови: /

Уговор УГ6: addNewCar(Car): signal

Веза са СК: СК4

Предуслови: Структурна и вредносна ограничења над објектом типа *Car* морају бити задовољена.

Постуслови: Креиран је нови објекат типа *Car*.

Уговор УГ7: deleteCar(CarId, Car, Car): signal

Веза са СК: СК5

Предуслови: Структурна и вредносна ограничења над објектом типа *Car* морају бити задовољена. Возило није у активном уговору о изнајмљивању.

Постуслови: Објекат типа *Car* је обрисан.

Уговор УГ8: getCar(CarId, Car): signal

Веза са СК: СК6

Предуслови: /

Постуслови: /

Уговор УГ9: updateCar(Car): signal

Веза са СК: СК6

Предуслови: Структурна и вредносна ограничења над објектом типа *Car* морају бити задовољена

Постуслови: Објекат типа *Car* је измењен.

Уговор УГ10: getAllClients(List<Client>): signal

Веза са СК: СК7

Предуслови: /

Постуслови: /

Уговор УГ11: addNewRental(Rental): signal

Веза са СК: СК7

Предуслови: Структурна и вредносна ограничења над објектом типа *Rental* морају бити задовољена.

Постуслови: Креиран је нови објекат типа *Rental*. Повезано возило није више доступно за изнајмљивање.

Уговор УГ12: getAllRentals(List<Rental>): signal

Веза са СК: СК8

Предуслови: /

Постуслови: /

Уговор УГ13: deleteRental(RentalId, Rental): signal

Веза са СК: СК8

Предуслови: Структурна ограничења над објектом типа *Rental* морају бити задовољена.

Постуслови: Објекат типа *Rental* је обрисан. Повезано возило постаје доступно за ново изнајмљивање.

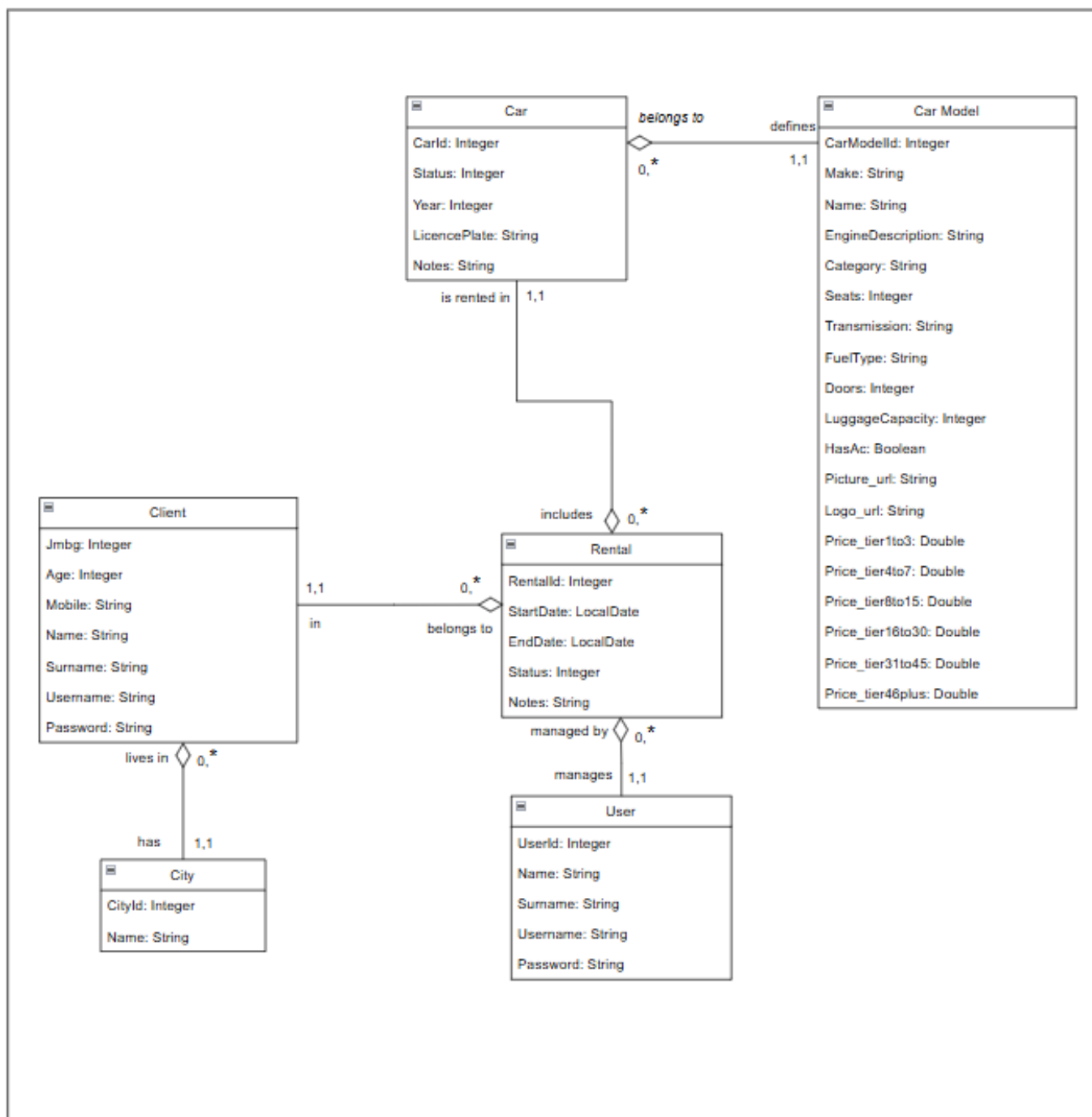
3.2.3. Структура софтверског система

Структура софтверског система представља начин на који су организоване компоненте унутар система и како међусобно функционишу, комуницирају и размењују податке у оквиру домена проблема. Да би се ова структура детаљно описала, користе се концептуални и релациони модели. Ови модели приказују ентитете, њихове особине и везе међу њима, што представља основу за пројектовање базе података и каснију имплементацију система [6].

Концептуални модел

Концептуални модел описује основну структуру софтверског система кроз приказ ентитета у софтверском систему кроз класе и њихове везе. У оквиру концептуалног модела дефинишу се концептуалне класе или објекти реалних система, њихови атрибути и асоцијације које модел визуелно приказује. Концептуалне класе или концепти описују структуру софтверског система, док њихове везе описују како класе комуницирају [6].

На следећој слици приказан је концептуални модел софтверског система за изнајмљивање возила који представља структуру ентитета и њихове међусобне односе.



Слика 4 Концептуални модел

Релациони модел

Релациони модел се прави на основу концептуалног модела. Релациони модел је кључни елемент у процесу пројектовања базе података. Везе између табела базе података, остварују се помоћу кључева, примарних и спољних [6].

Релациони модел за софтверски систем описан у овом раду садржи следеће релације:

- **Car**(carId, carModelId, status, year, licencePlate, notes)
- **CarModel**(carModelId, make, name, engineDescription, category, seats, transmission, fuelType, doors, luggageCapacity, hasAc, picture_url, logo_url, price_tier1to3, price_tier4to7, price_tier8to15, price_tier16to30, price_tier31to45, price_tier46plus)
- **City**(cityId, name)
- **Client**(jmbg, cityId, age, mobile, name, surname, username, password)
- **User**(userId, name, surname, username, password)
- **Rental**(rentalId, carId, userId, clientId, endDate, startDate, status, notes)

Табела Car		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	Име	Тип	Вредност атрибута	Међузависност атрибута једне табеле	Међузависност атрибута више табела	INSERT RESTRICTED CarModel UPDATE CASCADES Rental RESTRICTED CarModel DELETE RESTRICTED Rental
	CarId	int	Not null, Unique, > 0			
	carModelId	int	Not null			
	status	int	Notnul			
	year	int	Not null AND > 0			
	licencePlate	String	Not null AND unique			
	notes	String	Not null			

Табела 1 Вредносна и структурна ограничења за табелу Car

Табела CarModel		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	Име	Тип	Вредност атрибута	Међузависност атрибута једне табеле	Међузависност атрибута више табела	INSERT / UPDATE CASCADES Car DELETE RESTRICTED Car
	carModelId	int	Not null, unique, > 0			
	make	String	Not null			
	name	String	Not null			
	engineDescription	String	/			
	category	String	Not null			
	seats	int	Not null AND >0			
	transmission	String	Not null			
	fuelType	String	Not null			
	doors	int	>0			
	luggageCapacity	int	>0			
	hasAc	boolean	Not null			
	picture_url	String	/			
	logo_url	String	/			
	price_tier1to3	double	Not null AND >0			
	price_tier4to7	double	Not null AND >0			
	price_tier8to15	double	Not null AND >0			
	price_tier16to30	double	Not null AND >0			
	price_tier31to45	double	Not null AND >0			
	price_tier46plus	double	Not null AND >0			

Табела 2 Вредносна и структурна ограничења за табелу CarModel

Табела City		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	Име	Тип	Вредност атрибута	Међузависност атрибута једне табеле	Међузависност атрибута више табела	INSERT /
	cityId	int	Not null, unique, > 0			UPDATE CASCADES Client
	name	String	Not null			DELETE RESTRICTED Client

Табела 1 Вредносна и структурна ограничења за табелу City

Табела Client		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	Име	Тип	Вредност атрибута	Међузависност атрибута једне табеле	Међузависност атрибута више табела	
	jmbg	int	Not null, unique, > 0			INSERT RESTRICTED City
	cityId	int	Not null			UPDATE CASCADES Rental RESTRICTED City
	age	int	Not null			
	mobile	String	Not null			
	name	String	Not null			
	surname	String	Not null			DELETE RESTRICTED Rental
	username	String	Not null AND unique			
	password	String	Not null			

Табела 2 Вредносна и структурна ограничења за табелу Client

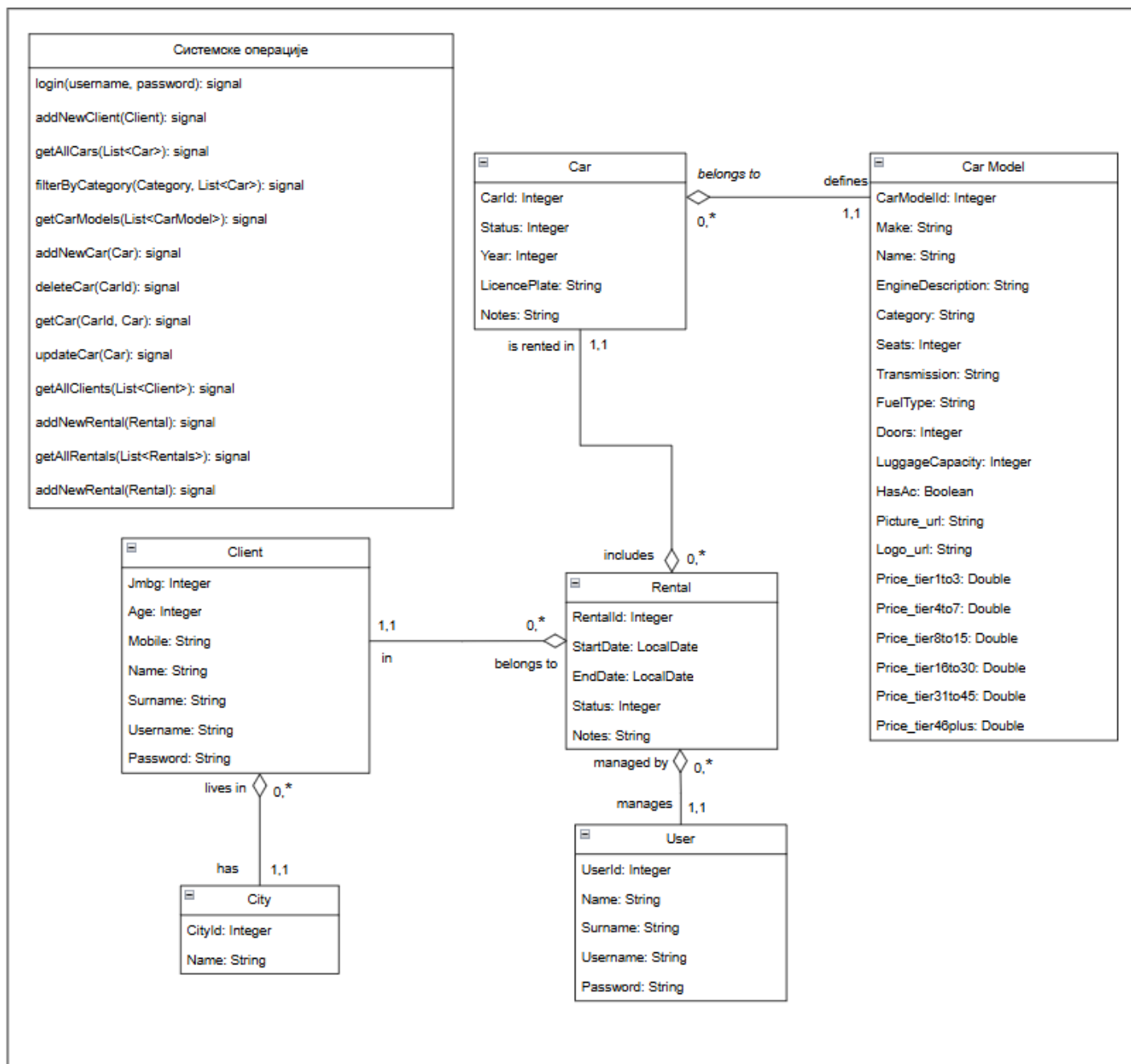
Табела User		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	Име	Тип	Вредност атрибута	Међузависност атрибута једне табеле	Међузависност атрибута више табела	INSERT / UPDATE CASCADES Rental DELETE RESTRICTED Rental
	userId	int	Not null, unique, >0			
	name	String	Not null			
	surname	String	Not null			
	username	String	Not null			
	password	String	Not null			

Табела 3 Вредносна и структурна ограничења за табелу User

Табела Rental		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	Име	Тип	Вредност атрибута	Међузависност атрибута једне табеле	Међузависност атрибута више табела	INSERT RESTRICTED Car, User, Client UPDATE RESTRICTED Car, User, Client DELETE /
	rentalId	int	Not null, unique, > 0			
	carId	int	Not null			
	userId	int	Not null			
	clientId	int	Not null			
	startDate	LocalDate	Not null	startDate < endDate		
	endDate	LocalDate	Not null	startDate < endDate		
	status	int	Not null AND unique			
	notes	String	Not null			

Табела 4 Вредносна и структурна ограничења за табелу Rental

Као резултат фазе анализе, добија се логичка структура и понашање софтверског система, који су приказани на следећој слици. Дијаграм обједињује концептуални модел система са уоченим системским операцијама, чиме се приказује веза између структуре података и функционалности система [6].



Слика 5 Софтверски систем

3.3.ПРОЈЕКТОВАЊЕ

Фазу пројектовања дефинише фаза у којој се прелази са логичке на физичку реализацију система. У овој фази се дефинише понашање, архитектура и структура софтверског система на основу прикупљених захтева и анализе [6].

Током фазе пројектовања користи се тринивојска архитектура, која систем дели на три нивоа: кориснички интерфејс, апликациону логику и складиште података. Фаза пројектовања подразумева [6]:

1. Пројектовање корисничког интерфејса: креирају се екранске форме које омогућавају кориснику интеракцију са системом, као и контролер корисничког интерфејса који обрађује захтеве корисника.
2. Пројектовање апликационе логике: на основу пословне логике дефинисане у фази анализе, пројектују се контролери апликационе логике и брокер базе података који омогућава комуникацију са складиштем података.
3. Пројектовање складишта података: пројектује се на основу доменског модела софтверског система и служи за трајно чување података.

На следећој слици приказана је тринивојска архитектура која се дефинише у фази пројектовања софтвера.



слика 6 Тринивојска архитектура софтверског система [6]

3.3.1. Пројектовање корисничког интерфејса

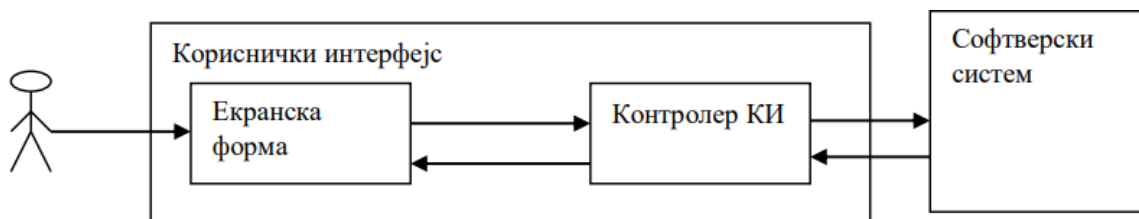
Кориснички интерфејс представља део софтверског система који има улогу посредника између система и крајњег корисника. Његова основна функција је да омогући приказ, унос и размену података на јасан и једноставан начин. Приликом пројектовања, потребно је обратити пажњу на структуру и међусобну повезаност компоненти корисничког интерфејса, као и начину комуницирања са софтверским системом [6].

Кориснички интерфејс се састоји из екранских форми и контролера [6].

Екранске форме прихватају податке и догађаје од корисника, прослеђују те податке контролеру графичког интерфејса и приказују добијене резултате [6].

Контролер корисничког интерфејса прима податке од екранских форми, претвара их у објекат који представља улазни аргумент за системску операцију, шаље захтеве апликационом серверу, прима резултате извршења и приказује их кориснику на одговарајући начин [6].

Структура корисничког интерфејса приказана је на следећој слици.



слика 7 Структура корисничког интерфејса [6]

СК1: Случај коришћења - Пријава клијента

Назив СК

Пријава клијента

Актори СК

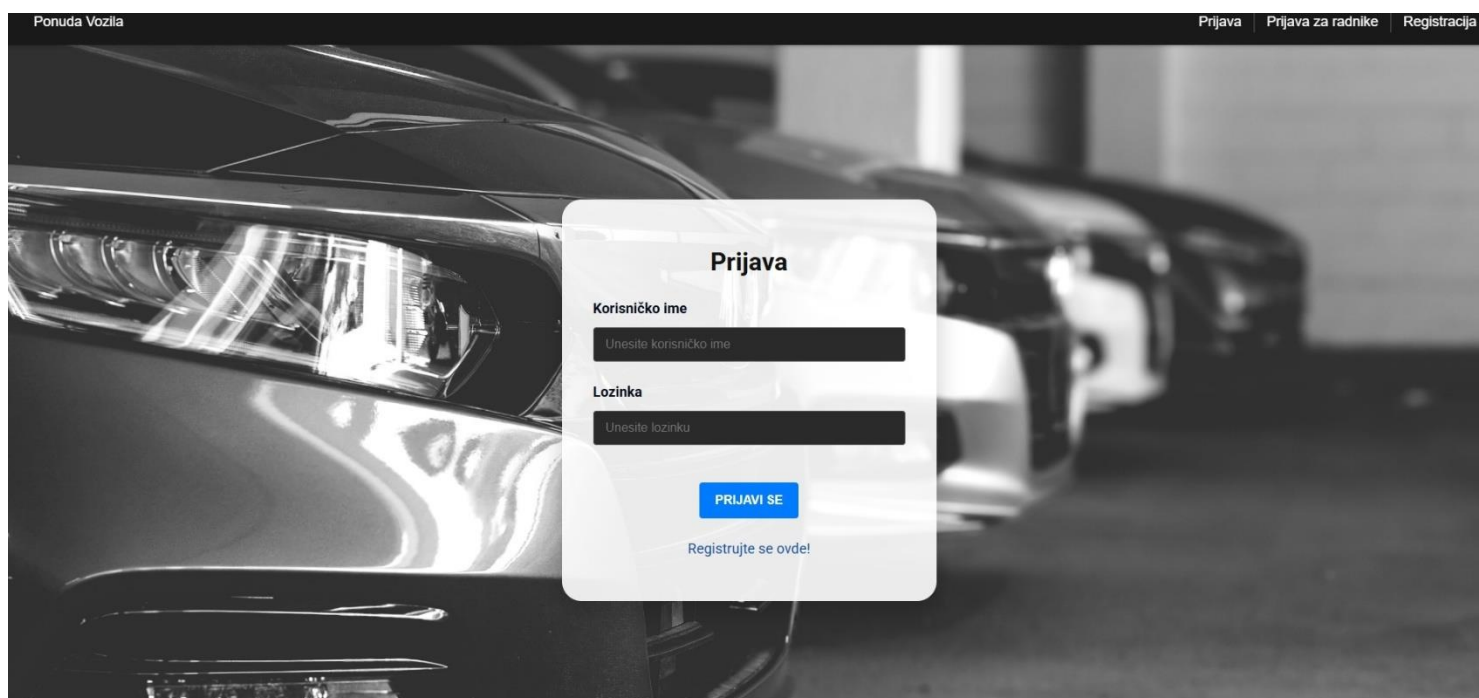
Клијент

Учесници СК

Клијент и систем (програм)

Предуслов

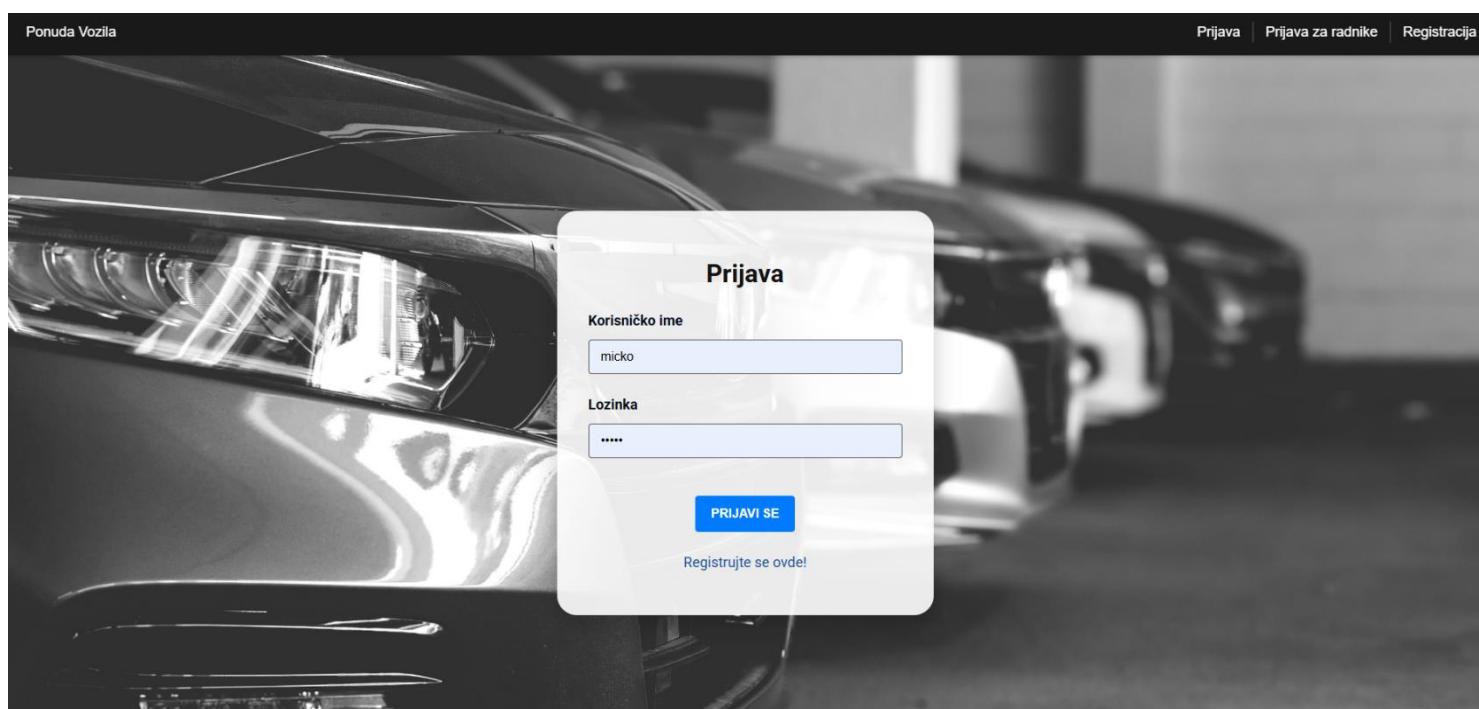
Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и приказује форму за пријаву клијента.



Слика 8 Пријава клијента – форма за пријаву клијента

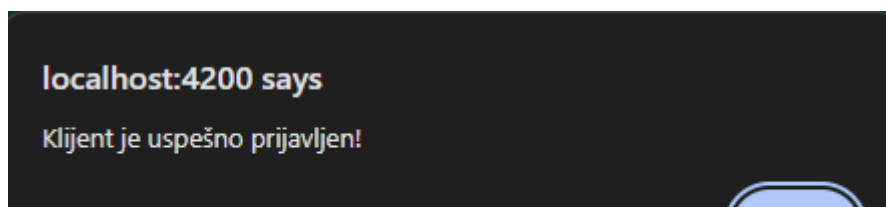
Основни сценарио СК

1. **Клијент** уноси податке за пријаву **клијента**. (АПУСО)
2. **Клијент** контролише да ли је коректно унео податке за пријаву **клијента**. (АНСО)
3. **Клијент** позива **систем** да пријави **клијента** са задатим вредностима. (АПСО)
Опис акције: Након уноса корисничког имена и лозинке потребно је кликнути на дугме "PRIJAVI SE" чиме се позива системска операција Login(username, password).



Слика 9 Пријава клијента – попуњена форма за пријаву клијента

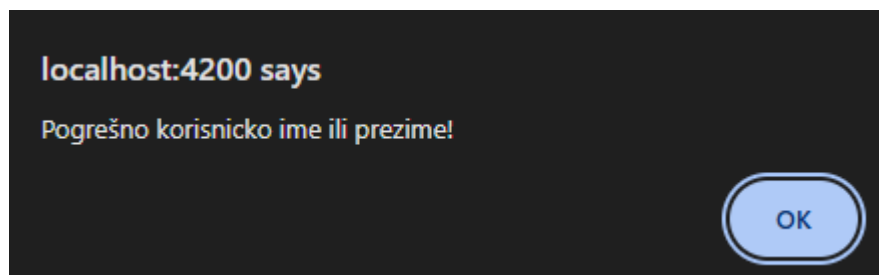
4. **Систем** пријављује **клијента**. (СО)
5. **Систем** враћа пријављеног **клијента**, приказује **клијенту** поруку: „**Klijent** је uspešno prijavljen!”. (ИА)



слика 10 Пријава клијента – успешна пријава

Алтернативна сценарија

- 5.1. Уколико **систем** не може да пријави **клијента**, приказује **клијенту** поруку:
„Pogrešno korisničko ime ili lozinka!“. (ИА)



слика 11 Пријава клијента – неуспешна пријава

СК2: Случај коришћења - Регистрација клијента

Назив СК

Регистрација клијента

Актори СК

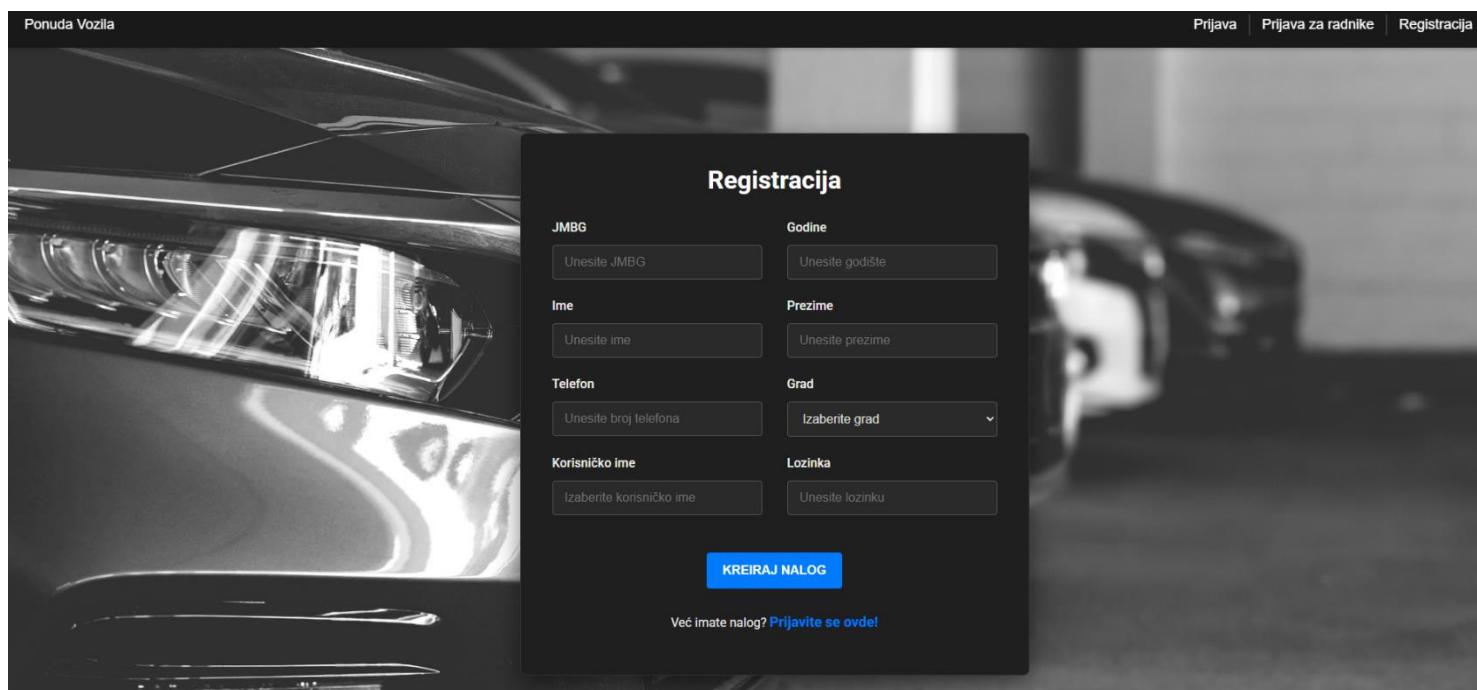
Клијент

Учесници СК

Клијент и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и приказује форму за регистрацију клијента.



Ponuda Vozila

Prijava | Prijava za radnike | Registracija

Registracija

JMBG	Godine
Unesite JMBG	Unesite godište
Ime	Prezime
Unesite ime	Unesite prezime
Telefon	Grad
Unesite broj telefona	Izaberite grad
Korisničko ime	Lozinka
Izaberite korisničko ime	Unesite lozinku

[KREIRAJ NALOG](#)

Već imate nalog? [Prijavite se ovdje!](#)

Слика 12 Регистрација клијента – почетна форма за регистрацију клијента

Основни сценарио СК

1. **Клијент** уноси податке за регистрацију **клијента**. (АПУСО)
2. **Клијент** контролише да ли је коректно унео податке за регистрацију **клијента**. (АНСО)
3. **Клијент** позива **систем** да региструје **клијента** са задатим вредностима. (АПСО)

Опис акције: Након уноса свих потребних података, потребно је кликнути на дугме "KREIRAJ NALOG" чиме се позива системска операција `addNewClient(Client)`.

Ponuda Vozila Prijava Prijava za radnike Registracija

Registracija

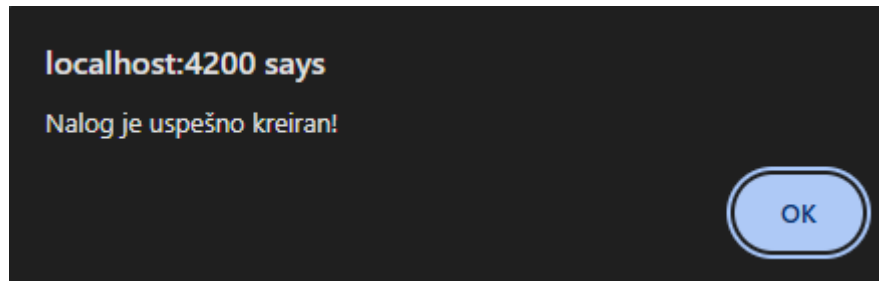
JMBG	Godine
1234567891239	24
Ime	Prezime
Dimitrije	Mitić
Telefon	Grad
0648617909	Beograd
Korisničko ime	Lozinka
mickica112	*****

[KREIRAJ NALOG](#)

Već imate nalog? [Prijavite se ovdje!](#)

Слика 13 Регистрација клијента – попуњена форма за регистрацију клијента

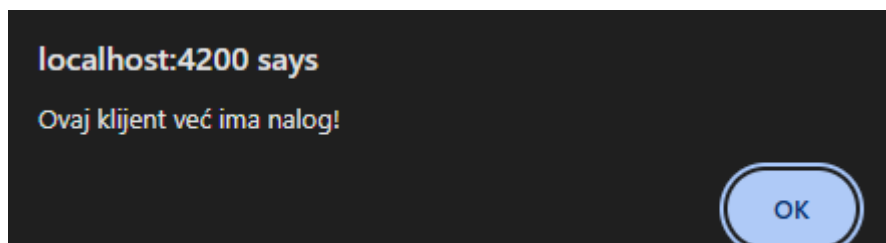
4. Систем региструје клијента. (CO)
5. Систем враћа регистрованог клијента, приказује клијенту поруку: „Klijent je uspešno registrovan!”(ИА)



Слика 14 Регистрација клијента – успешна регистрација

Алтернативна сценарија

- 5.1 Уколико **систем** не може да региструје **клијента**, приказује **клијенту** поруку:
„Ovaj **klijent** već ima nalog!”. (ИА)



слика 15 Регистрација клијента – неуспешна регистрација

СКЗ: Случај коришћења – Филтрирање возила

Назив СК

Филтрирање возила

Актори СК

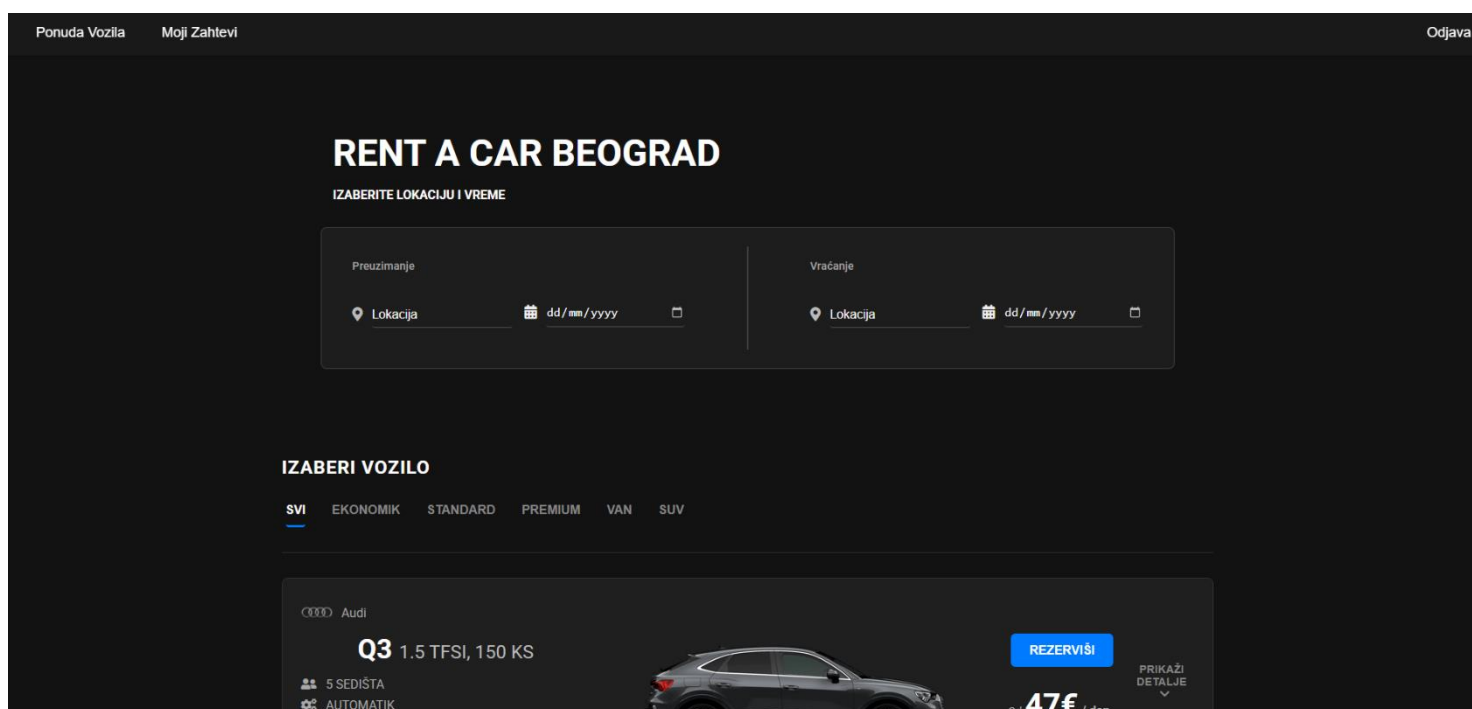
Клијент

Учесници СК

Клијент и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и приказује почетну страницу. Учитана је листа возила.



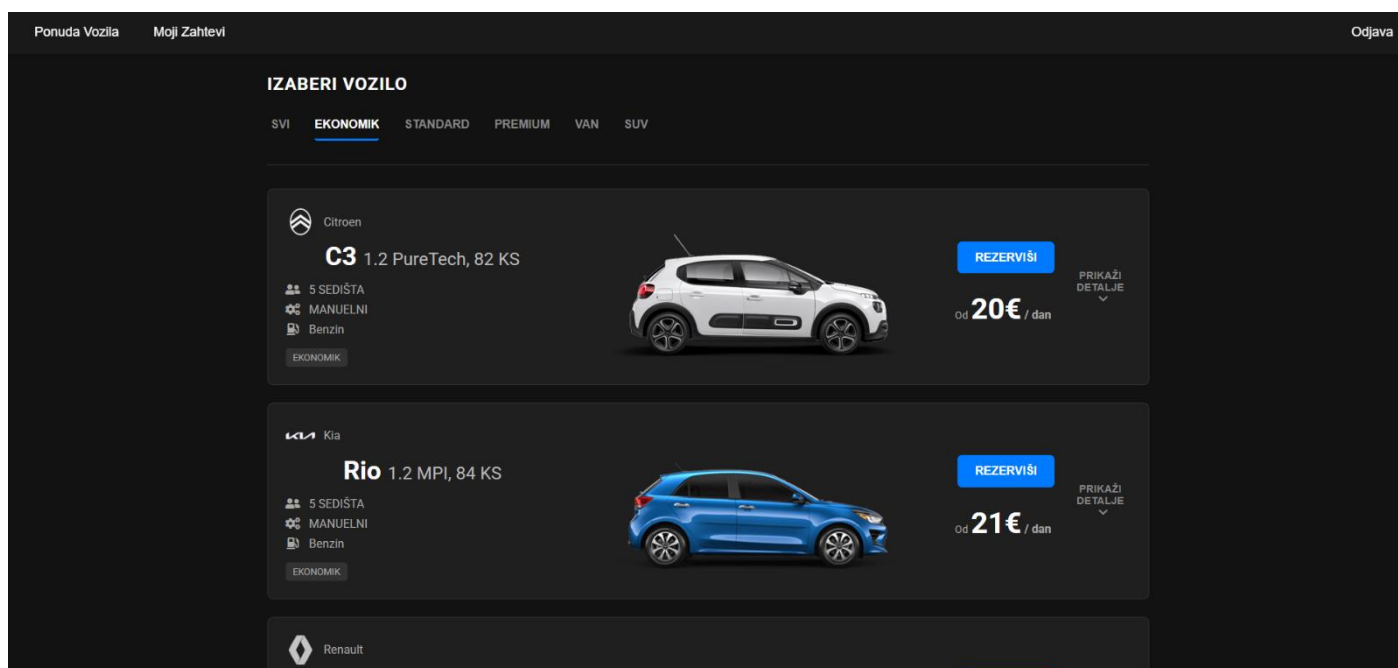
Слика 16 Филтрирање возила – приказ почетне листе возила

Основни сценарио СК

1. **Клијент бира** критеријуме на основу којих жели да претражује **возила** (АПУСО)
2. **Клијент позива** систем да прикаже доступна **возила** по задатим критеријумима. (АПСО)

Опис акције: Потребно је кликнути на неки од филтера категорије возила, кликом се позива системска операција `filterByCategory(category, List<Car>)`.

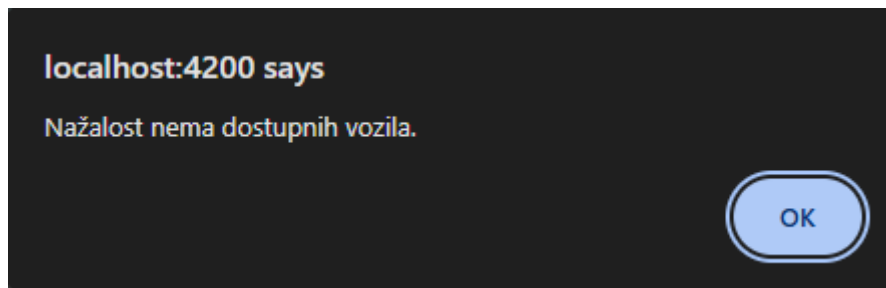
3. **Систем тражи** доступна возила по задатим критеријумима. (СО)
4. **Систем приказује** кориснику доступна возила. (ИА)



Слика 17 Филтрирање возила – приказ филтриране листе возила

Алтернативна сценарија

- 4.1. Уколико **систем** не може да нађе **возила**, приказује **клијенту** поруку: „Nažalost nema dostupnih vozila." (ИА)



Слика 18 Филтрирање возила – алтернативна сценарија 4.1.

СК4: Случај коришћења – Креирање возила

Назив СК

Креирање возила

Актори СК

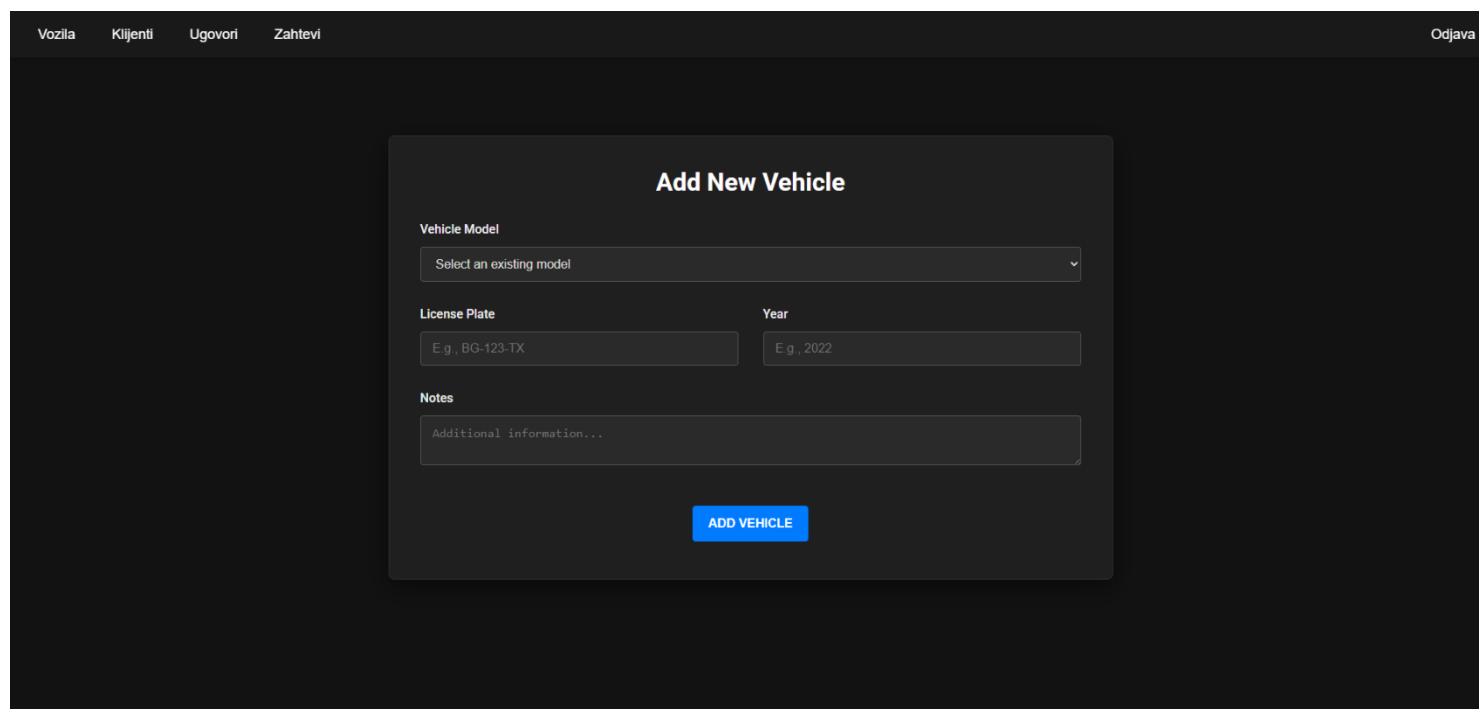
Администратор

Учесници СК

Администратор и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и администратор је пријављен под својом шифром. Форма за креирање возила је приказана. Учитана је листа модела возила.



The screenshot shows a web application interface with a dark theme. At the top, there is a navigation bar with links: 'Vozila', 'Klijenti', 'Ugovori', 'Zahtevi', and 'Odjava'. The main content area features a central form titled 'Add New Vehicle'. The form contains the following fields:

- Vehicle Model:** A dropdown menu with the placeholder text 'Select an existing model'.
- License Plate:** A text input field with the placeholder text 'E.g., BG-123-TX'.
- Year:** A text input field with the placeholder text 'E.g., 2022'.
- Notes:** A text area with the placeholder text 'Additional information...'.

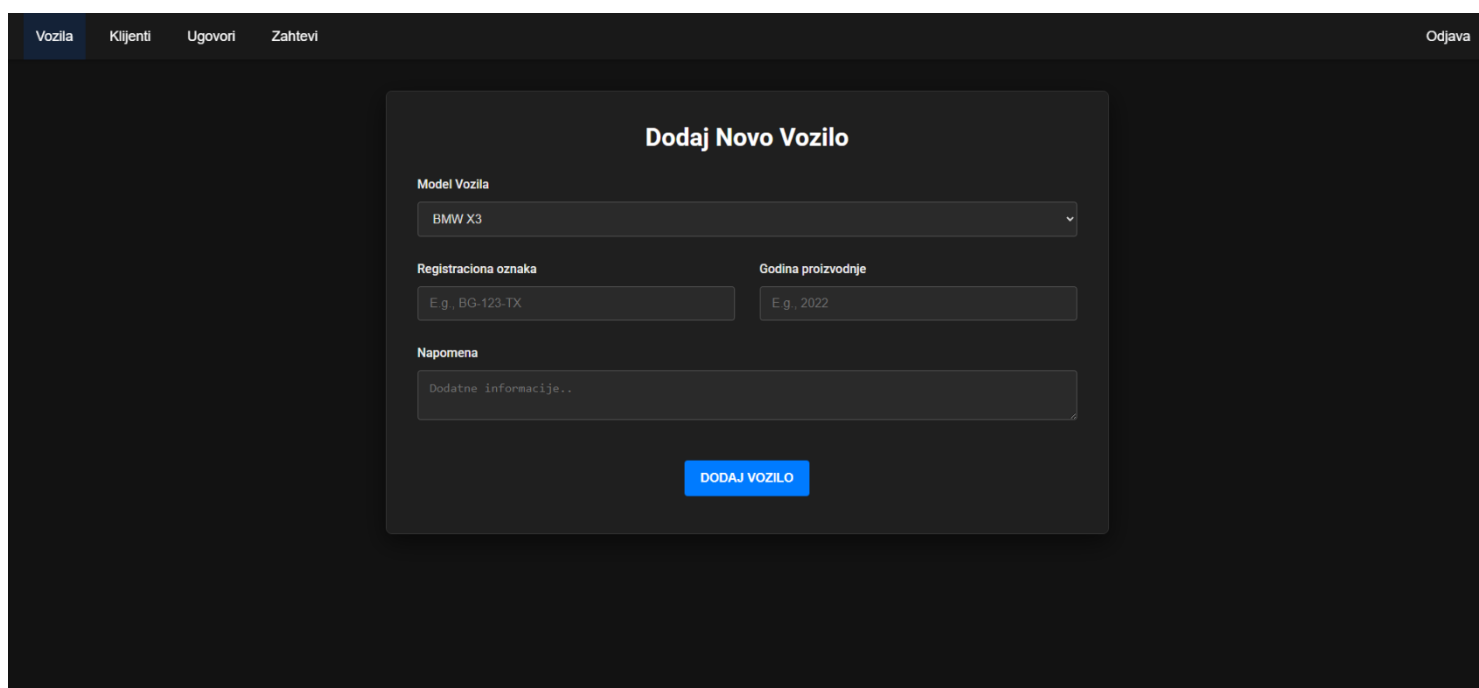
At the bottom of the form is a blue button labeled 'ADD VEHICLE'.

Слика 19 Креирање возила – пошетна форма за креирање возила

Основни сценарио СК

1. **Администратор уноси** податке за креирање **возила**. (АПУСО)
2. **Администратор контролише** да ли је коректно унео податке за креирање **возила**. (АНСО)
3. **Администратор позива систем** да креира **возило** са задатим вредностима. (АПСО)

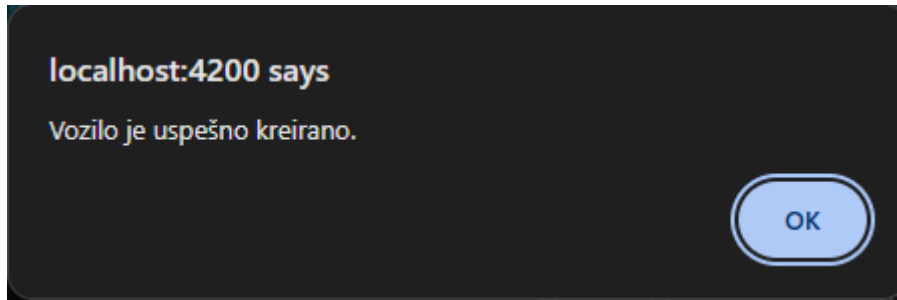
Опис акције: Након уноса потребних податка о возилу, потребно је кликнути на дугме "DODAJ VOZILO" чиме се позива системска операција `addNewCar(Car)`.



The screenshot shows a web application interface with a dark theme. At the top, there is a navigation bar with links: 'Vozila', 'Klijenti', 'Ugovori', 'Zahtevi', and 'Odjava'. The main content area features a form titled 'Dodaj Novo Vozilo'. The form contains the following fields: a dropdown menu for 'Model Vozila' with 'BMW X3' selected; two input fields for 'Registraciona oznaka' (containing 'E.g., BG-123-TX') and 'Godina proizvodnje' (containing 'E.g., 2022'); and a text area for 'Napomena' with the placeholder 'Dodatne informacije...'. A blue button labeled 'DODAJ VOZILO' is positioned at the bottom of the form.

Слика 20 Креирање возила – попуњена форма за креирање возила

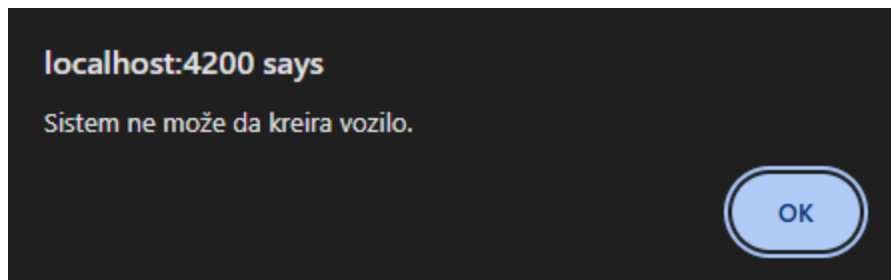
4. Систем креира возило. (CO)
5. Систем приказује администратору креирано возило и поруку: „Vozilo je uspešno kreirano.” (ИА)



Слика 21 Креирање возила – успешно креирано возило

Алтернативна сценарија

- 5.1. Уколико **систем** не може да креира **возило**, приказује **администратору** поруку:
„**Sistem** ne može da kreira **vozilo**.” (ИА)



Слика 22 Креирање возила – алтернативна сценарија 5.1.

СК5: Случај коришћења – Брисање возила

Назив СК

Брисање возила

Актори СК

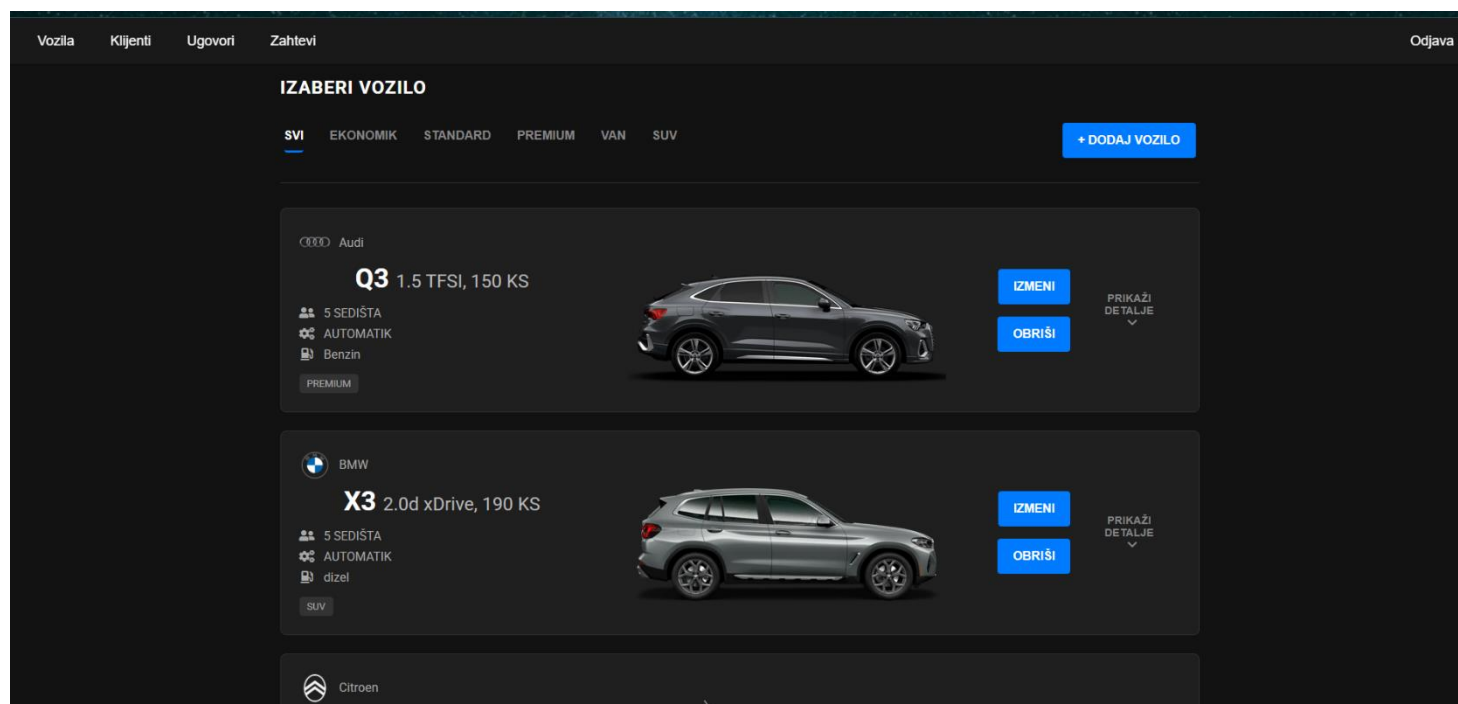
Администратор

Учесници СК

Администратор и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и администратор је пријављен под својом шифром. Учитана је листа возила.



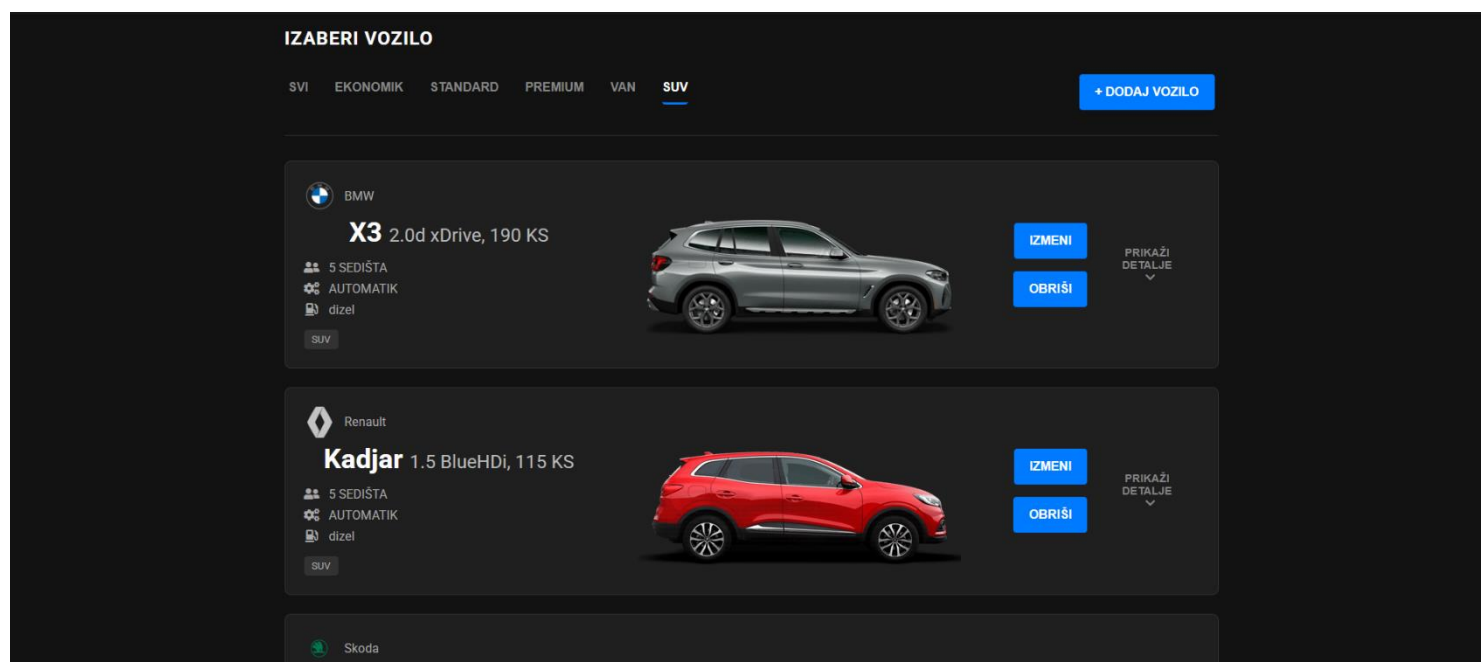
слика 23 Брисање возила – почетна страница са листом возила

Основни сценарио СК

1. **Администратор бира** критеријуме на основу којих жели да претражује возила (АПУСО)
2. **Администратор позива** систем да прикаже доступна **возила** по задатим критеријумима. (АПСО)

Опис акције: Потребно је кликнути на неки од филтера категорије возила, кликом се позива системска операција `filterByCategory(category, List<Car>)`.

3. **Систем тражи** доступна **возила** по задатим критеријумима. (СО)
4. **Систем приказује администратору** доступна **возила**. (ИА)



Слика 24 Брисање возила – приказана листа доступних возила по задатим критеријумима

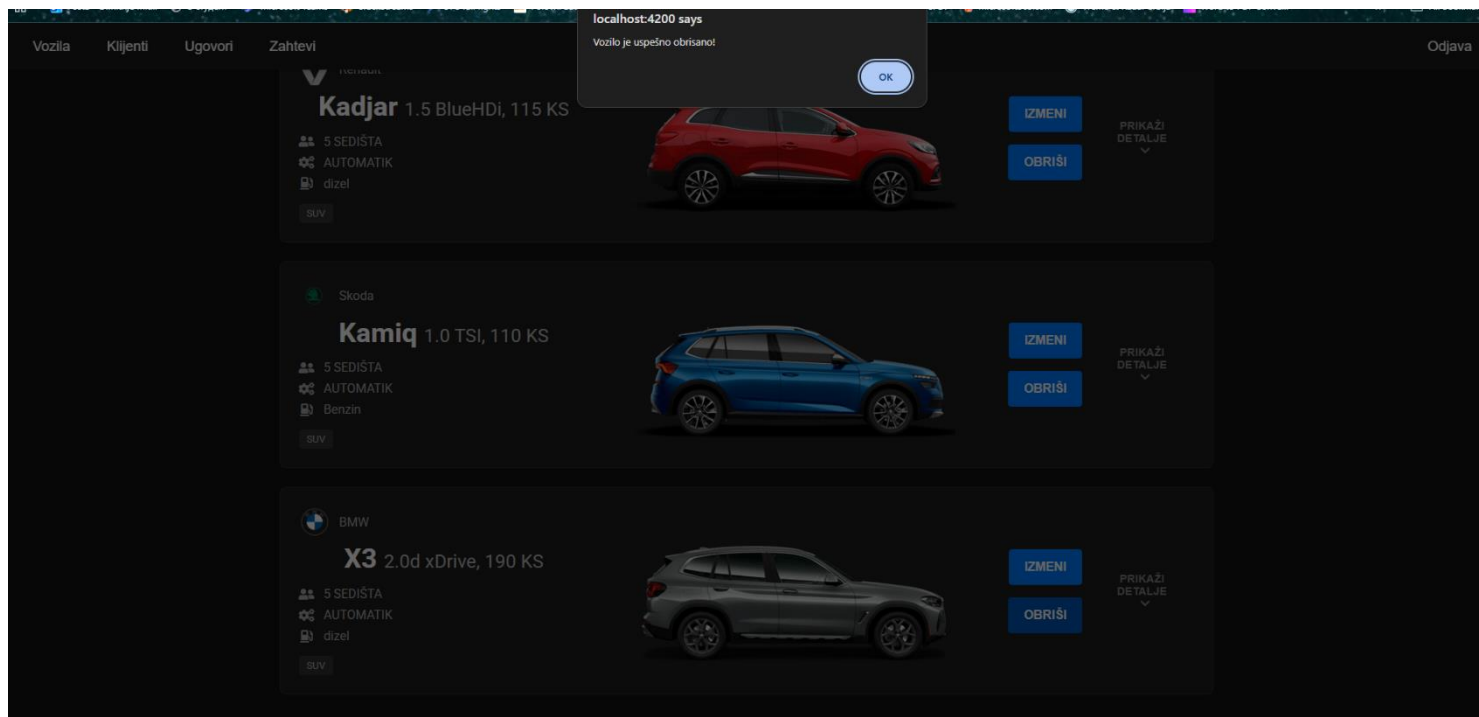
5. **Администратор бира возило** која жели да избрише. (АПУСО)

6. **Администратор позива систем** да избрише **возило**. (АПСО)

Опис акције: Након избора жељеног возила, потребно је кликнути на дугме "OBRIŠI" чиме се позива системска операција deleteCar(CarId, Car)

7. **Систем брише возило**. (СО)

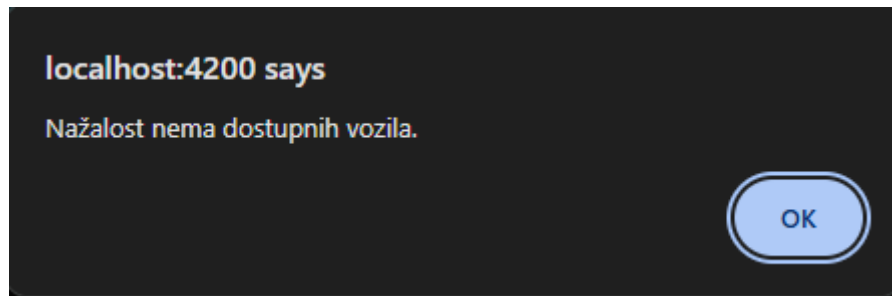
8. **Систем приказује администратору** поруку: „Vozilo je uspešno obrisano.” (ИА)



Слика 25 Брисање возила – успешно брисање возила

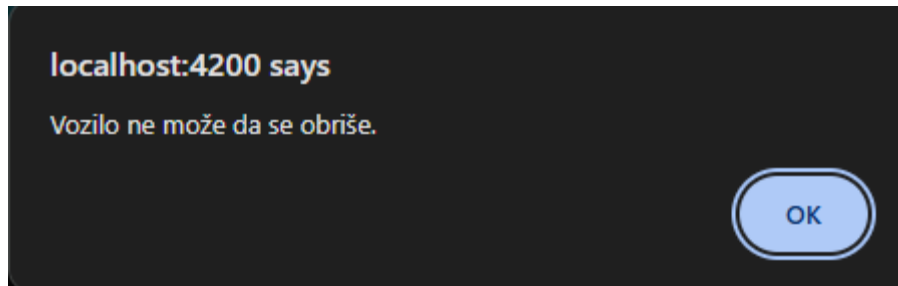
Алтернативна сценарија

- 4.1. Уколико **систем** не може да пронађе доступна возила, приказује поруку о грешци: „Nažalost nema dostupnih vozila.“. Прекида се извршење сценарија. (ИА)



Слика 27 Брисање возила – алтернативна сценарија 4.1.

- 8.1. Уколико **систем** не може да избрише возило, приказује **администратору** поруку: „Vozilo ne može da se obriše.” (ИА)



Слика 28 Брисање возила – алтернативна сценарија 8.1.

СК6: Случај коришћења – Измена возила

Назив СК

Измена возила

Актори СК

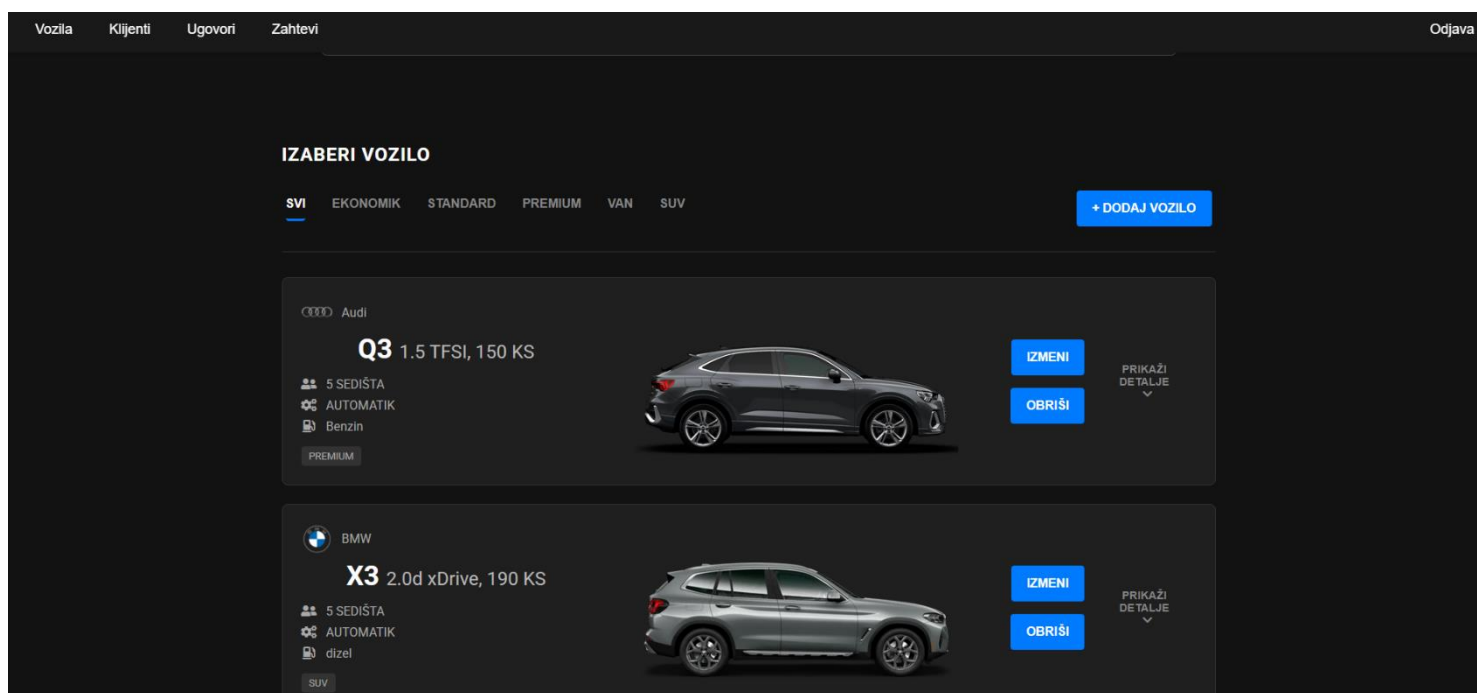
Администратор

Учесници СК

Администратор и систем (програм)

Предуслов

Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути и администратор је пријављен под својом шифром. Приказана је форма за приказ возила која позива систем да учита листу возила. Учитана је листа возила и модела возила.



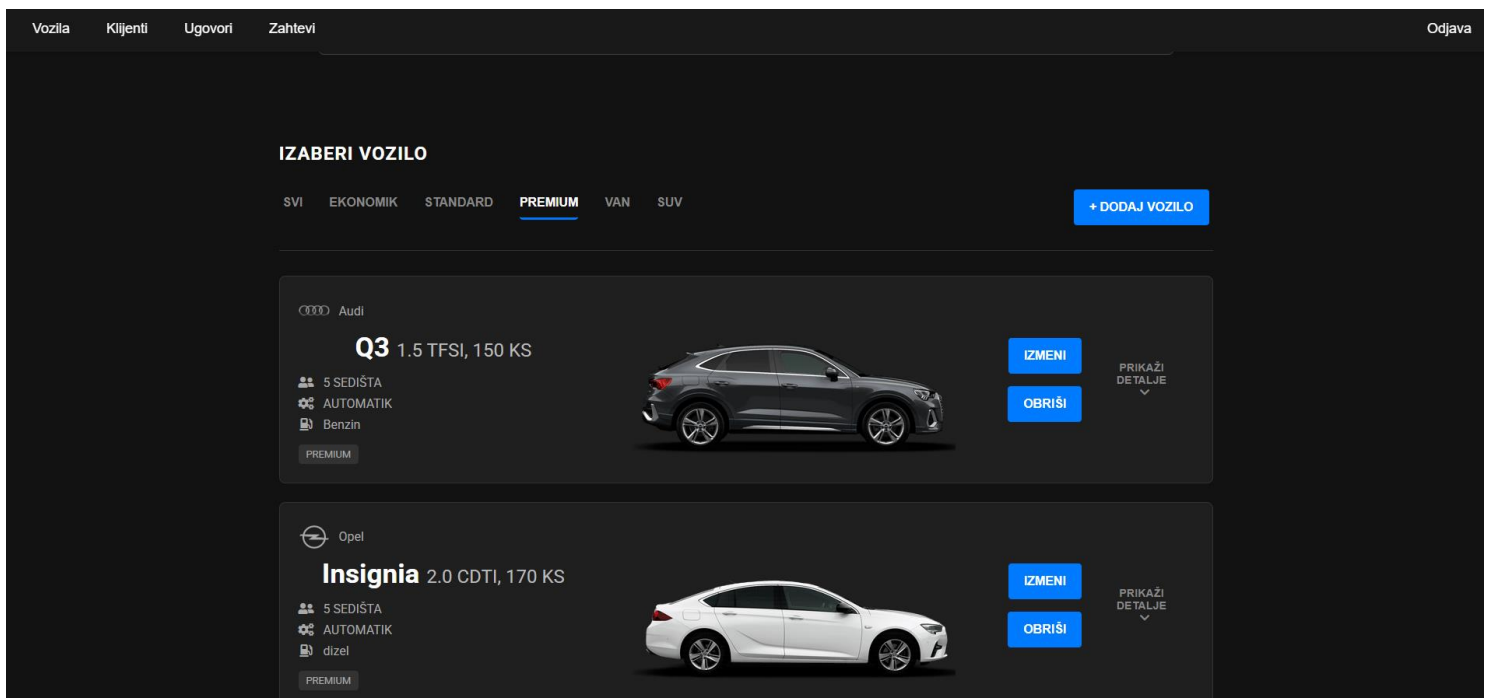
слика 29 Измена возила – почетна страница са листом возила

Основни сценарио СК

1. **Администратор бира** критеријуме на основу којих жели да претражује возила (АПУСО)
2. **Администратор позива** систем да прикаже доступна **возила** по задатим критеријумима. (АПСО)

Опис акције: Потребно је кликнути на неки од филтера категорије возила, кликом се позива системска операција `filterByCategory(category, List<Car>)`.

3. **Систем тражи** доступна **возила** по задатим критеријумима. (СО)
4. **Систем приказује** кориснику доступна **возила**. (ИА)

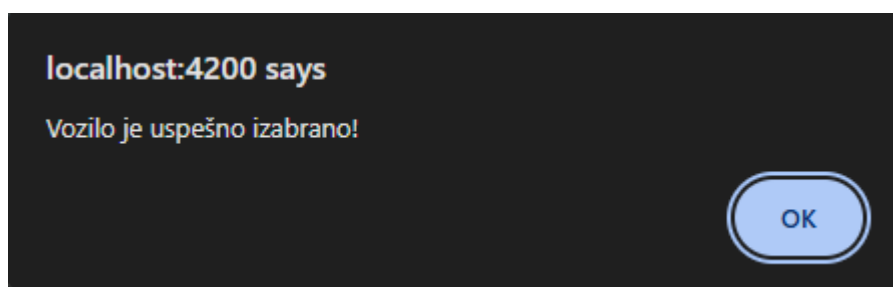


Слика 30 Измена возила – листа возила по задатим критеријумима

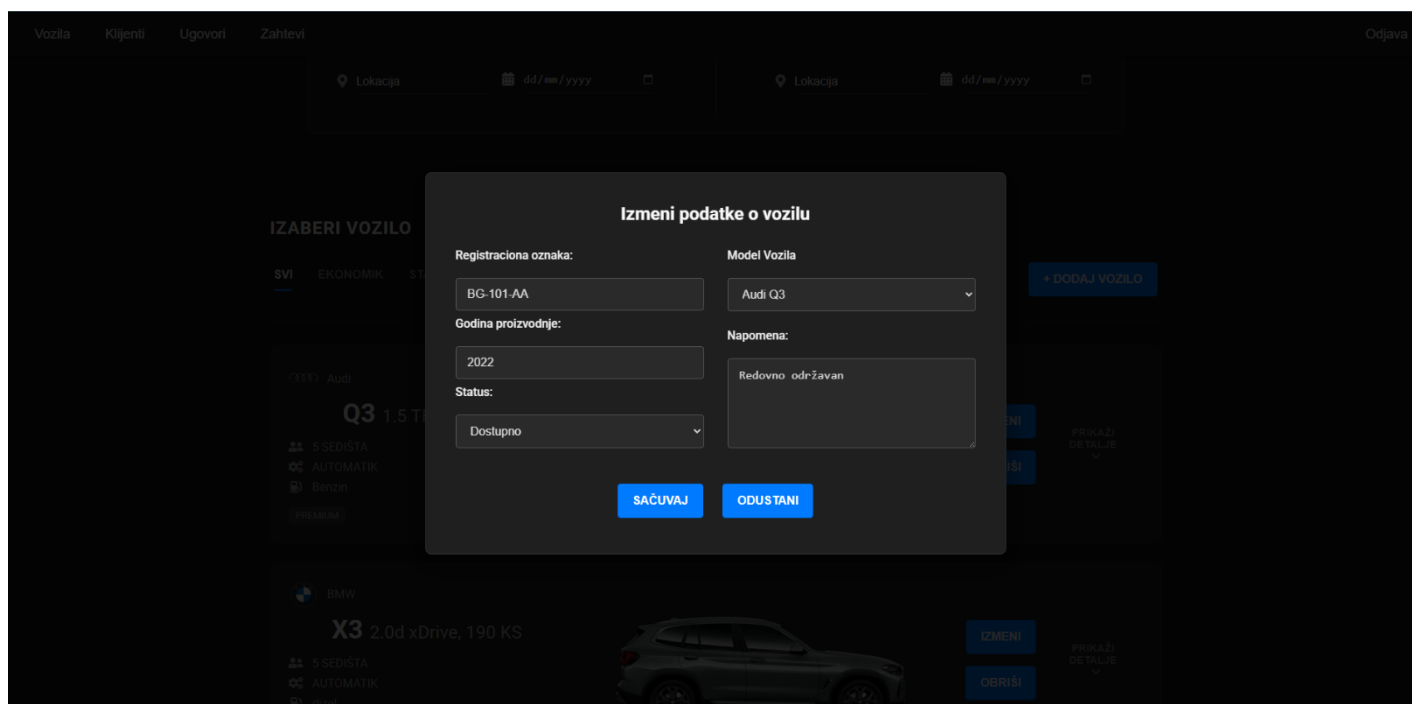
5. **Администратор бира возило** које жели да измени. (АПУСО)
6. **Администратор позива** систем да учита изабрано **возило**. (АПСО)

Опис акције: Након избора жељеног возила измену потребно је кликнути на дугме "IZMENI", кликом се позива систем да отвори форму за измену.

7. **Систем учитава возило** за измену. (СО)
8. **Систем приказује администратору возило** за измену и поруку „Vozilo je uspešno izabrano.” (ИА)



Слика 31 Измена возила – успешно изабрано возило



Слика 32 Измена возила – приказ изабраног возила

9. Администратор мења податке о возилу. (АПУСО)

The screenshot shows a web application interface for managing vehicles. A modal window titled "Izmeni podatke o vozilu" is open, allowing an administrator to edit vehicle information. The background shows a list of vehicles, including an Audi Q3 1.5 TFSI and a BMW X3 2.0d xDrive, 190 KS. The modal form contains the following fields:

- Registraciona oznaka:** BG-101-AA
- Model Vozila:** Audi Q3
- Godina proizvodnje:** 2022
- Status:** Dostupno
- Napomena:** Redovno promenjen

Buttons at the bottom of the modal are "SAČUVAJ" (Save) and "ODUSTANI" (Cancel).

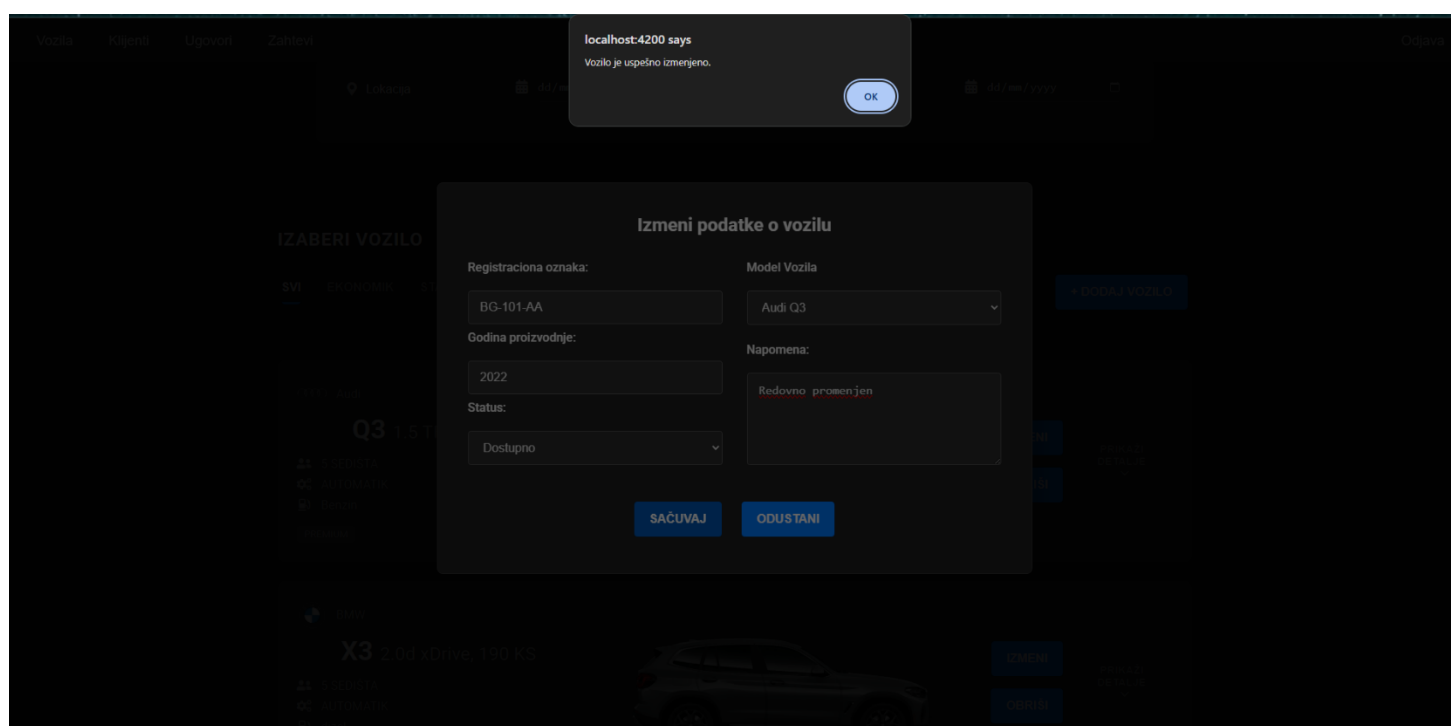
Слика 33 Измена возила – администратор мења податке о возилу

10. **Администратор** **позива** **систем** да сачува измењене податке о возилу. (АПСО)

Опис акције: Након измене података, потребно је кликнути на дугме "SAČUVAJ", кликом се позива системска операција `updateCar(Car)`.

11. **Систем** **чува** измењене податке о возилу. (СО)

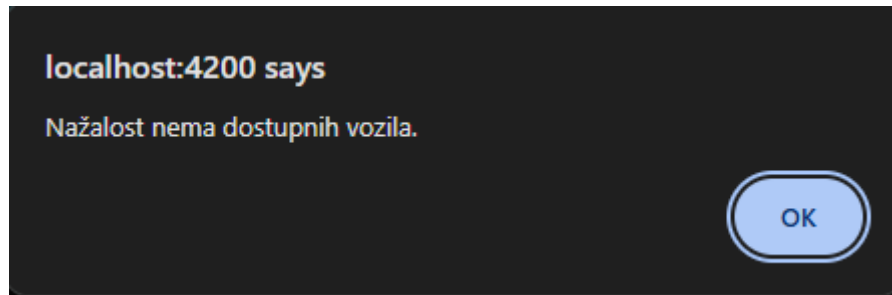
12. **Систем** **приказује** **администратору** измењено возило и поруку: „Vozilo je uspešno izmenjeno.” (ИА)



Слика 34 Измена возила – успешно измењено возило

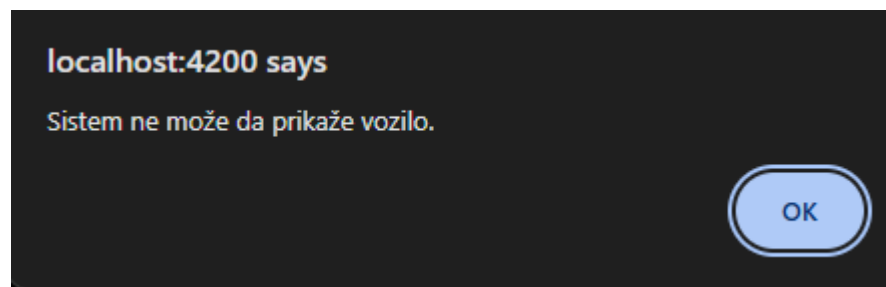
Алтернативна сценарија

- 4.1. Уколико **систем** не може да пронађе доступна возила, приказује поруку о грешци: „Nažalost nema dostupnih vozila.“. Прекида се извршење сценарија. (ИА)



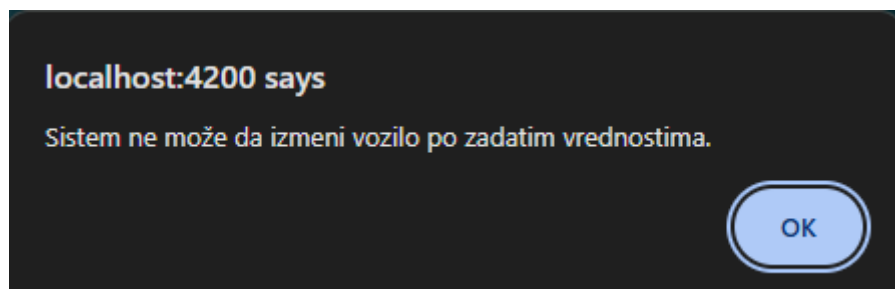
Слика 35 Измена возила – алтернативна сценарија 4.1.

- 8.1. Уколико **систем** не може да прикаже возило за измену, приказује поруку о грешци: „Sistem ne može da prikaže vozilo.“. Прекида се извршење сценарија. (ИА)



Слика 36 Измена возила – алтернативна сценарија 8.1.

- 12.1. Уколико **систем** не може да измени **возило**, приказује **администратору** поруку: „**Sistem** ne može da izmeni **vozilo** po zadatim vrednostima.” (ИА)



Слика 37 Измена возила – алтернативна сценарија 12.1.

СК7: Случај коришћења – Креирање уговора о изнајмљивању

Назив СК

Креирање уговора о изнајмљивању

Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов

Систем је укључен и администратор је пријављен под својом шифром. Форма за креирање уговора о изнајмљивању је приказана. Учитана је листа клијента и листа возила.

The screenshot shows a web application interface with a dark theme. At the top, there is a navigation bar with links: 'Vozila', 'Klijenti', 'Ugovori', 'Zahtevi', and 'Odjava'. The main content area features a form titled 'Kreiraj Ugovor o Iznajmljivanju'. The form contains several input fields: 'Datum početka' (Start Date) and 'Datum završetka' (End Date), both with date pickers showing 'dd/mm/yyyy'; 'Klijent' (Client) and 'Vozilo' (Vehicle), both with dropdown menus showing 'Izaberite klijenta' and 'Izaberite vozilo' respectively; and a 'Napomene' (Notes) section with a text area containing the placeholder 'Dodatne informacije...'. At the bottom of the form is a blue button labeled 'KREIRAJ UGOVOR'.

слика 38 Креирање уговора о изнајмљивању – почетна форма за креирање уговора о изнајмљивању

Основни сценарио СК

1. **Администратор уноси** податке за креирање уговора о изнајмљивању. (АПУСО)
2. **Администратор контролише** да ли је коректно унео податке за креирање уговора о изнајмљивању. (АНСО)

Vozila Klijenti Ugovori Zahtevi Odjava

Kreiraj Ugovor o Iznajmljivanju

Datum početka 11/10/2025 Datum završetka 16/10/2025

Klijent Milan Dimic Vozilo Rapid (BG-120-AA)

Napomene asdasdasd

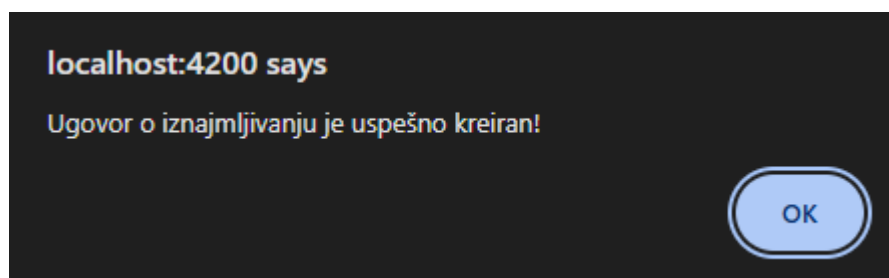
KREIRAJ UGOVOR

Слика 39 Креирање уговора о изнајмљивању – попуњена форма за креирање уговора о изнајмљивању

3. **Администратор** **позива** **систем** да креира уговор о изнајмљивању са задатим вредностима. (АПСО)

Опис акције: Након уноса података о уговору, потребно је кликнути на дугме "KREIRAJ UGOVOR" чиме се позива системска операција addNewRental(Rental)

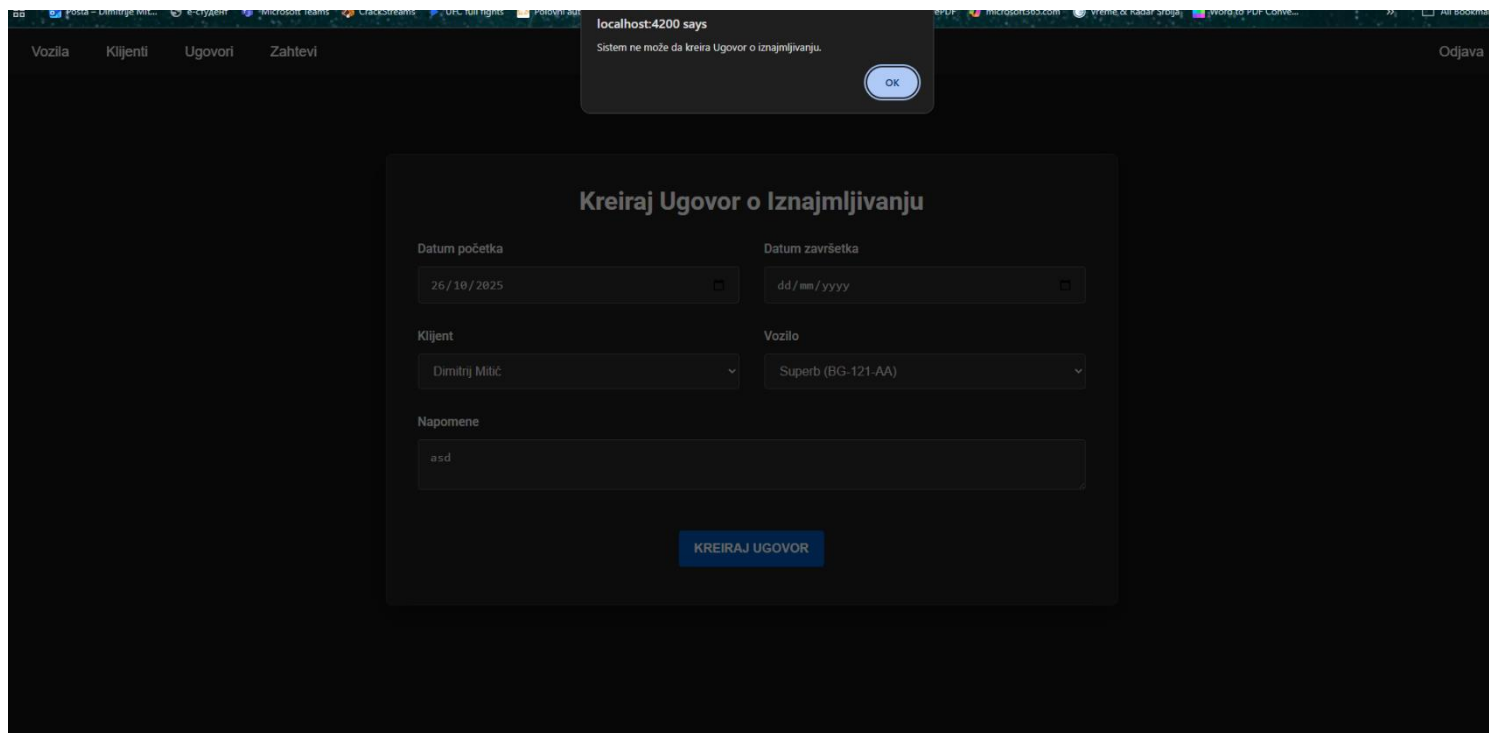
4. **Систем** **креира** уговор о изнајмљивању. (СО)
5. **Систем** **приказује** **администратору** креиран уговор о изнајмљивању и поруку: „Ugovor o iznajmljivanju je uspešno kreiran.” (ИА)



*Слика 40 Креирање уговора о изнајмљивању
– успешно креиран уговор о изнајмљивању*

Алтернативна сценарија

- 5.1. Уколико **систем** не може да креира **уговор о изнајмљивању**, приказује **администратору** поруку: „**Sistem** ne može da kreira ugovor o iznajmljivanju.” (ИА)



Слика 41 Креирање уговора о изнајмљивању – алтернативна сценарија 5.1.

СК8: Случај коришћења – Брисање уговора о изнајмљивању

Назив СК

Брисање уговора о изнајмљивању

Актори СК

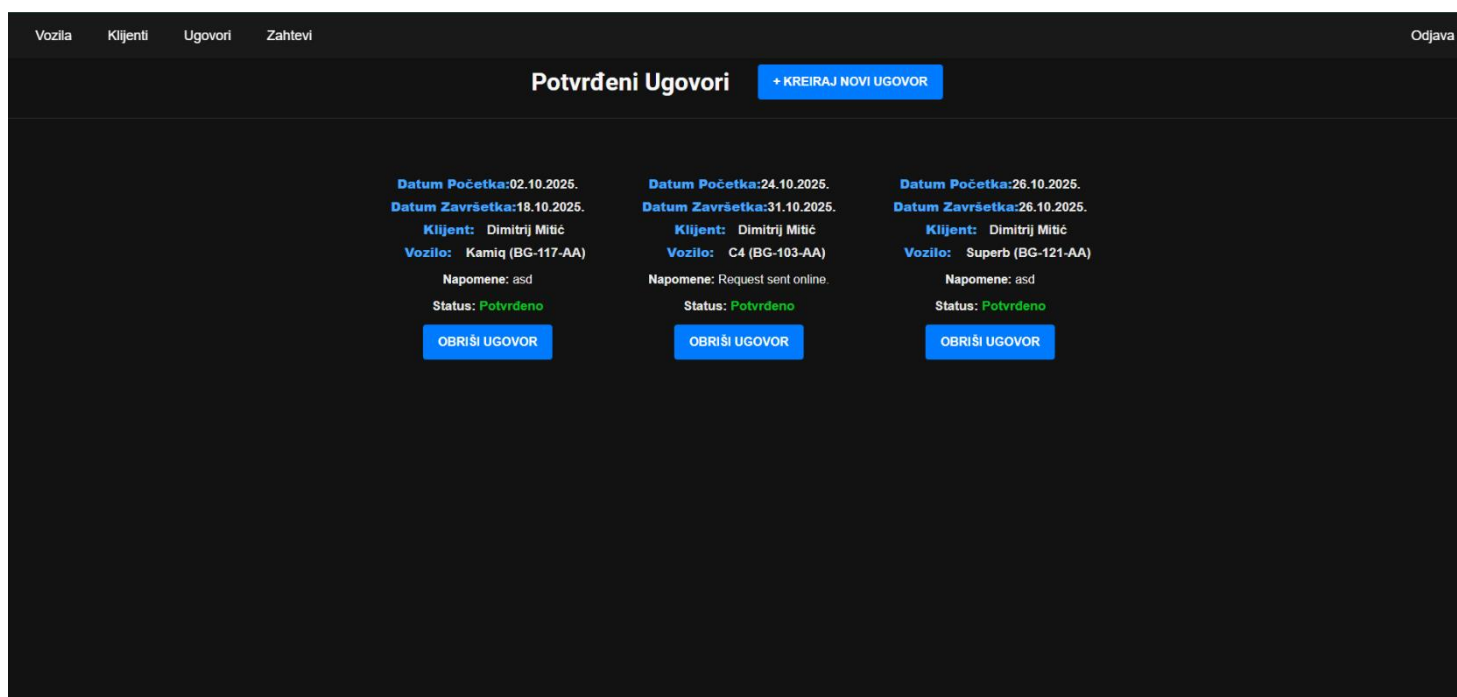
Администратор

Учесници СК

Администратор и систем (програм)

Предуслов

Систем је укључен и администратор је пријављен под својом шифром. Приказана је форма за приказ уговора о изнајмљивању. Учитана је листа уговора о изнајмљивању.



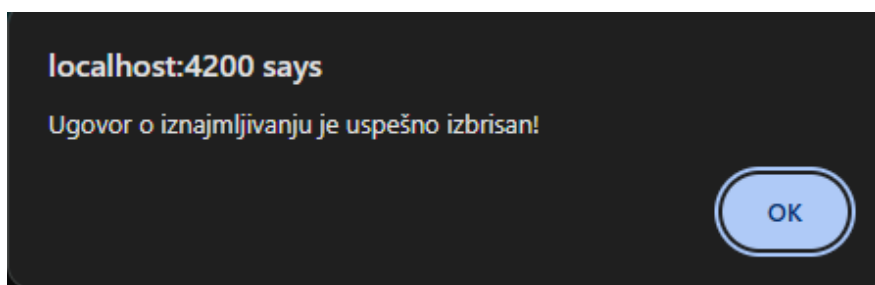
Слика 43 Брисање уговора о изнајмљивању – почетна страница са листом уговора о изнајмљивању

Основни сценарио СК

1. **Администратор бира уговор о изнајмљивању** који жели да избрише. (АПУСО)
2. **Администратор позива систем** да избрише уговор о изнајмљивању са задатим вредностима. (АПСО)

Опис акције: Након избора жељеног уговора, потребно је кликнути на дугме "OBRIŠI UGOVOR" чиме се позива системска операција `deleteRental(RentalId, Rental)`

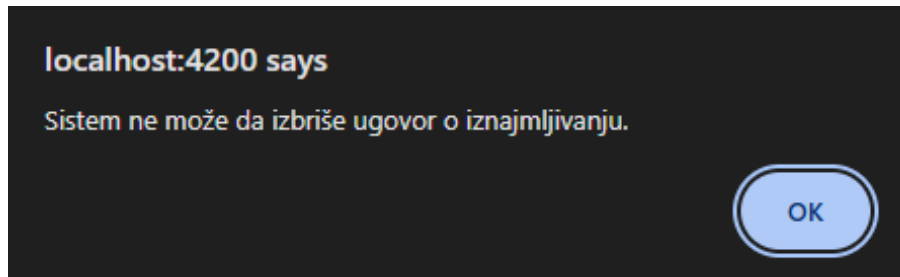
3. **Систем брише уговор о изнајмљивању.** (СО)
4. **Систем приказује администратору поруку:** „Ugovor o iznajmljivanju je uspešno izbrisan!“. (ИА)



*Слика 44 Брисање уговора о изнајмљивању –
успешно избрисан уговор о изнајмљивању*

Алтернативна сценарија

- 4.1. Уколико **систем** не може да избрише уговор о изнајмљивању, приказује **администратору** поруку: „**Sistem** ne može da izbriše ugovor o iznajmljivanju.”. (ИА)



*Слика 45 Брисање уговора о изнајмљивању –
алтернативни сценарија 7.1.*

3.3.2. Пројектовање апликационе логике

Апликациона логика (енгл. *application logic*) представља средњи ниво тронивојске архитектуре. Она обрађује захтеве послате са корисничког интерфејса, примењује пословна правила и комуницира са нивоом базе података [6, 22].

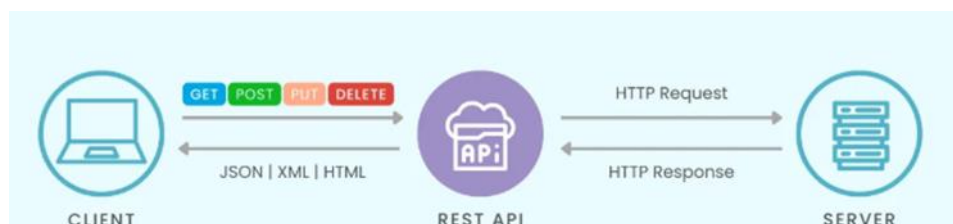
Овај ниво има улогу посредника између корисничког интерфејса и базе података. Апликациона логика осигурава независно пројектовање корисничког интерфејса, пословних правила и базе података што омогућава лакше одржавање, надоградњу и тестирање апликације, као и увођење нових функционалности без нарушавања постојећег система. Апликациона логика се састоји од: контролера, пословне логике и брокера базе података [6, 22].

Пројектовање ових елемената, како је предвиђено упрошћеном Лармановом методом, доприноси стварању модуларног, скалабилног и лако одрживог система [6].

Комуникација са клијентом

Најчешћи метод комуникације између клијентске апликације и серверске апликације је *REST*. *API* које је заснован на *REST* архитектонском стилу се зове *RESTful API*. У архитектури овог софтверског система, *RESTful API* представља кључни механизам који омогућава размену података између клијентске апликације и серверске апликације. Они омогућавају размену података на јасан, стандардизован и поуздан начин, користећи *HTTP* протокол и формате попут *JSON*-а за пренос података [21] .

Комуникација клијента и сервера приказана је на следећој слици.



Слика 46 Комуникација клијента и сервера преко *REST API* [23]

На слици је приказан принцип комуникације између клијента и сервера који се заснива

на *REST* архитектонском стилу. Клијент шаље *HTTP* захтев серверу користећи методе као што су *GET*, *POST*, *PUT* или *DELETE*, док сервер на основу примљеног захтева обрађује податке и шаље *HTTP* одговор назад. Размена података се најчешће врши у форматима *JSON*, *XML* или *HTML* [23].

Контролер апликационе логике

Контролер представља улаз апликационе логике и има улогу да управља извршавањем *HTTP* захтева и одговора. Контролер је посредник између корисничког интерфејса и пословне логике система. Када прими долазни *HTTP* захтев, контролер прослеђује захтев одговарајућој класи пословне логике и резултат враћа клијенту [6, 24].

У оквиру *Spring Boot* развојног окружења, контролери се најчешће креирају употребом класа означених анотацијом *@RestController*, која омогућава једноставну реализацију *RESTful* веб сервиса и директно враћање података у *JSON* формату [6, 24].

Пословна логика

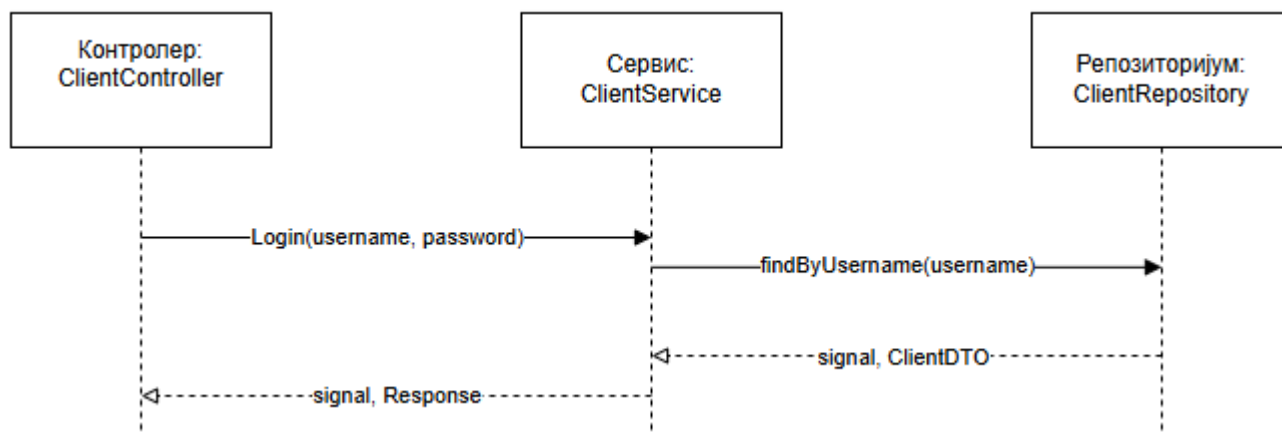
Пословна логика представља скуп правила пословне логике као што су валидација, услови, ограничења и дозволе. Пословна логика је утврђена током фазе анализе, преноси се у фазу пројектовања и постаје део апликационе логике [6, 25].

Уговор УГ1: login(username, password): signal

Веза са СК: СК1

Предуслови: /

Постуслови: Клијент је успешно пријављен.



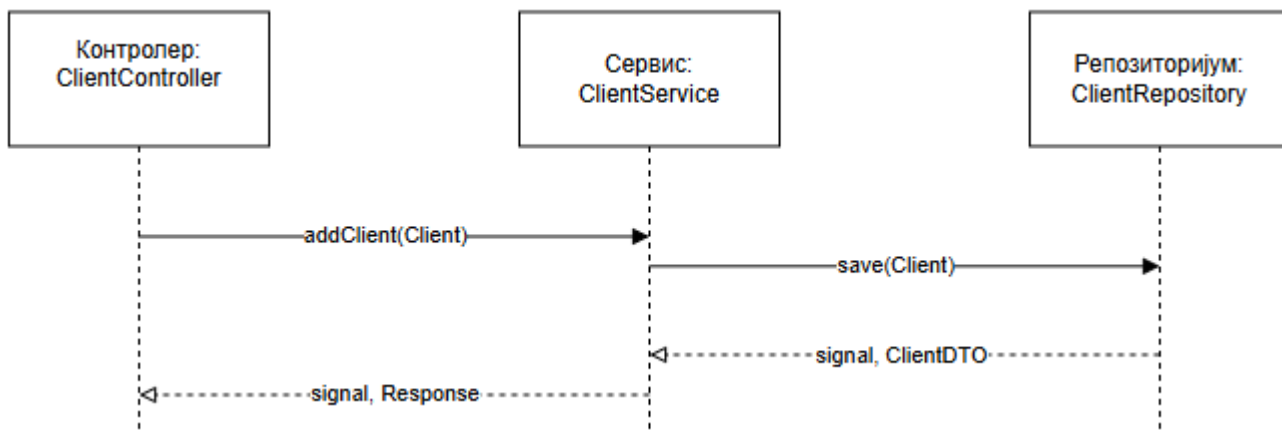
Дијаграм 20 УГ1: login

Уговор УГ2: addNewClient(Client): signal

Веза са СК: СК2

Предуслови: Структурна и вредносна ограничења над објектом типа *Client* морају бити задовољена.

Постуслови: Креиран је нови објекат типа *Client*.



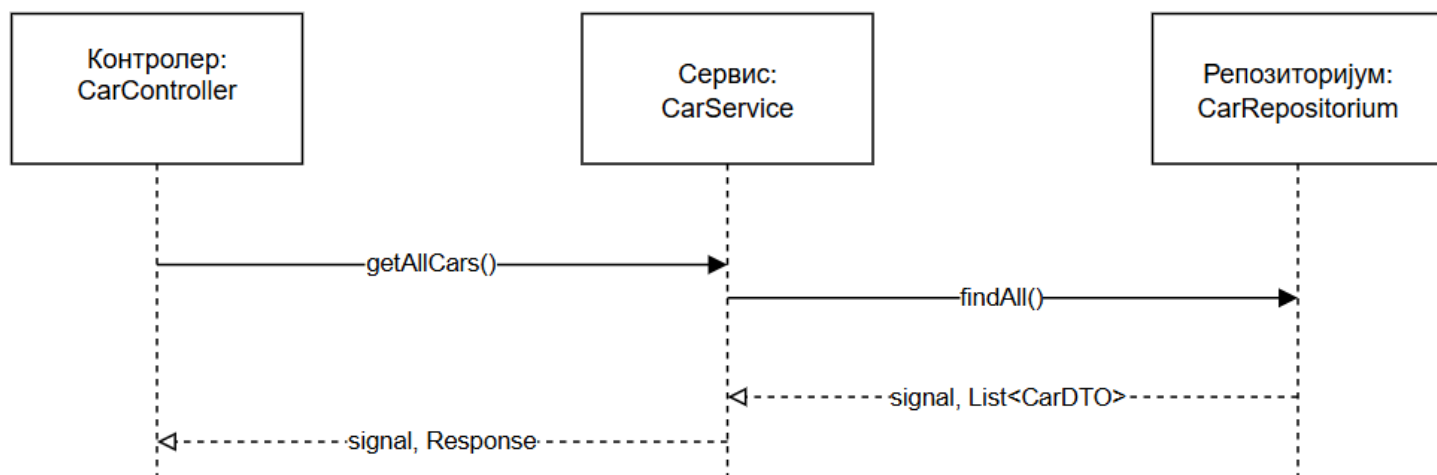
Дијаграм 21 УГ2: addNewClient

Уговор УГ3: getAllCars(List<Car>): signal

Веза са СК: СК3, СК5, СК6, СК7

Предуслови: /

Постуслови: /



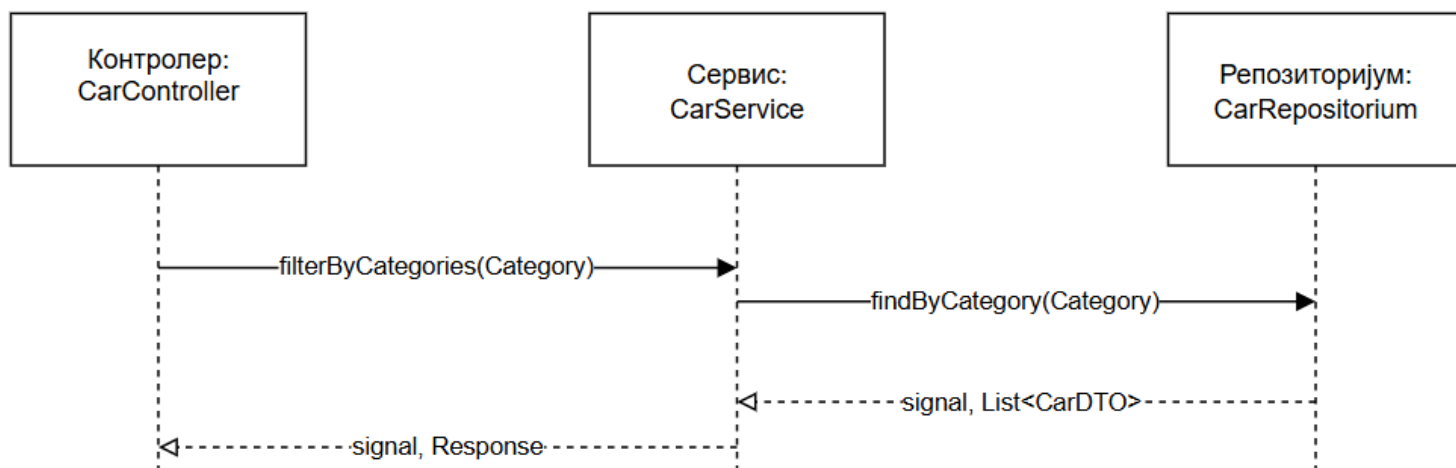
Дијаграм 22 УГ3: getAllCars

Уговор УГ4: filterByCategory(Category, List<Car>): signal

Веза са СК: СК3, СК5, СК6

Предуслови: /

Постуслови: Систем приказује возила која припадају задатој категорији.



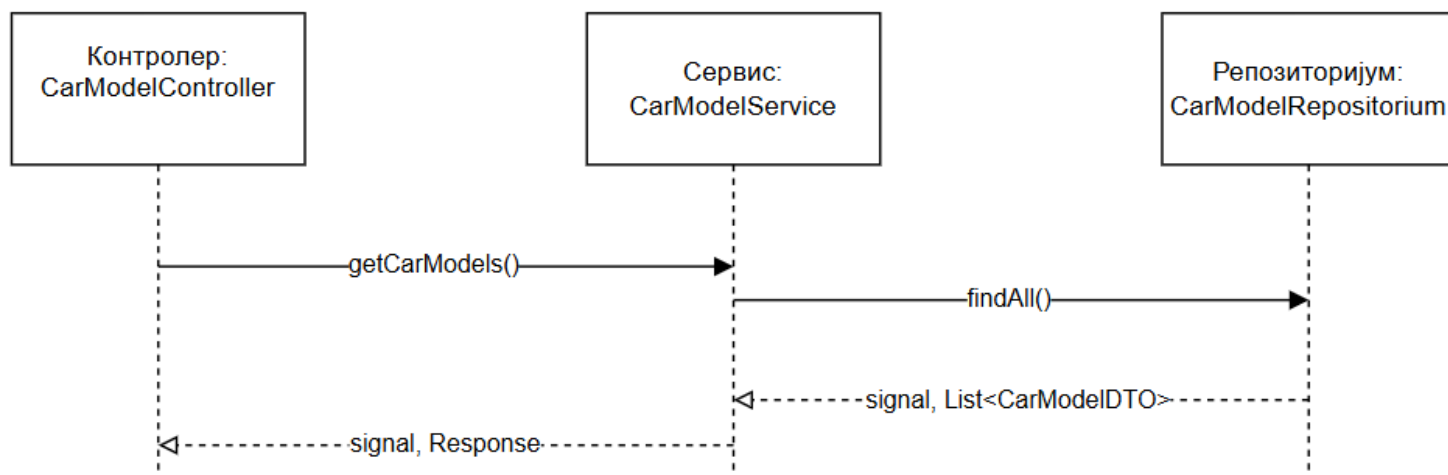
Дијаграм 23 УГ4: filterByCategories

Уговор УГ5: getCarModels(List<CarModel>): signal

Веза са СК: СК4, СК6

Предуслови: /

Постуслови: /



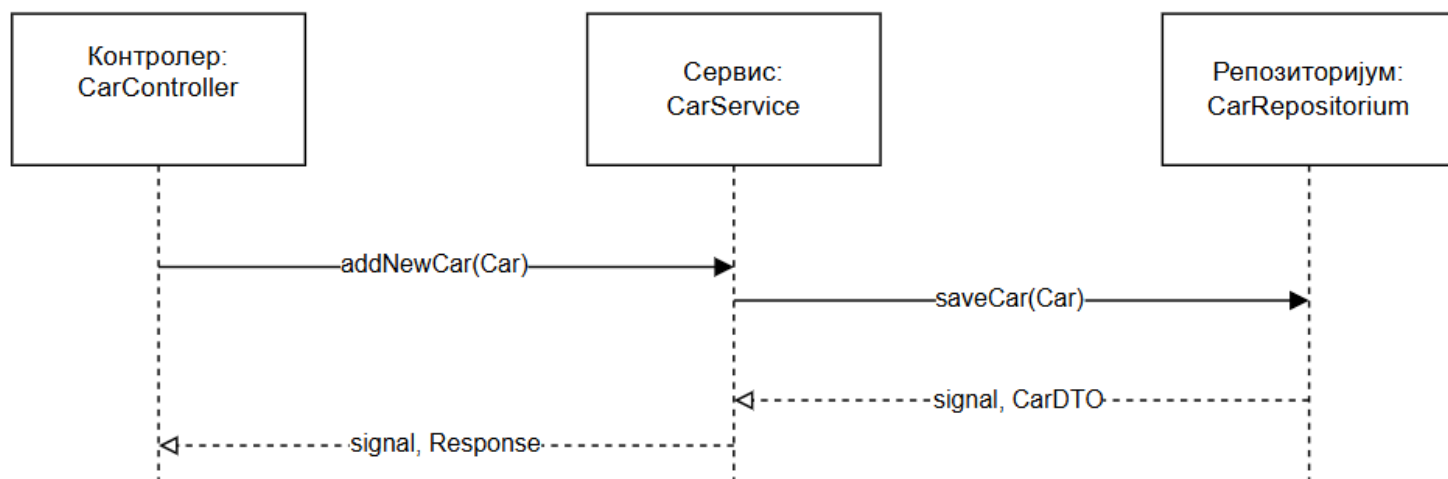
Дијаграм 24 УГ5: getCarModels

Уговор УГ6: addNewCar(Car): signal

Веза са СК: СК4

Предуслови: Структурна и вредносна ограничења над објектом типа *Car* морају бити задовољена.

Постуслови: Креиран је нови објекат типа *Car*.



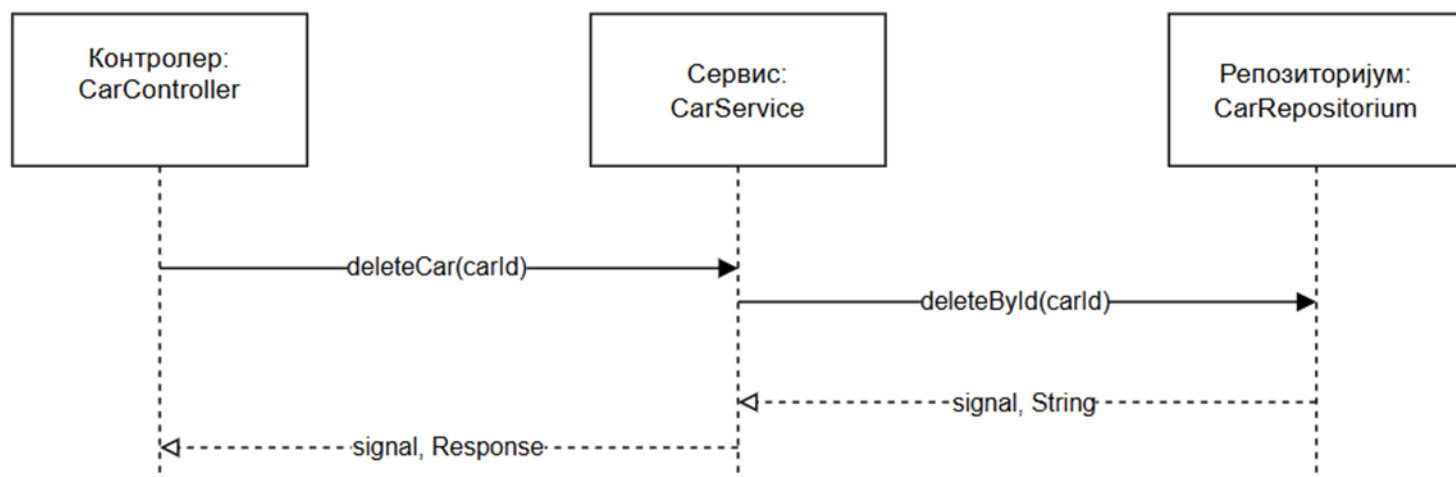
Дијаграм 25 УГ6: addNewCar

Уговор УГ7: deleteCar(CarId, Car): signal

Веза са СК: СК5

Предуслови: Структурна и вредносна ограничења над објектом типа *Car* морају бити задовољена. Возило није у активном уговору о изнајмљивању.

Постуслови: Објекат типа *Car* је обрисан.



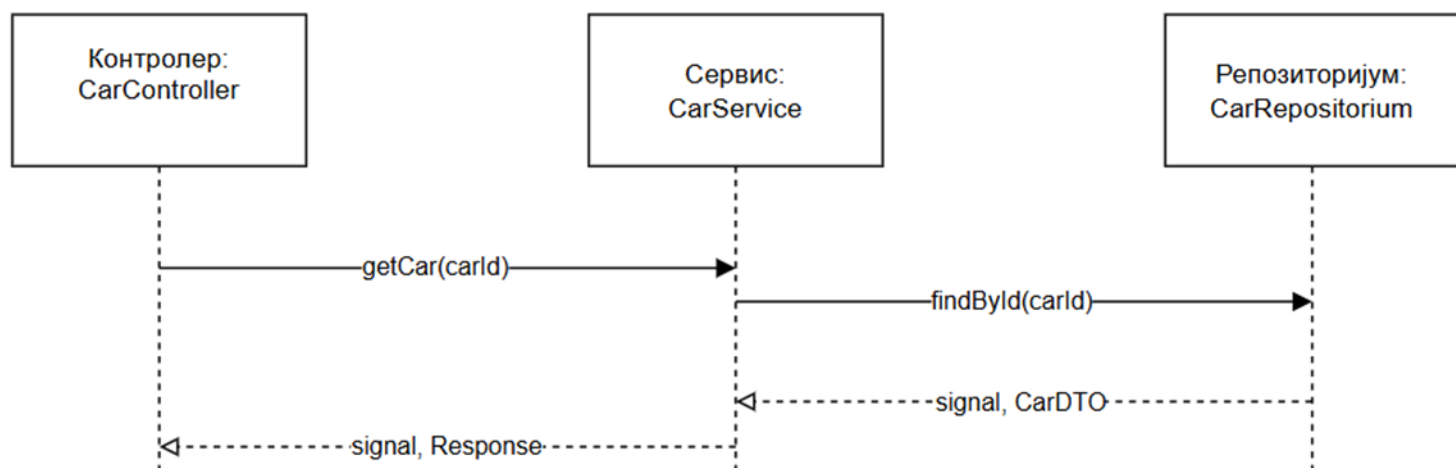
Дијаграм 26 УГ7: deleteCar

Уговор УГ8: getCar(CarId, Car): signal

Веза са СК: СК6

Предуслови: /

Постуслови:



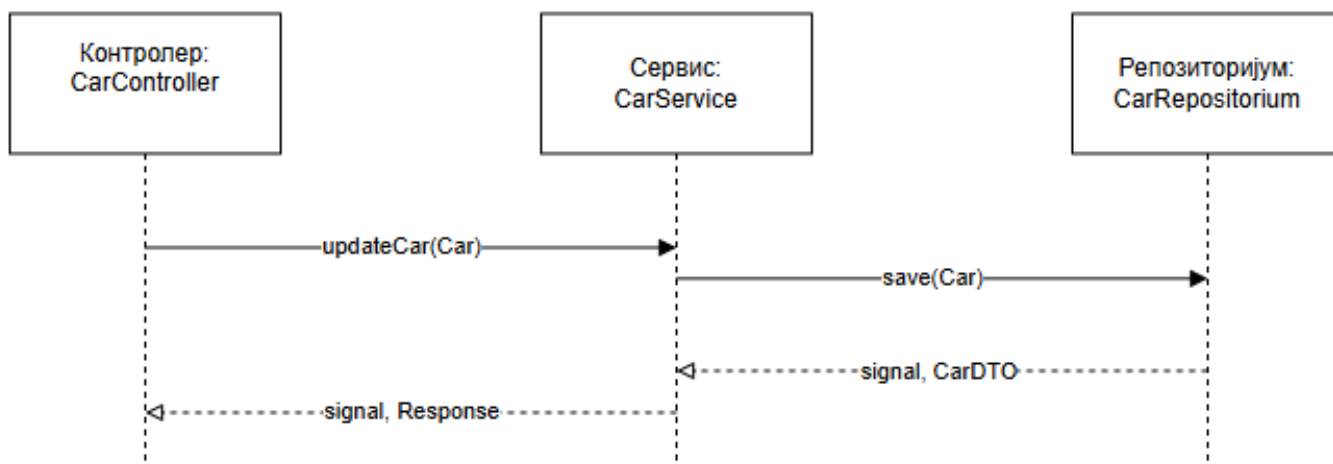
Дијаграм 27 УГ8: getCar

Уговор УГ9: updateCar(Car): signal

Веза са СК: СК6

Предуслови: Структурна и вредносна ограничења над објектом типа *Car* морају бити задовољена

Постуслови: Објекат типа *Car* је измењен.



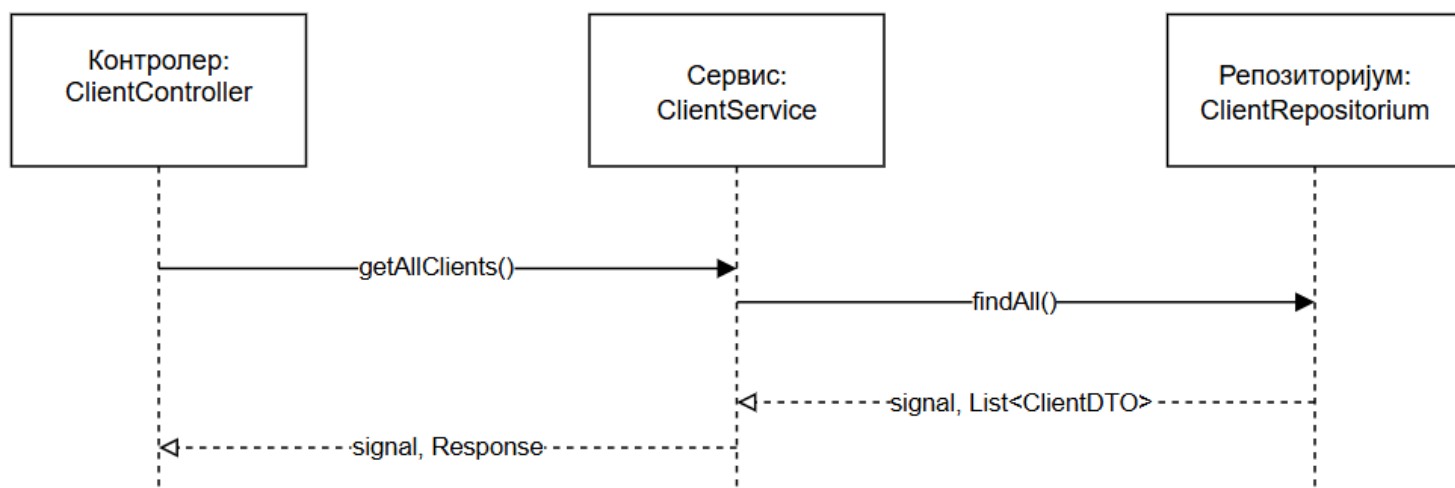
Дијаграм 28 УГ9: updateCar

Уговор УГ10: getAllClients(List<Client>): signal

Веза са СК: СК7

Предуслови: /

Постуслови: /



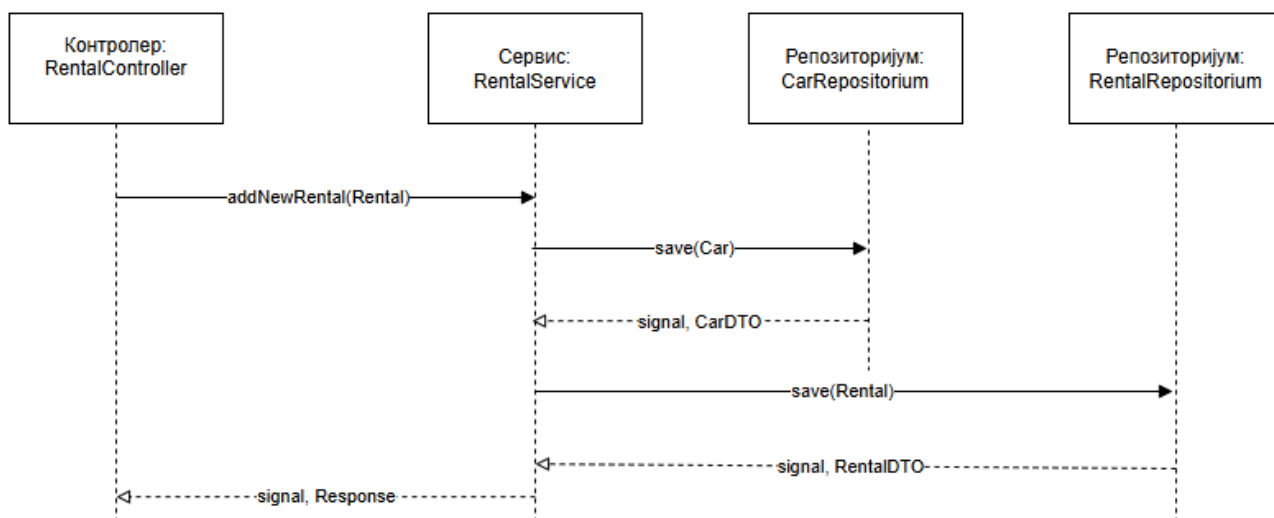
Дијаграм 29 УГ10: getAllClients

Уговор УГ11: addNewRental(Rental): signal

Веза са СК: СК7

Предуслови: Структурна и вредносна ограничења над објектом типа *Rental* морају бити задовољена.

Постуслови: Креиран је нови објекат типа *Rental*. Повезано возило није више доступно за изнајмљивање.



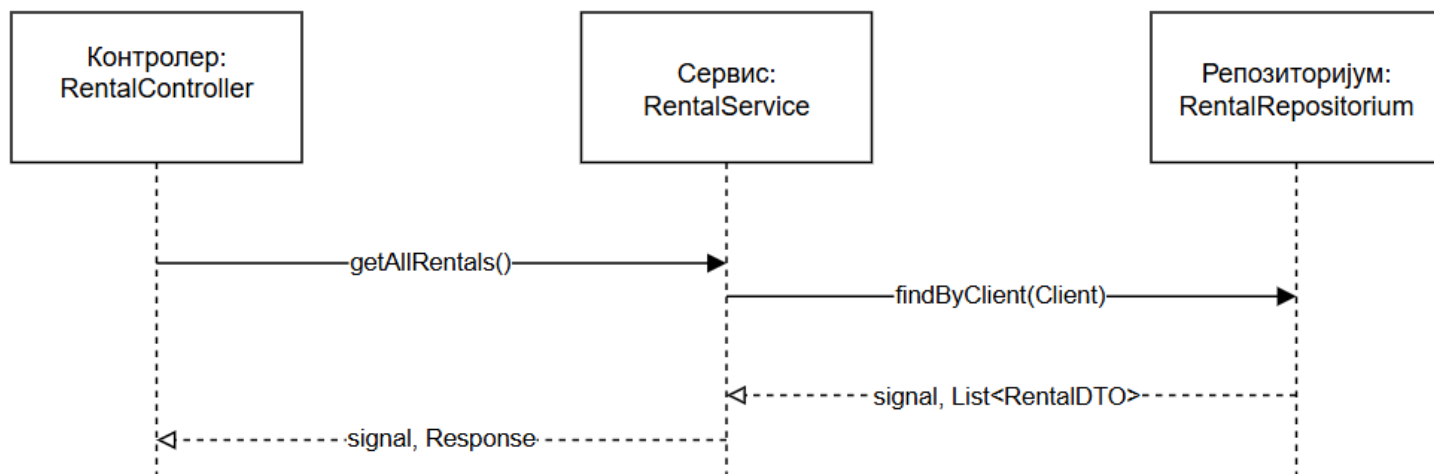
Дијаграм 30 УГ11: addNewRental

Уговор УГ12: getAllRentals(List<Rental>): signal

Веза са СК: СК8

Предуслови: /

Постуслови: /



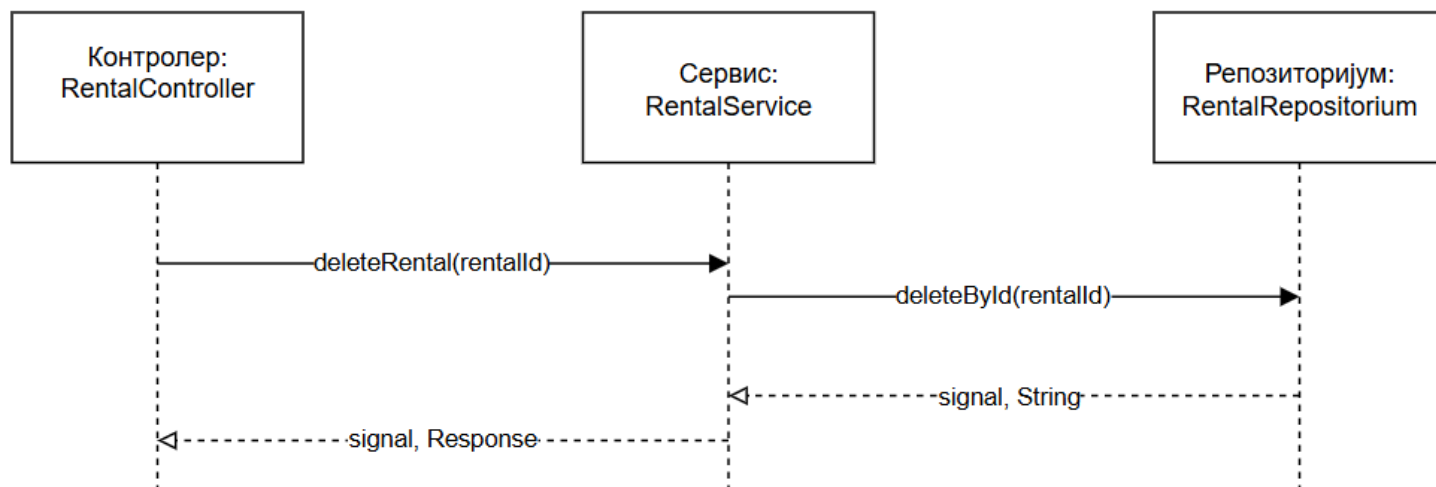
Дијаграм 31 УГ12: getAllRentals

Уговор УГ13: deleteRental(RentalId, Rental): signal

Веза са СК: СК8

Предуслови: Структурна ограничења над објектом типа *Rental* морају бити задовољена.

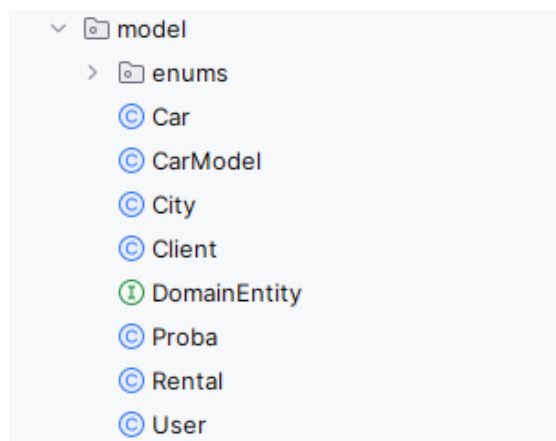
Постуслови: Објекат типа *Rental* је обрисан. Повезано возило постаје доступно за ново изнајмљивање.



Дијаграм 32 УГ13: deleteRental

Пројектовање структуре софтверског система

На основу концептуалног модела, пројектујемо структуру софтверског система кроз доменске класе које представљају ентитете серверске апликације.



слика 47 Доменске класе

У наставку су приказане доменске класе са својим атрибутима и методама.

```
@Entity
public class Car implements Serializable, DomainEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(nullable = false, unique = true)
    private String licensePlate;

    @Column(nullable = false)
    private int year;

    @Column(nullable = false)
    private int status;

    private String notes;

    @ManyToOne(fetch = FetchType.LAZY)
```



```

@JoinColumn(name = "car_model_id", nullable = false)
private CarModel carModel;

public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getLicensePlate() { return licensePlate; }
public void setLicensePlate(String licensePlate) { this.licensePlate =
licensePlate; }
public int getYear() { return year; }
public void setYear(int year) { this.year = year; }
public int getStatus() { return status; }
public void setStatus(int status) { this.status = status; }
public String getNotes() { return notes; }
public void setNotes(String notes) { this.notes = notes; }
public CarModel getCarModel() { return carModel; }
public void setCarModel(CarModel carModel) { this.carModel = carModel; }
}

```

Listing 1 Доменска класа Car

```

@Entity
public class CarModel implements Serializable, DomainEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(nullable = false)
    private String make;
    @Column(nullable = false)
    private String name;
    private String engineDescription;
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private VehicleCategory category;
    @Column(nullable = false)
    private int seats;
}

```

```

@Enumerated(EnumType.STRING)
@Column(nullable = false)
private TransmissionType transmission;

@Enumerated(EnumType.STRING)
@Column(nullable = false)
private FuelType fuelType;
private int doors;
private int luggageCapacity;
private boolean hasAc;

@Column(length = 512)
private String pictureUrl;

@Column(length = 512)
private String logoUrl;

private double priceTier1to3;
private double priceTier4to7;
private double priceTier8to15;
private double priceTier16to30;
private double priceTier31to45;
private double priceTier46plus;

public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getMake() { return make; }
public void setMake(String make) { this.make = make; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getEngineDescription() { return engineDescription; }
public void setEngineDescription(String engineDescription) {
this.engineDescription = engineDescription; }
public VehicleCategory getCategory() { return category; }
public void setCategory(VehicleCategory category) { this.category =
category; }
public int getSeats() { return seats; }
public void setSeats(int seats) { this.seats = seats; }
public TransmissionType getTransmission() { return transmission; }

```

```

    public void setTransmission(TransmissionType transmission) {
this.transmission = transmission; }
    public FuelType getFuelType() { return fuelType; }
    public void setFuelType(FuelType fuelType) { this.fuelType = fuelType; }
    public int getDoors() { return doors; }
    public void setDoors(int doors) { this.doors = doors; }
    public int getLuggageCapacity() { return luggageCapacity; }
    public void setLuggageCapacity(int luggageCapacity) {
this.luggageCapacity = luggageCapacity; }
    public boolean isHasAc() { return hasAc; }
    public void setHasAc(boolean hasAc) { this.hasAc = hasAc; }
    public String getPictureUrl() { return pictureUrl; }
    public void setPictureUrl(String pictureUrl) { this.pictureUrl =
pictureUrl; }
    public String getLogoUrl() { return logoUrl; }
    public void setLogoUrl(String logoUrl) { this.logoUrl = logoUrl; }
    public double getPriceTier1to3() { return priceTier1to3; }
    public void setPriceTier1to3(double priceTier1to3) { this.priceTier1to3
= priceTier1to3; }
    public double getPriceTier4to7() { return priceTier4to7; }
    public void setPriceTier4to7(double priceTier4to7) { this.priceTier4to7
= priceTier4to7; }
    public double getPriceTier8to15() { return priceTier8to15; }
    public void setPriceTier8to15(double priceTier8to15) {
this.priceTier8to15 = priceTier8to15; }
    public double getPriceTier16to30() { return priceTier16to30; }
    public void setPriceTier16to30(double priceTier16to30) {
this.priceTier16to30 = priceTier16to30; }
    public double getPriceTier31to45() { return priceTier31to45; }
    public void setPriceTier31to45(double priceTier31to45) {
this.priceTier31to45 = priceTier31to45; }
    public double getPriceTier46plus() { return priceTier46plus; }
    public void setPriceTier46plus(double priceTier46plus) {
this.priceTier46plus = priceTier46plus; }
}

```

Listing 1 Доменска класа CarModel

```

@Entity
public class City implements Serializable, DomainEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", nullable = false, updatable = false, unique = true)
    private int id;
    @Column(name = "name", nullable = false, updatable = true)
    private String name;

    public City() {
    }
    public City(int id, String name) {
        this.id = id;
        this.name = name;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    @Override
    public String toString() {
        return "City{" +
            "id=" + id +
            ", name='" + name + '\'' +
            '}';
    }
}

```

Listing 2 Доменска класа City

```

@Entity
public class Client implements Serializable, DomainEntity {

    @Id
    @Column(name = "jmbg", nullable = false, updatable = false, unique =
true)
    private String jmbg;
    @Column(name = "name", nullable = false, updatable = true)
    private String name;
    @Column(name = "surname", nullable = false, updatable = true)
    private String surname;
    @Column(name = "age", nullable = false, updatable = true)
    private int age;
    @ManyToOne
    @JoinColumn(name = "city_id", nullable = false, updatable = true)
    private City city;
    @Column(name = "mobile", nullable = false, updatable = true)
    private String mobile;
    @Column(name = "username", nullable = true, updatable = false)
    private String username;
    @Column(name = "password", nullable = true, updatable = false)
    private String password;
    public Client()
    public Client(String jmbg, String name, String surname, int age, City
city, String mobile, String username, String password) {
        this.jmbg = jmbg;
        this.name = name;
        this.surname = surname;
        this.age = age;
        this.city = city;
        this.mobile = mobile;
        this.username = username;
        this.password = password;
    }
    public String getJmbg() {
        return jmbg;
    }
    public void setJmbg(String jmbg) {
        this.jmbg = jmbg;
    }
}

```

```

public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public String getSurname() {
    return surname;
}
public void setSurname(String surname) {
    this.surname = surname;
}
public int getAge() {
    return age;
}
public void setAge(int age) {
    this.age = age;
}
public City getCity() {
    return city;
}
public void setCity(City city) {
    this.city = city;
}
public String getMobile() {
    return mobile;
}
public void setMobile(String mobile) {
    this.mobile = mobile;
}
public String getUsername() {
    return username;
}
public void setUsername(String username) {
    this.username = username;
}
public String getPassword() {
    return password;
}
public void setPassword(String password) {

```

```

        this.password = password;
    }
    @Override
    public String toString() {
        return "Client{" +
            "jmbg='" + jmbg + '\'' +
            ", name='" + name + '\'' +
            ", surname='" + surname + '\'' +
            ", city='" + (city != null ? city.getName() : null) +
            '\'';
    }
}

```

Listing 3 Доменска класа Client

```

@Entity
public class Rental implements Serializable, DomainEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "rental_id", nullable = false, updatable = false, unique =
true)
    private int rentalId;

    @Column(name = "start_date", nullable = false, updatable = true)
    private LocalDate startDate;

    @Column(name = "end_date", nullable = false, updatable = true)
    private LocalDate endDate;

    @Column(name = "notes", nullable = true, updatable = true)
    private String notes;

    @ManyToOne
    @JoinColumn(name = "car_id", nullable = false, updatable = true)
    private Car car;

    @ManyToOne
    @JoinColumn(name = "client_jmbg", nullable = false, updatable = true)

```

```

private Client client;
@Column(name = "status", nullable = false, updatable = true)
private int status;
@ManyToOne
@JoinColumn(name = "user_id", nullable = true, updatable = true)
private User user;
public Rental() {}
public Rental(int rentalId, LocalDate startDate, LocalDate endDate,
String notes, Car car, Client client, int status, User user) {
    this.rentalId = rentalId;
    this.startDate = startDate;
    this.endDate = endDate;
    this.notes = notes;
    this.car = car;
    this.client = client;
    this.status = status;
    this.user = user;
}
public int getRentalId() {
    return rentalId;
}
public void setRentalId(int rentalId) {
    this.rentalId = rentalId;
}
public LocalDate getStartDate() {
    return startDate;
}
public void setStartDate(LocalDate startDate) {
    this.startDate = startDate;
}
public LocalDate getEndDate() {
    return endDate;
}
public void setEndDate(LocalDate endDate) {
    this.endDate = endDate;
}
public String getNotes() {
    return notes;
}
public void setNotes(String notes) {

```



```

        this.notes = notes;
    }
    public Car getCar() {
        return car;
    }
    public void setCar(Car car) {
        this.car = car;
    }
    public Client getClient() {
        return client;
    }
    public void setClient(Client client) {
        this.client = client;
    }
    public int getStatus() {
        return status;
    }
    public void setStatus(int status) {
        this.status = status;
    }
    public User getUser() {
        return user;
    }
    public void setUser(User user) {
        this.user = user;
    }
    @Override
    public String toString() {
        return "Rental{" +
            "rentalId=" + rentalId +
            ", startDate=" + startDate +
            ", endDate=" + endDate +
            ", car=" + (car != null ? car.getLicensePlate() : null) +
            ", client=" + (client != null ? client.getName() : null) +
            ", status=" + status +
            '}';
    }
}

```

Listing 4 Доменска класа Rental

```

@Entity
public class User implements Serializable, DomainEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", nullable = false, updatable = false, unique = true)
    private int id;
    @Column(name = "name", nullable = false)
    private String name;
    @Column(name = "surname", nullable = false)
    private String surname;
    @Column(name = "username", nullable = false)
    private String username;
    @Column(name = "password", nullable = false)
    private String password;
    public User() {}
    public User(int id, String name, String surname, String username, String
password) {
        this.id = id;
        this.name = name;
        this.surname = surname;
        this.username = username;
        this.password = password;
    }
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getSurname() { return surname; }
    public void setSurname(String surname) { this.surname = surname; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    @Override
    public String toString() {
        return "User{" +
            "id=" + id +
            ", name='" + name + '\'' +

```

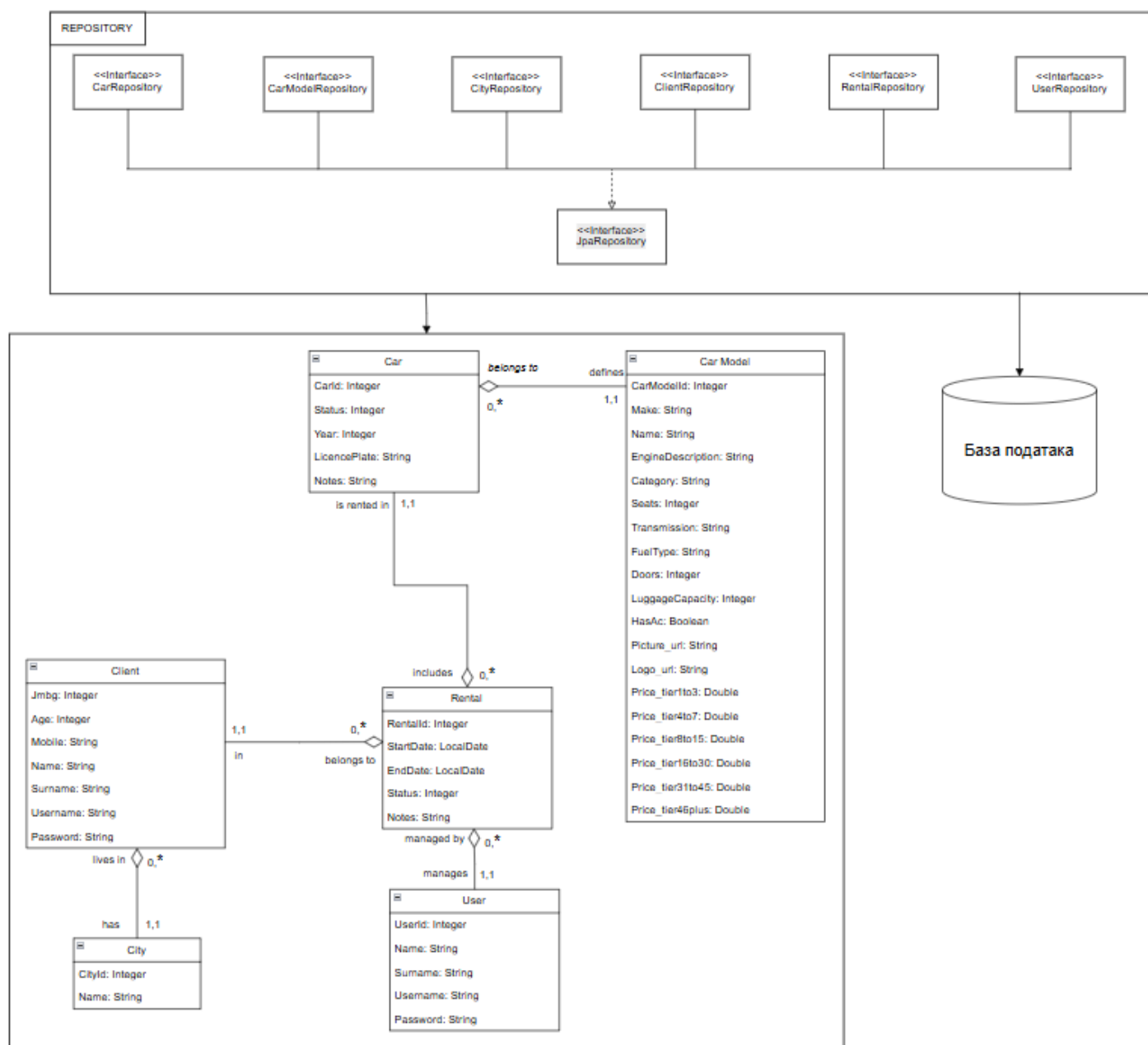
```
        ", surname='" + surname + '\'' +  
        ", username='" + username + '\'' +  
        '}';  
    }  
}
```

Listing 5 Доменска класа User

3.3.4. Пројектовање брокера базе података

Брокер базе података је веза између пословне логике и базе података. У *Spring Boot* систему, за управљање перзистентношћу података, користи се *Spring Data JPA* (*Java Persistence API - JPA*). *JPA* представља *Java* интерфејс за рад са базама података. *JPA* омогућава лаку имплементацију објектно-релационог мапирања (енгл. *Object Relation Mapping - ORM*). *ORM* је процес чувања *Java* објеката директно у табеле базе података. Назив објекта који се чува постане назив табеле, а сваки атрибут постаје колона у табели. Један од најпопуларнијих *ORM* алата је *Hibernate*. *JPA* је интерфејс, *Hibernate* је имплементација интерфејса. Кључна особина *JPA* интерфејса је да извршава *CRUD* операције без уноса *SQL* упита [26].

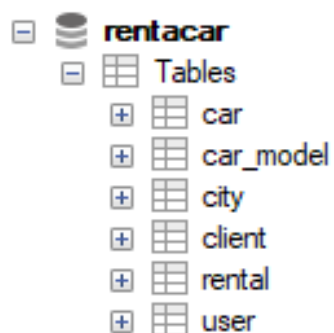
На наредној слици приказана је комуникација система са базом података.



слика 48 Комуникација са базом података

3.3.5. Пројектовање складишта података

На основу доменских класа и релационог модела софтвера, пројектују су табеле базе података. Систем за управљање базом података који је коришћен је *MySQL* [6].



Слика 49 Табеле базе података

☐	Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update
☐	carId	int	11		☒	☒	☐	☒	☐	☐
☐	carModelId	int	11		☐	☒	☐	☐	☐	☐
☐	status	int	11		☐	☒	☐	☐	☐	☐
☐	year	int	11		☐	☒	☐	☐	☐	☐
☐	licensePlate	varchar	255		☐	☒	☐	☐	☐	☐

Слика 50 Табела Car

☐	Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update
☐	carModelId	int	11		☒	☒	☐	☒	☐	☐
☐	make	varchar	255		☐	☒	☐	☐	☐	☐
☐	name	varchar	255		☐	☒	☐	☐	☐	☐
☐	engineDescription	varchar	255		☐	☐	☐	☐	☐	☐
☐	category	enum	'EKONOMIK',		☐	☒	☐	☐	☐	☐
☐	seats	int	11		☐	☒	☐	☐	☐	☐
☐	transmission	enum	'AUTOMATIK'		☐	☒	☐	☐	☐	☐
☐	fuelType	enum	'Benzin', 'd		☐	☒	☐	☐	☐	☐
☐	doors	int	11		☐	☒	☐	☐	☐	☐
☐	luggageCapacity	int	11		☐	☒	☐	☐	☐	☐
☐	hasAc	bit	1		☐	☒	☐	☐	☐	☐
☐	picture_url	varchar	512		☐	☐	☐	☐	☐	☐
☐	logo_url	varchar	512		☐	☐	☐	☐	☐	☐
☐	price_tier1to30	double			☐	☒	☐	☐	☐	☐
☐	price_tier1to3	double			☐	☒	☐	☐	☐	☐
☐	price_tier31to45	double			☐	☒	☐	☐	☐	☐
☐	price_tier46plus	double			☐	☒	☐	☐	☐	☐
☐	price_tier4to7	double			☐	☒	☐	☐	☐	☐
☐	price_tier8to15	double			☐	☒	☐	☐	☐	☐

Слика 51 Табела CarModel

☐	Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update
☐	cityId	int	11		☒	☒	☐	☒	☐	☐
☐	name	varchar	255		☐	☒	☐	☐	☐	☐

Слика 52 Табела City

☐	Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update
☐	jmbg	varchar	255		☒	☒	☐	☐	☐	☐
☐	cityId	int	11		☐	☒	☐	☐	☐	☐
☐	age	int	11		☐	☒	☐	☐	☐	☐
☐	mobile	varchar	255		☐	☒	☐	☐	☐	☐
☐	name	varchar	255		☐	☒	☐	☐	☐	☐
☐	password	varchar	255		☐	☐	☐	☐	☐	☐
☐	surname	varchar	255		☐	☒	☐	☐	☐	☐
☐	username	varchar	255		☐	☐	☐	☐	☐	☐

Слика 53 Табела Client

☐	Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update
☐	userId	int	11		☒	☒	☐	☒	☐	☐
☐	name	varchar	255		☐	☒	☐	☐	☐	☐
☐	password	varchar	255		☐	☒	☐	☐	☐	☐
☐	surname	varchar	255		☐	☒	☐	☐	☐	☐
☐	username	varchar	255		☐	☒	☐	☐	☐	☐

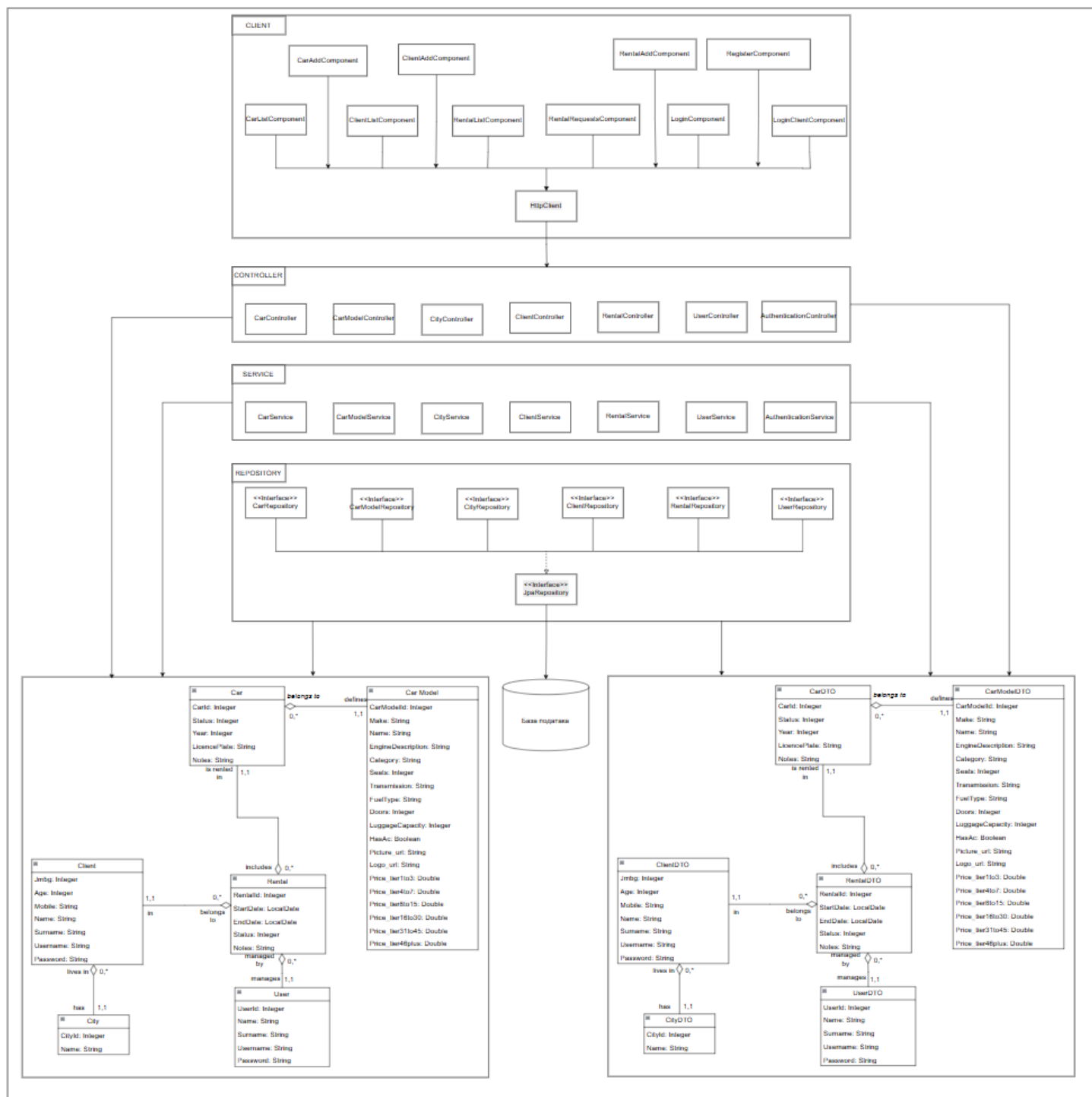
Слика 54 Табела User

☐	Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update
☐	rentalId	int	11		☒	☒	☐	☒	☐	☐
☐	carId	int	11		☐	☒	☐	☐	☐	☐
☐	userId	int	11		☐	☐	☐	☐	☐	☐
☐	clientId	varchar	255		☐	☒	☐	☐	☐	☐
☐	end_date	date			☐	☒	☐	☐	☐	☐
☐	start_date	date			☐	☒	☐	☐	☐	☐
☐	status	int	11		☐	☒	☐	☐	☐	☐
☐	notes	varchar	255		☐	☐	☐	☐	☐	☐

Слика 55 Табела Rental

3.3.7. Конечна архитектура софтверског система

Конечна архитектура софтверског система приказује структуру и начин организације клијентске и серверске апликације. Она описује ток података и комуникацију између свих слојева током процеса изнајмљивања возила. Приказана је на следећој слици.



Слика 56 Конечна архитектура софтверског система

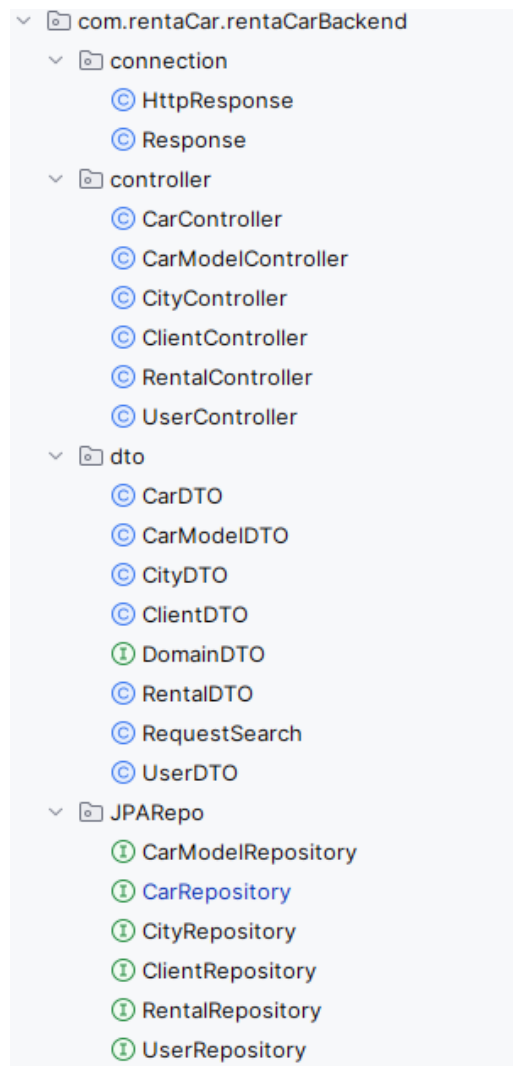
3.4.ИМПЛЕМЕНТАЦИЈА

У оквиру ове фазе, компоненте добијене током фазе пројектовања се имплементирају користећи специфичне технологије [6].

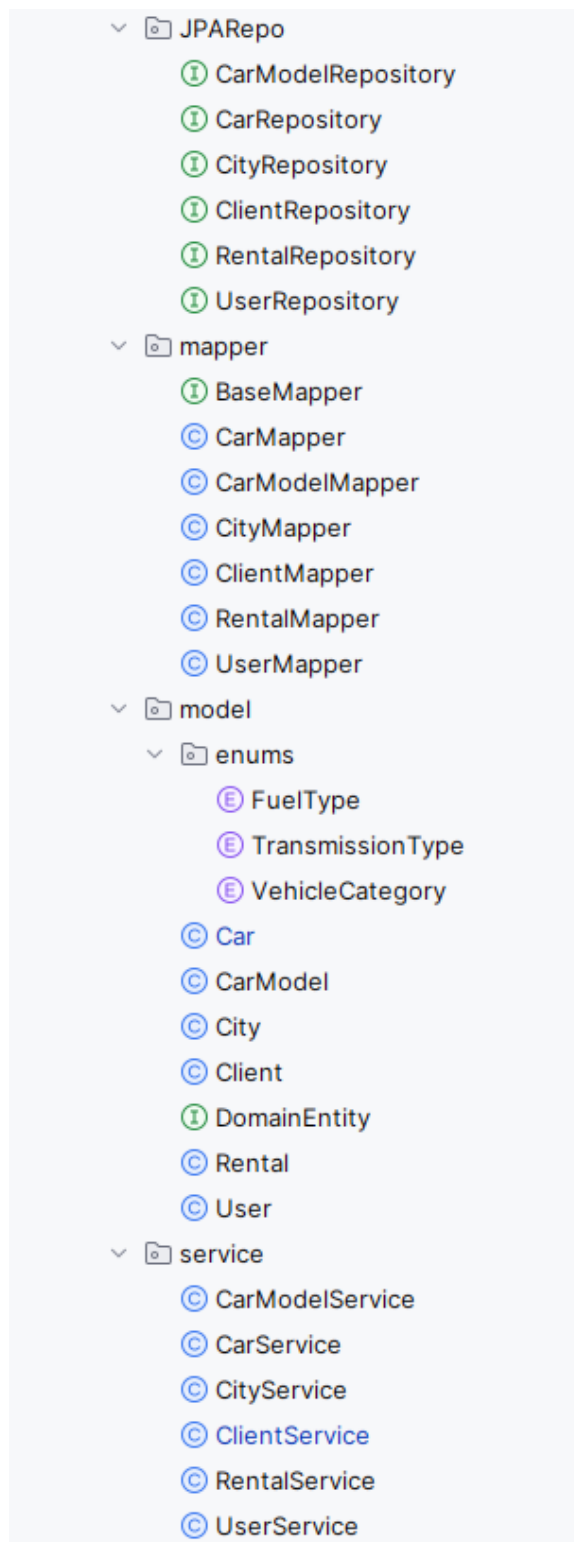
Серверска апликација развијена је у програмском језику *Java* уз коришћење *Spring Boot* развојног оквира у развојном окружењу *IntelliJ IDEA*. Њене улоге су да обради захтеве клијената, извршава пословну логику и управља подацима у релационој бази *MySQL*.

Клијентска апликација реализована је у програмском језику *TypeScript* уз помоћ *Angular* развојног оквира, у развојном окружењу *WebStorm*. Њена улога је да обезбеди кориснички интерфејс и *RESTful API* комуникацију са сервером.

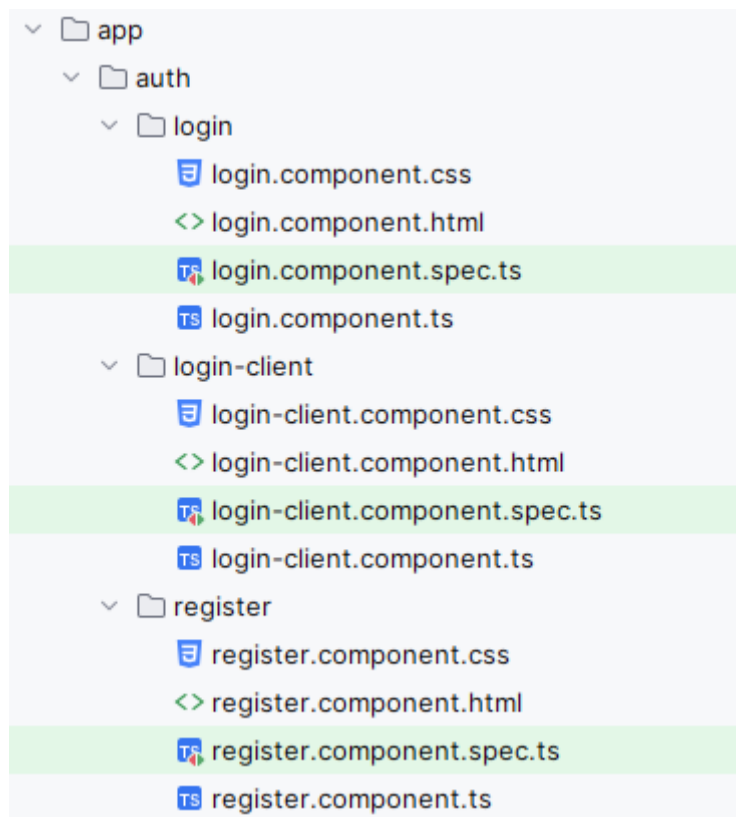
На основу архитектуре софтверског система имплементиране су следеће софтверске класе које представљају основну структуру серверске апликације. Свака класа има јасно дефинисану улогу у систему [6].



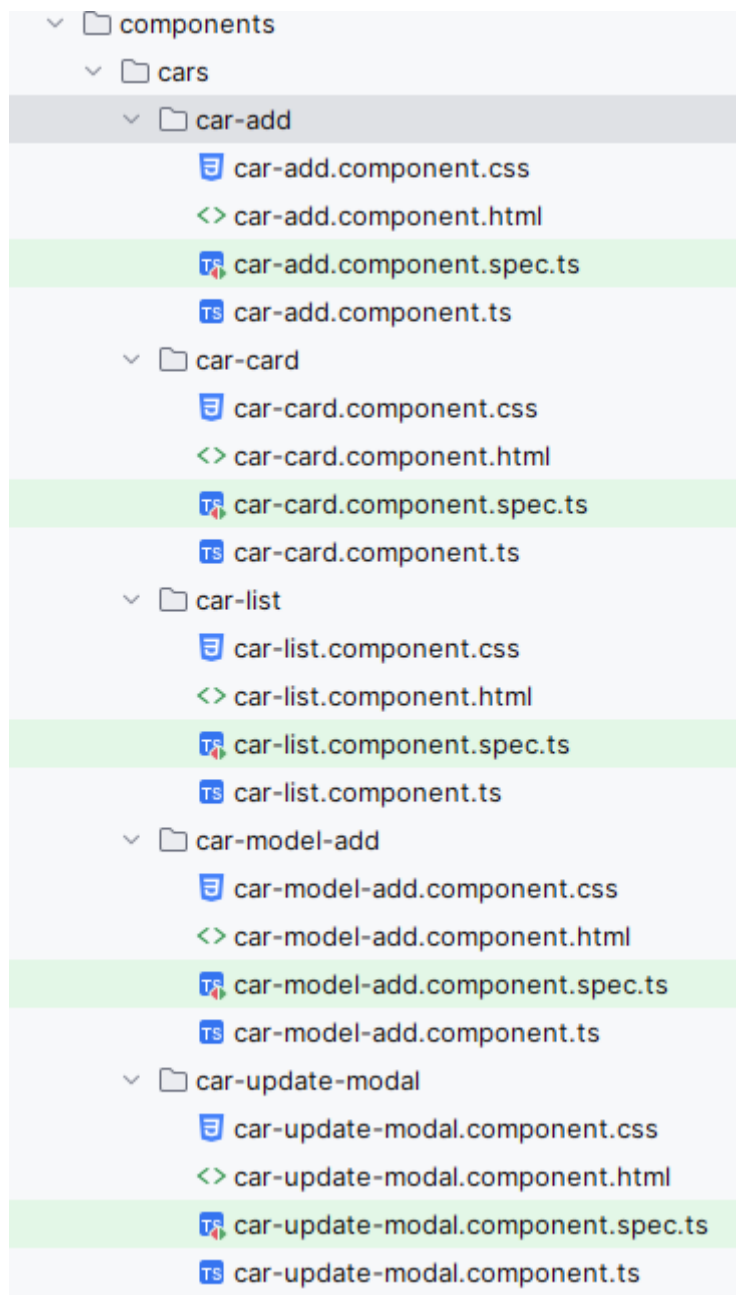
*Слика 57 Пакети и класе серверске апликације
(први део)*



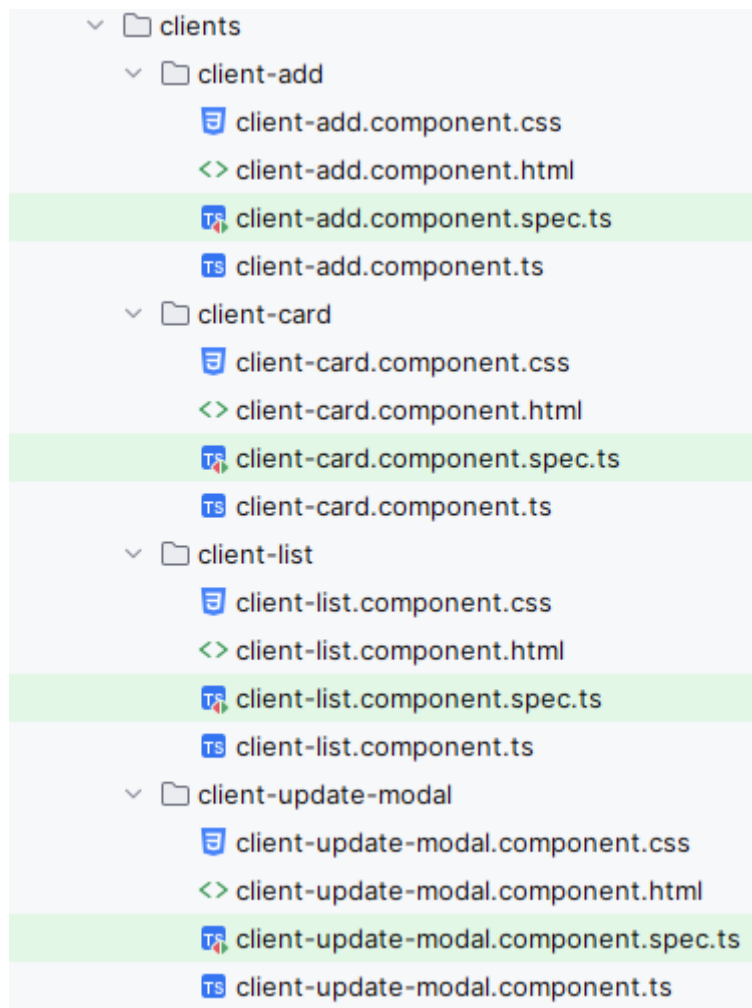
*Слика 58 Пакети и класе серверске
апликације (други део)*



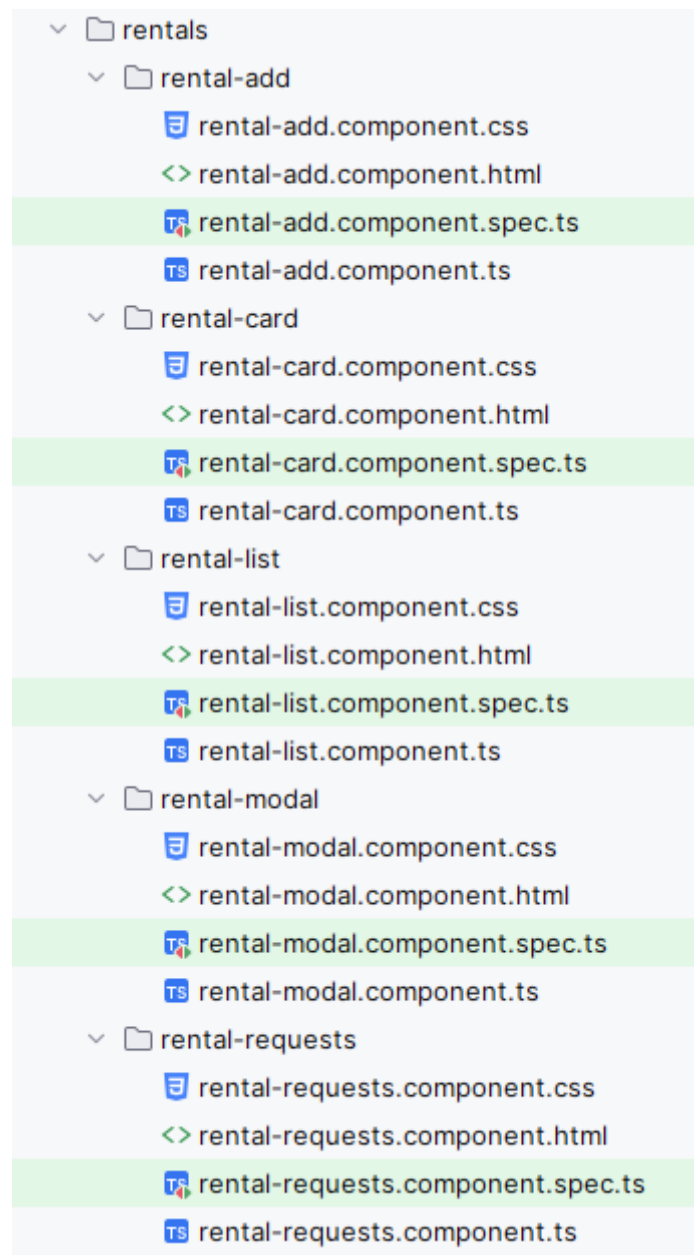
Слика 59 Пакети и класе клијентске апликације за аутентификацију (први део)



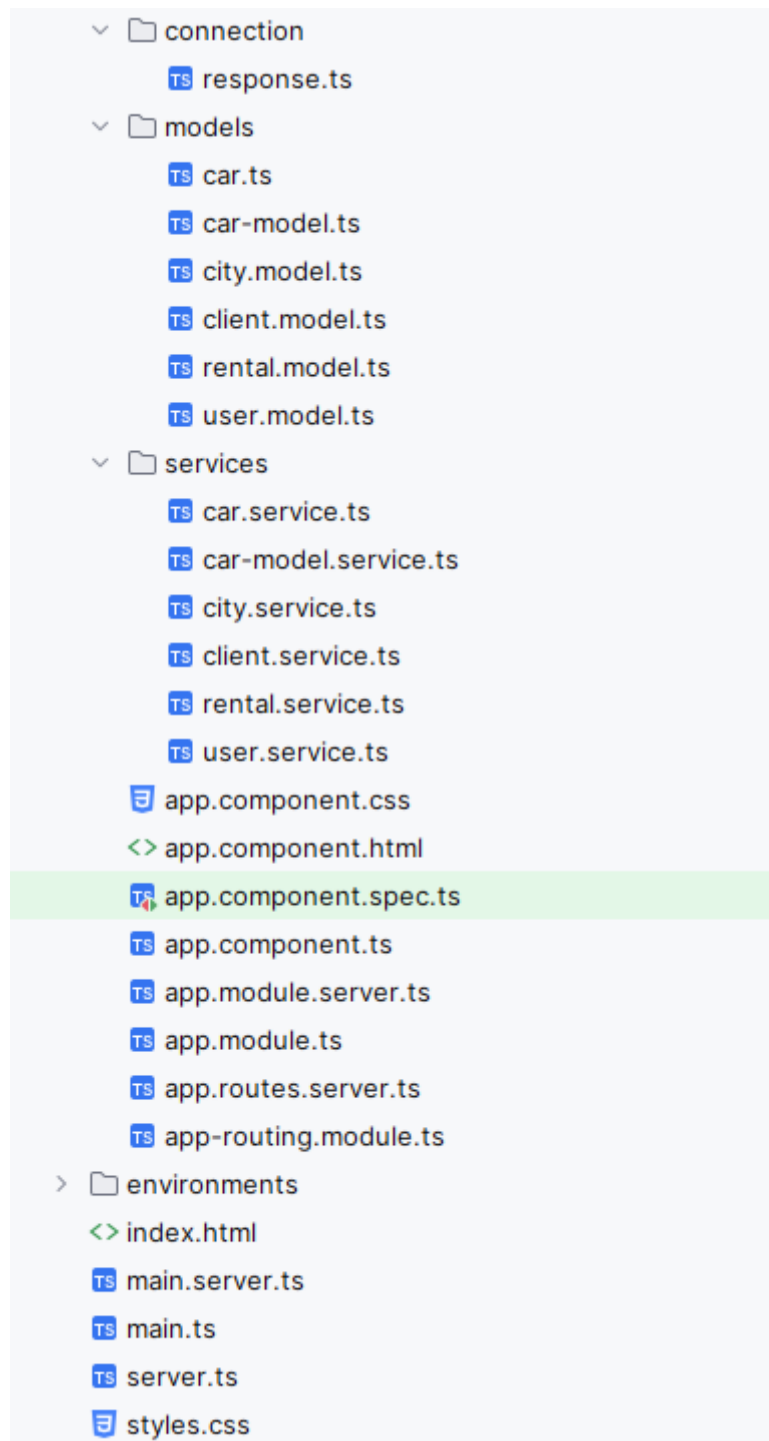
Слика 60 Пакети и класе компонената клијентске апликације (први део)



*Слика 61 Пакети и класе компонената клијентске
апликације (други део)*



Слика 62 Пакети и класе компонената клијентске апликације (трећи део)



Слика 63 Пакети и класе компонената клијентске апликације (четврти део)

```

import java.util.Map;
@CrossOrigin("http://localhost:4200")
@RestController
@RequestMapping("/cars")
public class CarController {
    private final CarService carService;
    public CarController(CarService carService) {
        this.carService = carService;
    }
    @PostMapping("/add")
    public ResponseEntity<Response> addCar(@RequestBody CardTO car) {
        String result = carService.addCar(car);
        HttpStatus status = result.equals("Successfully created a
car!") ? HttpStatus.OK : HttpStatus.BAD_REQUEST;
        return ResponseEntity.status(status)
            .body(HttpResponse.getResponseWithData(result,
Map.of("value", result), status));
    }
    @GetMapping("/all")
    public ResponseEntity<Response> getAll() {
        return ResponseEntity.ok(
            HttpResponse.getResponseWithData(
                "Successfully found cars!",
                Map.of("values", carService.getAll()),
                HttpStatus.OK));
    }
    @GetMapping("/search/{searchRequest}")
    public ResponseEntity<Response>
searchCar(@PathVariable("searchRequest") String searchRequest) {
        return ResponseEntity.ok(
            HttpResponse.getResponseWithData(
                "Searched cars:",
                Map.of("values",
carService.searchCars(searchRequest)),
                HttpStatus.OK));
    }
    @PostMapping("/update")
    public ResponseEntity<Response> updateCar(@RequestBody CardTO car) {
        String result = carService.updateCar(car);
        HttpStatus status = result.equals("Successfully updated the

```

```

car!") ? HttpStatus.OK : HttpStatus.BAD_REQUEST;

        return ResponseEntity.status(status)
            .body(HttpResponse.getResponseWithData(result,
Map.of("value", result), status));
    }

    @GetMapping("/getCar/{carId}")
    public ResponseEntity<Response> findCarById(@PathVariable("carId")
int carId) {
        return ResponseEntity.ok(
            HttpResponse.getResponseWithData(
                "Successfully found the car!",
                Map.of("value", carService.getCar(carId,
Car)),
                HttpStatus.OK));
    }

    @GetMapping("/withStatus/{status}")
    public ResponseEntity<Response>
getWithStatus(@PathVariable("status") int status) {
        return ResponseEntity.ok(
            HttpResponse.getResponseWithData(
                "Successfully found the cars!",
                Map.of("values",
carService.getWithStatus(status)),
                HttpStatus.OK));
    }

    @PostMapping("/delete")
    public ResponseEntity<Response> deleteCar(@RequestBody int carId) {
        String result = carService.deleteCar(carId);

        HttpStatus status = result.equals("Successfully deleted the
car!") ? HttpStatus.OK : HttpStatus.BAD_REQUEST;

        return ResponseEntity.status(status)
            .body(HttpResponse.getResponseWithData(result,
Map.of("value", result), status));
    }
}

```

Listing 6 Klaca CarController

```

public class CarDTO implements Serializable {

    private int id;
    private String licensePlate;
    private int year;
    private int status;
    private String notes;
    private CarModelDTO carModel;

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getLicensePlate() { return licensePlate; }
    public void setLicensePlate(String licensePlate) { this.licensePlate =
licensePlate; }
    public int getYear() { return year; }
    public void setYear(int year) { this.year = year; }
    public int getStatus() { return status; }
    public void setStatus(int status) { this.status = status; }
    public String getNotes() { return notes; }
    public void setNotes(String notes) { this.notes = notes; }
    public CarModelDTO getCarModel() { return carModel; }
    public void setCarModel(CarModelDTO carModel) { this.carModel =
carModel; }
}

```

Listing 7 Објект за пренос података класа CarDTO (Data Transfer Object)

```

public interface CarRepository extends JpaRepository<Car, Integer> {

    List<Car> findByLicensePlateStartsWith(String licensePlate);

    List<Car> findByStatus(int status);

}

```

Listing 8 Интерфејс CarRepository

```

@Component
public class CarMapper {
    private final CarModelMapper carModelMapper;
    public CarMapper(CarModelMapper carModelMapper) {
        this.carModelMapper = carModelMapper;
    }
    public Car toDomainEntity(CarDTO carDTO, CarModel carModel) {
        if (carDTO == null) return null;
        Car car = new Car();
        if (carDTO.getId() != 0) {
            car.setId(carDTO.getId());
        }
        car.setLicensePlate(carDTO.getLicensePlate());
        car.setYear(carDTO.getYear());
        car.setStatus(carDTO.getStatus());
        car.setNotes(carDTO.getNotes());
        car.setCarModel(carModel);
        return car;
    }
    public CarDTO toDomainDTO(Car car) {
        if (car == null) return null;

        CarDTO carDTO = new CarDTO();
        carDTO.setId(car.getId());
        carDTO.setLicensePlate(car.getLicensePlate());
        carDTO.setYear(car.getYear());
        carDTO.setStatus(car.getStatus());
        carDTO.setNotes(car.getNotes());
        if (car.getCarModel() != null) {
            carDTO.setCarModel(carModelMapper.toDomainDTO(car.getCarModel()));
        }
        return carDTO;
    }
}

```

Listing 9 Κλάσα CarMapper

```

@Service
@Transactional
public class CarService {
    private final CarRepository carRepository;
    private final CarMapper carMapper;
    private final CarModelRepository carModelRepository;
    public CarService(CarRepository carRepository, CarMapper carMapper,
CarModelRepository carModelRepository) {
        this.carRepository = carRepository;
        this.carMapper = carMapper;
        this.carModelRepository = carModelRepository;
    }
    public List<CarDTO> getAll() {
        List<Car> cars = carRepository.findAll();
        return cars.stream()
            .map(carMapper::toDomainDTO)
            .collect(Collectors.toList());
    }
    public List<CarDTO> getWithStatus(int status) {
        List<Car> cars = carRepository.findByStatus(status);
        return cars.stream()
            .map(carMapper::toDomainDTO)
            .collect(Collectors.toList());
    }
    public List<CarDTO> searchCars(String searchParam) {
        List<Car> cars =
carRepository.findByLicensePlateStartsWith(searchParam);
        return cars.stream()
            .map(carMapper::toDomainDTO)
            .collect(Collectors.toList());
    }
    public String addCar(CarDTO carDTO) {
        try {
            CarModel carModel =
carModelRepository.findById(carDTO.getCarModel().getId())
                .orElseThrow(() -> new RuntimeException("CarModel not
found with id: " + carDTO.getCarModel().getId()));
            Car car = carMapper.toDomainEntity(carDTO, carModel);
            carRepository.save(car);
            return "Successfully created a car!";
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
}

```

```

        return "Cannot create this car! Error: " + ex.getMessage();
    }
}

public String deleteCar(int carId) {
    try {
        if (carRepository.existsById(carId)) {
            carRepository.deleteById(carId);
            return "Successfully deleted the car!";
        } else {
            return "Car with id " + carId + " not found.";
        }
    } catch (Exception ex) {
        ex.printStackTrace();
        return "Cannot delete this car! Error: " + ex.getMessage();
    }
}

public CarDTO getCar(int carId) {
    Car carEntity = carRepository.findById(carId).orElse(null);
    return carMapper.toDomainDTO(carEntity);
}

public String updateCar(CarDTO carDTO) {
    try {
        carRepository.findById(carDTO.getId())
            .orElseThrow(() -> new RuntimeException("Car not found
with id: " + carDTO.getId()));
        CarModel carModel =
carModelRepository.findById(carDTO.getCarModel().getId())
            .orElseThrow(() -> new RuntimeException("CarModel not
found with id: " + carDTO.getCarModel().getId()));
        Car car = carMapper.toDomainEntity(carDTO, carModel);
        carRepository.save(car);
        return "Successfully updated the car!";
    } catch (Exception ex) {
        ex.printStackTrace();
        return "Cannot update this car! Error: " + ex.getMessage();
    }
}
}

```

Listing 10 Класа CarService

4. ТЕСТИРАЊЕ

Тестирање представља пету фазу у животном циклусу развоја софтвера [6]. Основна сврха тестирања је провера исправности софтверске апликације, да је у складу са техничким захтевима дефинисаним током фазе пројектовања и да задовољава потребе крајњих корисника на ефикасан и поуздан начин [27].

Процес тестирања се обично спроводи кроз две фазе[27]:

- Верификација – проверава да ли софтверски систем ради на начин дефинисан у фази анализе.
- Валидација - проверава да ли софтверски систем ради оно што корисник очекује да ради.

Постоје различити типови тестирања, укључујући мануелно и аутоматизовано тестирање. Мануелно тестирање се спроводи без употребе алата за тестирање, док се аутоматизовано ослања на специјализоване алате за тестирање [27].

У зависности од фазе развоја, тестирање се спроводи на више нивоа, и сваки ниво је фокусиран на одређену фазу [27]:

- Јединично тестирање (енгл. *Unit Testing*) је тестирање појединачних компоненти или јединица како би се утврдило да је исправна за даље коришћење.
- Интеграционо тестирање (енгл. *Integration Testing*) је тестирање интеракција две или више компоненти које су већ јединично тестиране.
- Системско тестирање (енгл. *System Testing*) је тестирање комплетне и интегрисане апликације као целине, симулира рад крајњег корисника и проверавају се сви случајеви коришћења од почетка до краја.
- Прихватно тестирање (енгл. *Acceptance Testing*) осигурава да софтверски систем испуњава захтеве корисника пре него што се испоручи.

У оквиру овог пројекта, фокус је био на мануелном тестирању, где је свака компонента апликације детаљно проверена. Тестирање је спроведено из две улоге. Прва улога је улога крајњег корисника, где су све акције извршаване преко клијентске *Angular* апликације. Овај приступ, проверава функционалност система без познавања његове унутрашње структуре. Друга улога је техничка, где је коришћена *Postman* платформа за директно тестирање сваке тачке на *REST* архитектонском стилу. На овај начин су тестирани сви гранични случајеви и сценарији за обраду грешака, уз велики број различитих улазних података.

5. ЗАКЉУЧАК

Овај рад је обухватио целокупан процес развоја софтверског система за изнајмљивање возила, од идејне фазе до конкретне имплементације. У раду су детаљно представљени теоријски и практични аспекти изградње модерног веб решења, са посебним освртом на коришћене технологије и примењену методу развоја. Систем је реализован употребом програмских језика *Java* и *TypeScript*, уз коришћење *Spring Boot* развојног оквира за серверску страну и *Angular* развојног оквира за клијентску страну апликације. Целокупан развојни циклус је вођен принципима упрошћене Ларманове методе, чија је примена детаљно објашњена у трећем поглављу, док су коришћене технологије описане у другом.

Током израде рада, детаљно је обрађен процес развоја веб апликације са фокусом на *Spring Boot* развојни оквир уз примену вишеслојне архитектуре. Истражен је и примењен приступ развоју који се ослања на итеративно-инкрементални модел, где су као кључни артефакти коришћени дијаграми случајева коришћења, секвенци и класа за моделовање система.

Развијена апликација омогућава кориснику да на једноставан и прегледан начин претражује доступну флоту возила. Имплементирана је функционалност филтрирања, као и динамичког израчунавања укупне цене на основу изабраног периода изнајмљивања. Након регистрације и пријаве, корисник може да пошаље захтев за резервацију и да прати његов статус. Да би се заокружио пословни процес, дефинисана је и улога администратора, којем су на располагању алати за управљање целокупном флотом возила, администрацију клијената и, што је најважније, обраду пристиглих захтева за изнајмљивање.

Клијентска страна апликације је креирана са циљем да пружи интуитивно и ефикасно корисничко искуство. Обезбеђена је јасна навигација и логичка повезаност компоненти, што резултира интерфејсом који је једноставан за коришћење и визуелно складан. Тиме је постигнуто ефикасно софтверско решење за конкретан доменски проблем, уз висок ниво корисничког задовољства.

Потенцијал за даље унапређење овог система је велики. Као први следећи корак, намеће се имплементација система за безбедно онлајн плаћање, чиме би се корисницима омогућило да одмах по резервацији изврше уплату и тиме гарантују изнајмљивање. Функционалност би се могла додатно унапредити увођењем напредних филтера, по цени, врсти мењача, типу горива, као и визуелног календара доступности за свако возило. Увођењем система за остављање оцена и рецензија, агенција би добила вредне повратне информације, док би будући корисници имали више поверења у услугу. Коначно, систем би се могао проширити и модулом за промоције и програме верности (енгл. *loyalty*) за сталне клијенте. Захваљујући модуларној архитектури и праћењу добрих пракси током развоја, додавање ових и сличних функционалности у будућности не би представљало велики технички изазов.

6. ЛИТЕРАТУРА

1. Manasa R (2024). The Evolution of Human Travel: From Walking to AI-Powered Vehicles and Beyond. <https://www.isme.in/the-evolution-of-human-travel-from-walking-to-ai-powered-vehicles-and-beyond-manasa-r/> [приступљено: 23. октобар 2025.]
2. Cromer, George C. , Cromer, Orville C., Foster, Christopher G., Purdy, Ken W. (2025). History of automobile. Automobile. Encyclopedia Britannica. <https://www.britannica.com/technology/automobile/History-of-the-automobile> [приступљено: 23. октобар 2025.]
3. Rae, John B., Binder, Alan K. (2025). Automotive industry. Encyclopedia Britannica. <https://www.britannica.com/technology/automotive-industry> [приступљено: 23. октобар 2025.]
4. Jasiński, Jakub (2023). Travel cart. <https://imperiumromanum.pl/en/curiosities/travel-cart/> [приступљено: 23. октобар 2025.]
5. Clark, Arnold. (2020). History of car rental: what has changed? <https://www.arnoldclarkrental.com/latest-news/275-history-of-car-rental-what-has-changed> [приступљено: 23. октобар 2025.]
6. др Влајић, С. (2020). Пројектовање софтвера – скрипта, Факултет организационих наука. https://www.researchgate.net/publication/344459754_PROJEKTOVANE_SOFTVER_A_SKRIPTA_-_RADNI_MATERIJAL_-_2020 [приступљено: 9. октобар 2025.]
7. GeeksForGeeks. (2023). What is Single Page Application? <https://www.geeksforgeeks.org/javascript/what-is-single-page-application/> [приступљено: 9. октобар 2025.]
8. GeeksForGeeks. (2025). What is an API (Application Programming Interface). <https://www.geeksforgeeks.org/software-testing/what-is-an-api/> [приступљено: 9. октобар 2025.]

9. Evans, Benjamin J., Flanagan, David. (2014). Java in a Nutshell. https://www.r-5.org/files/books/computers/languages/java/main/Benjamin_Evans_David_Flanagan-Java_in_a_Nutshell_6th_ed-EN.pdf<https://www.geeksforgeeks.org/java/java-features/> [приступљено: 9. октобар 2025.]
10. Kochiva (2025). 8 Features of Java Programming Language . <https://kochiva.com/blog/features-of-java/> [приступљено: 10. октобар 2025.]
11. др Влајић, С. (2005) Napredni koncepti Jave i web programiranje u Javi. https://www.researchgate.net/publication/303920636_Napredni_koncepti_Jave_i_web_programiranje_u_Javi [приступљено: 23. октобар 2025.]
12. Siva, Prasad, Reddy K., Upadhyayula, Sai. (2023). Beginning Spring Boot 3 Build Dynamic Cloud-Native Java Applications and Microservices Second Edition <https://studylib.net/doc/26150383/beginning-spring-boot-3--siva-prasad-reddy-katamreddy-etc...> [приступљено: 10. октобар 2025.]
13. GeeksForGeeks. (2025). Spring Boot – Architecture <https://www.geeksforgeeks.org/springboot/spring-boot-architecture/> [приступљено: 10. октобар 2025.]
14. TutorialsPoint. Spring Boot – Introduction. https://www.tutorialspoint.com/spring_boot/spring_boot_introduction.htm [приступљено: 10. октобар 2025.]
15. IBM. What is Java Spring Boot? <https://www.ibm.com/think/topics/java-spring-boot> [приступљено: 10. октобар 2025.]
16. Gunkar, Rahun. (2025). Spring Boot – Architecture <https://www.geeksforgeeks.org/springboot/introduction-to-spring-boot/> [приступљено: 10. октобар 2025.]
17. GeeksForGeeks. (2025). SQL Tutorial <https://www.geeksforgeeks.org/sql/sql-tutorial/> [приступљено: 10. октобар 2025.]
18. GeeksForGeeks. (2025). MySQL Tutorial. <https://www.geeksforgeeks.org/mysql/mysql-tutorial/> [приступљено: 10. октобар 2025.]

19. GeeksForGeeks. (2025). Introduction to TypeScript. <https://www.geeksforgeeks.org/typescript/introduction-to-typescript/> [приступљено: 10. октобар 2025.]
20. GeeksForGeeks. (2025). Explain the Architecture Overview of Angular ? <https://www.geeksforgeeks.org/angular-js/explain-the-architecture-overview-of-angular/> [приступљено: 10. октобар 2025.]
21. Amazon. What is a RESTful API? <https://aws.amazon.com/what-is/restful-api/> [приступљено: 23. октобар 2025.]
22. GeeksForGeeks. (2023). Introduction of 3-Tier Architecture in DBMS. <https://www.geeksforgeeks.org/dbms/introduction-of-3-tier-architecture-in-dbms-set-2/> [приступљено: 23. октобар 2025.]
23. Yu, Irene. (2023). PART 2 REST API components & How to read them. <https://www.skiplevel.co/blog/part-2-rest-api-components-how-to-read-them> [приступљено: 23. октобар 2025.]
24. GeeksForGeeks. (2025). SPRING – REST CONTROLLER. <https://www.geeksforgeeks.org/advance-java/spring-rest-controller/> [приступљено: 13. октобар 2025.]
25. Mong'are, Sospeter. (2025). Business Logic vs. Application Logic in Systems Design. <https://dev.to/msnmongare/business-logic-vs-application-logic-in-systems-design-17ip> [приступљено: 13. октобар 2025.]
26. GeeksForGeeks. (2023.). Spring Boot - Spring Data JPA <https://www.geeksforgeeks.org/java/spring-boot-spring-data-jpa/> [приступљено: 13. октобар 2025.]
27. Gupta, Aditya (2023). What is Software Testing? <https://www.geeksforgeeks.org/software-testing/software-testing-basics/> [приступљено: 13. октобар 2025.]